

机器学习概述



教学内容



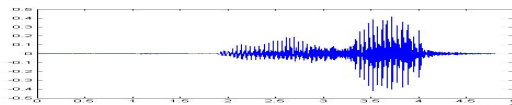
- 机器学习概念与原理
- 线性回归
 - 定义
 - 经验风险最小化(最小均方误差)
 - 结构风险最小化
 - 最大似然估计
 - 最大后验估计
- 人工神经网络
 - 前馈、卷积、循环神经网络回顾
- 网络优化与正则化

机器学习 $\approx$ 构建一个映射函数



► 语音识别

$f($



$) = \text{“你好”}$

► 图像识别

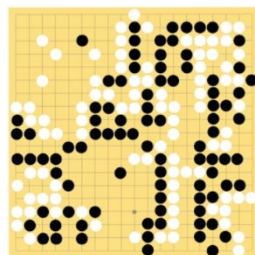
$f($



$) = \text{“猫”}$

► 围棋

$f($



$) = \text{“5-5” (落子位置)}$

► 机器翻译

$f($

“你好！”

用户输入

$) = \text{“Hello!”}$

机器

为什么要“机器学习”？



现实世界的问题都比较复杂！很难通过规则来手工实现。



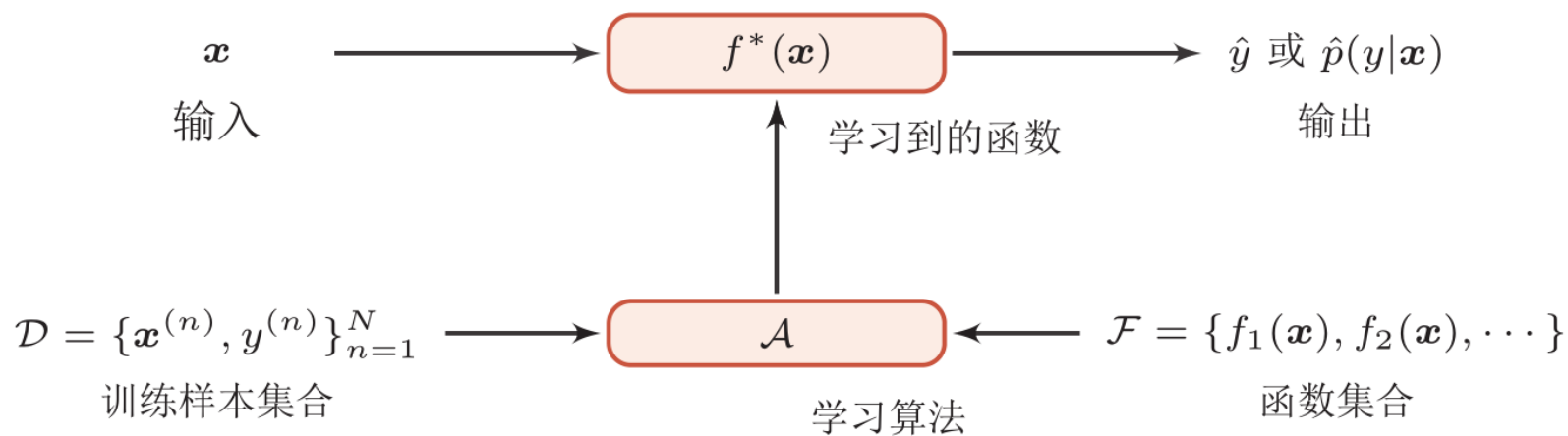
2	6	8	9	3	4	7	5	6
3	4	7	9	5	5	6	7	2
5	8	7	0	9	4	3	5	4
5	2	3	4	9	5	6	7	8

什么是机器学习？



机器学习：通过算法使得机器能从大量数据中学习规律从而对新的样本做决策。

- **规律：**决策（预测）函数



独立同分布 $p(\mathbf{x}, y)$

机器学习三要素



01 模型

- 线性方法: $f(x, \theta) = w^T x + b$

- 广义线性方法: $f(x, \theta) = w^T \phi(x) + b$

如果 $\phi(x)$ 为可学习的非线性基函数, $f(x, \theta)$ 就等价于神经网络。

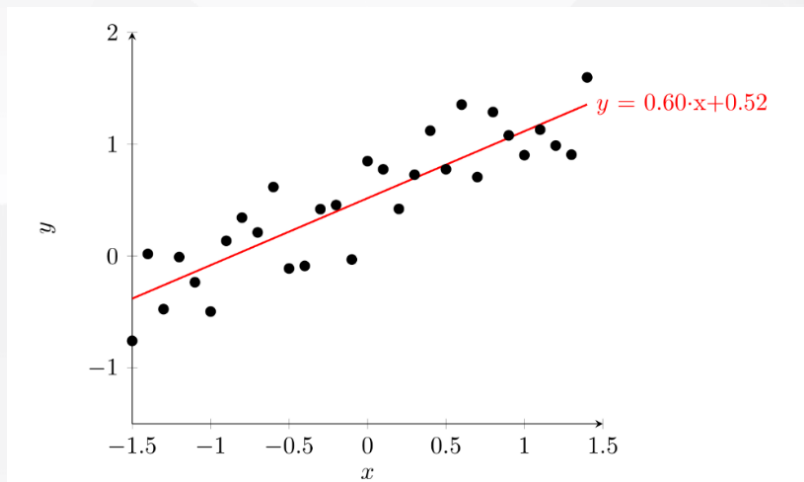
02 学习准则

- 期望风险: $R(f) = E_{(x,y) \sim p(x,y)} [L(f(x), y)]$

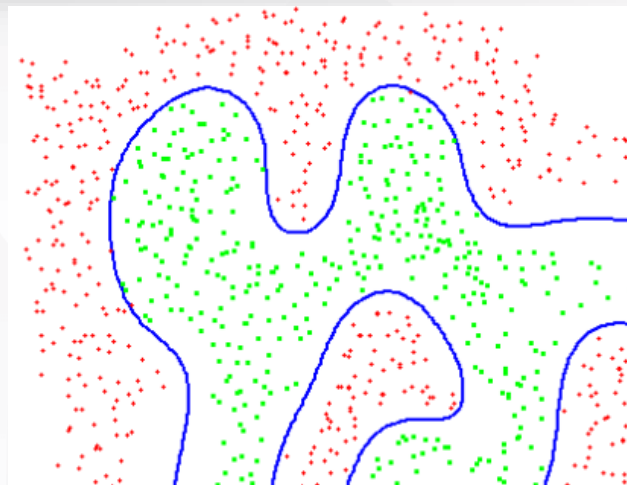
03 优化

- 梯度下降

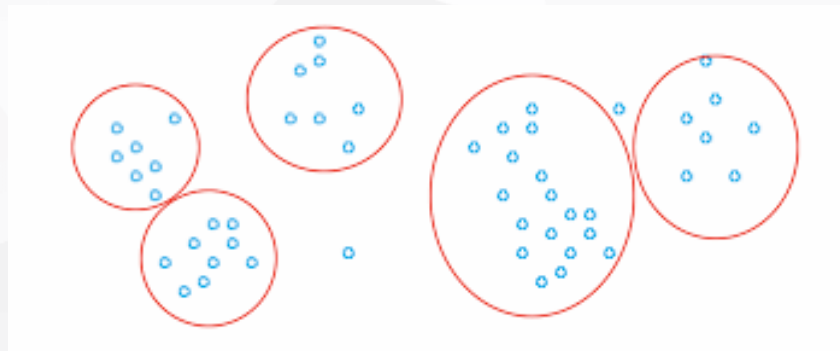
常见的机器学习问题



回归



分类



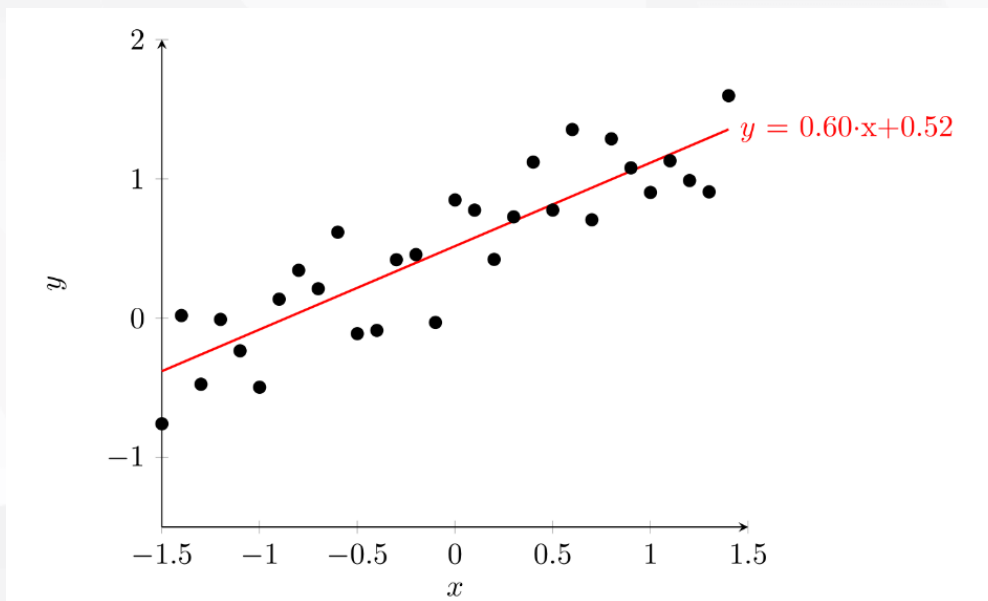
聚类

模型



- 以线性回归（Linear Regression）为例
- 模型：

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b$$



学习准则



- 损失函数

0-1损失函数

$$\mathcal{L}(y, f(x, \theta)) = \begin{cases} 0 & \text{if } y = f(x, \theta) \\ 1 & \text{if } y \neq f(x, \theta) \end{cases}$$

平方损失函数

$$\mathcal{L}(y, \hat{y}) = (y - f(x, \theta))^2$$

学习准则



- 期望风险未知，通过**经验风险**近似

训练数据： $D = \{x^{(i)}, y^{(i)}\}, i \in [1, N]$

$$R_D^{emp}(\theta) = \frac{1}{N} \sum_{n=1}^N L(y^{(n)}, f(x^{(n)}, \theta))$$

- **经验风险最小化**

在选择合适的风险函数后，我们寻找一个参数 θ^* ，使得经验风险函数最小化。

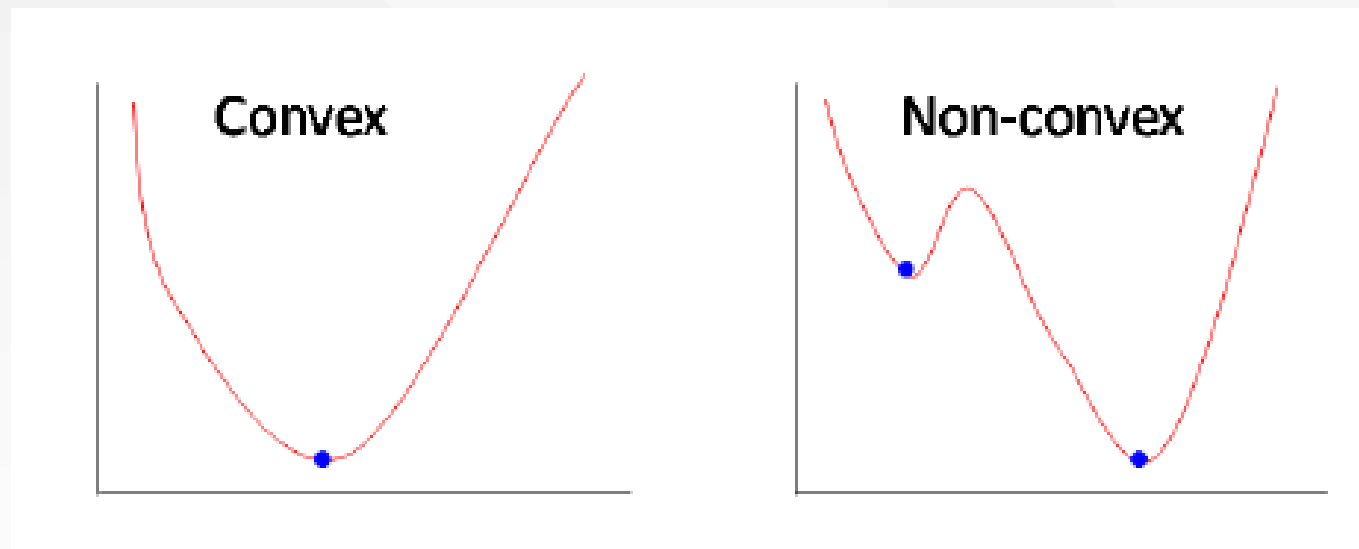
$$\theta^* = \arg \min R_D^{emp}(\theta)$$

机器学习问题  最优化问题

最优化问题



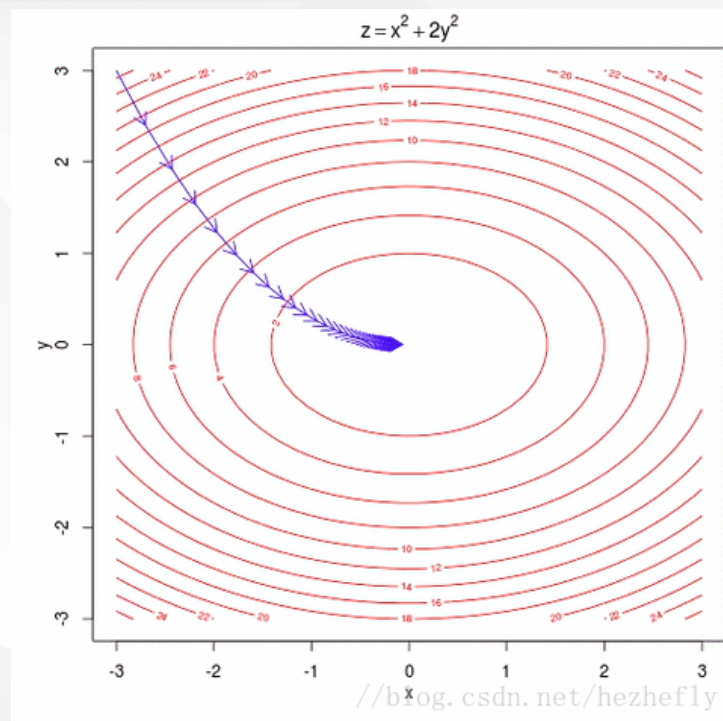
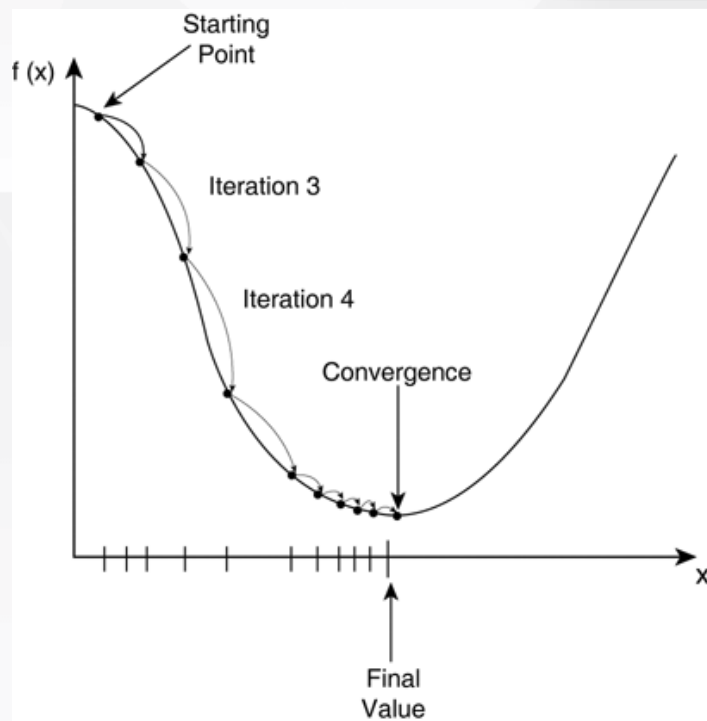
机器学习问题 $\xrightarrow{\text{转化成}}$ 最优化问题



$$\min_{\mathbf{x}} f(\mathbf{x})$$

梯度下降法

Gradient Descent



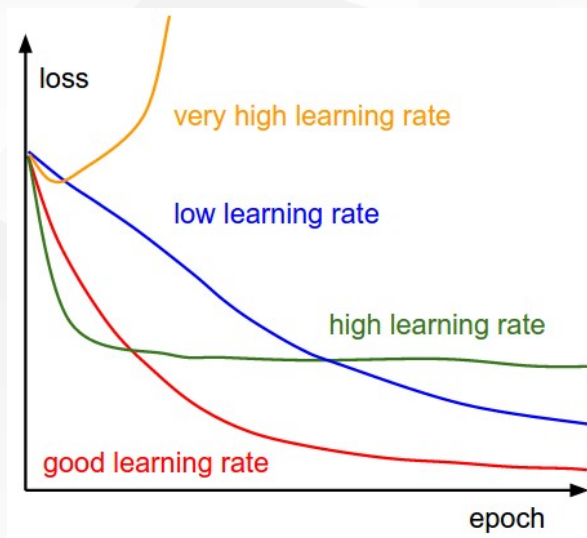
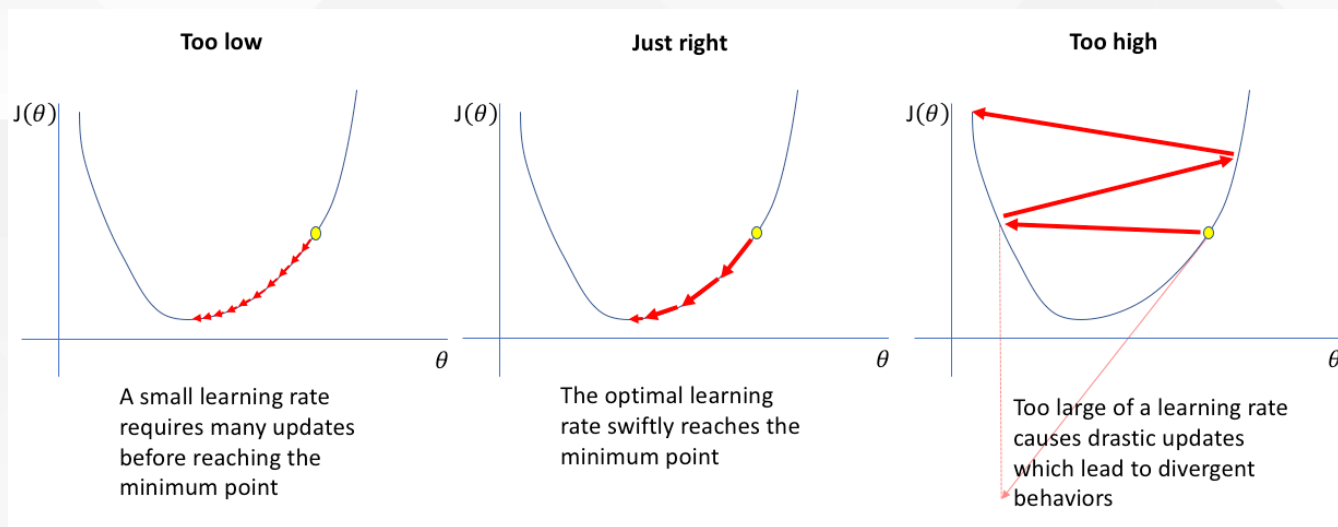
$$\begin{aligned}\theta_{t+1} &= \theta_t - \alpha \frac{\partial \mathcal{R}(\theta)}{\partial \theta_t} \\ &= \theta_t - \alpha \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}(\theta_t; x^{(i)}, y^{(i)})}{\partial \theta}.\end{aligned}$$

搜索步长 α 中也叫作**学习率**（Learning Rate）

学习率



学习率是十分重要的超参数！



随机梯度下降法



- 随机梯度下降法（Stochastic Gradient Descent, SGD）也叫增量梯度下降，每个样本都进行更新

$$\theta_{t+1} = \theta_t - \alpha \frac{\partial \mathcal{L}(\theta_t; x^{(t)}, y^{(t)})}{\partial \theta},$$

- 更新小批量（Mini-Batch）随机梯度下降法

随机梯度下降法



算法 2.1: 随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α

```
1 随机初始化  $\theta$ ;  
2 repeat  
3   对训练集  $\mathcal{D}$  中的样本随机重排序;  
4   for  $n = 1 \cdots N$  do  
5     从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;  
      // 更新参数  
6      $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; \mathbf{x}^{(n)}, y^{(n)})}{\partial \theta}$ ;  
7   end  
8 until 模型  $f(\mathbf{x}; \theta)$  在验证集  $\mathcal{V}$  上的错误率不再下降;  
   输出:  $\theta$ 
```



Why?

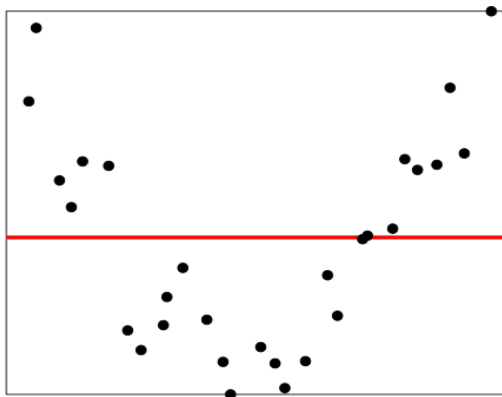
机器学习=优化?



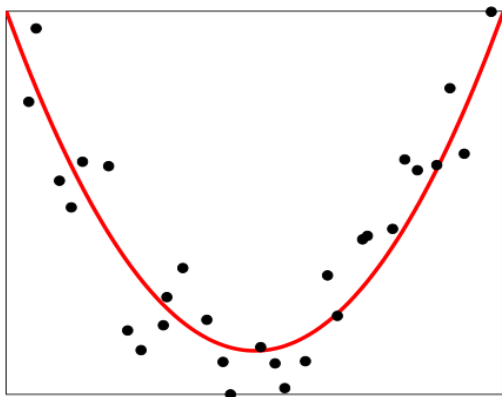
机器学习 = 优化

NO!!!

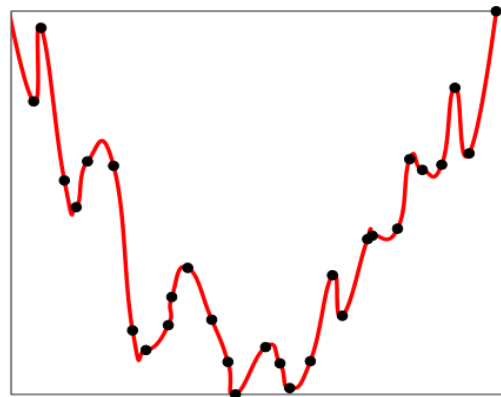
欠拟合



正常



过拟合



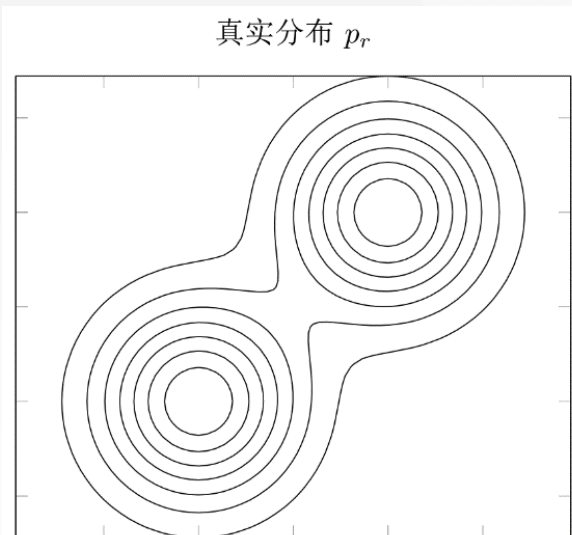
过拟合：经验风险最小化原则很容易导致模型在训练集上错误率很低，但是在未知数据上错误率很高。

泛化错误



期望风险

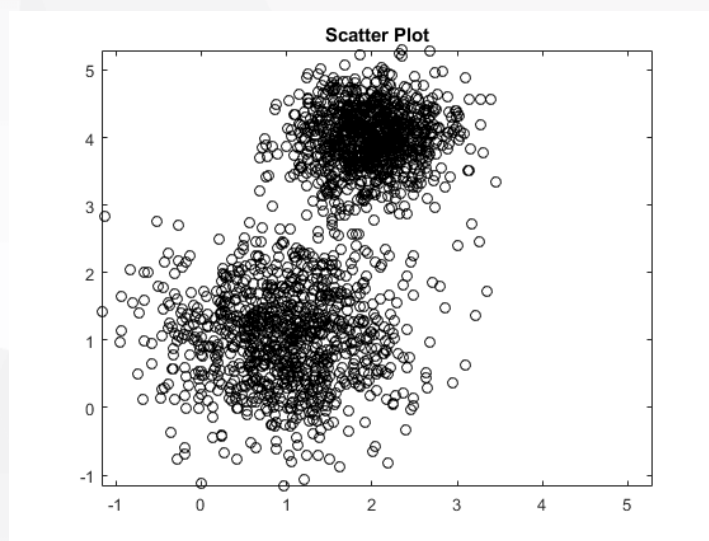
$$R(f) = E_{(x,y) \sim p(x,y)} [L(f(x), y)]$$



$\neq$

经验风险

$$R_D^{emp}(\theta) = \frac{1}{N} \sum_{n=1}^N L(y^{(n)}, f(x^{(n)}, \theta))$$



$$\mathcal{G}_D(f) = \mathcal{R}(f) - \mathcal{R}_D^{emp}(f)$$

泛化错误

如何减少泛化错误？



优化

经验风险最小

正则化

降低模型复杂度



正则化 (regularization)



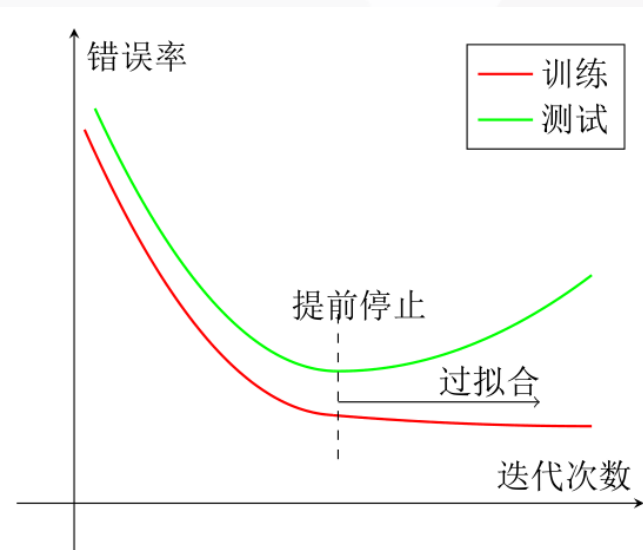
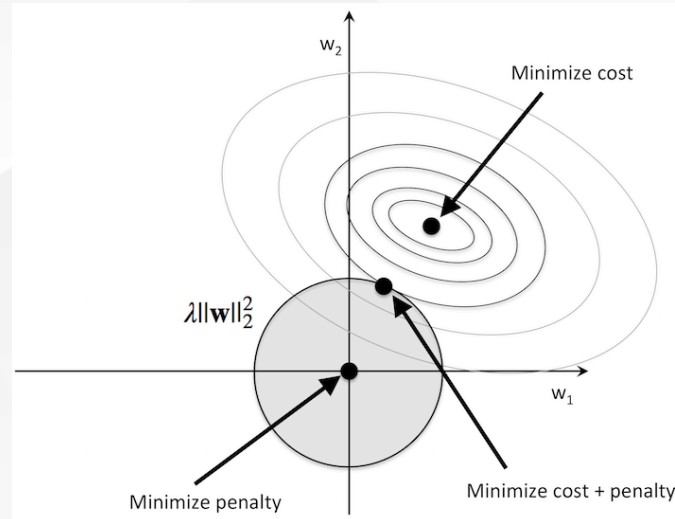
所有损害优化的方法都是正则化

增加优化约束

L1/L2约束
数据增强

干扰优化过程

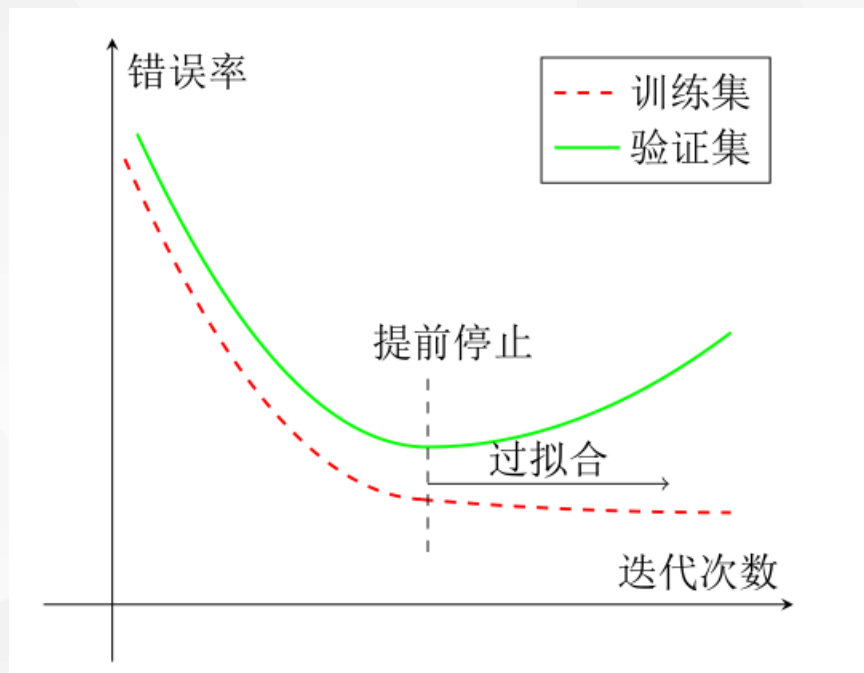
权重衰减
提前停止
随机梯度下降



提前停止



- 我们使用一个验证集（Validation Dataset）来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降，就停止迭代。



线性回归 (Linear Regression)



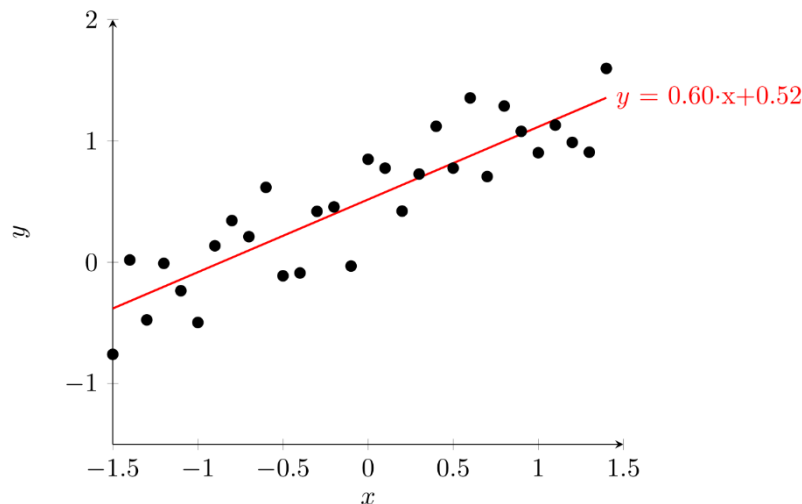
- 模型: $f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b$
- 增广权重向量和增广特征向量

$$\hat{\mathbf{x}} = \mathbf{x} \oplus 1 \triangleq \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} x_1 \\ \vdots \\ x_k \\ 1 \end{bmatrix},$$

$$\hat{\mathbf{w}} = \mathbf{w} \oplus b \triangleq \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} = \begin{bmatrix} w_1 \\ \vdots \\ w_k \\ b \end{bmatrix},$$



$$f(\mathbf{x}; \hat{\mathbf{w}}) = \hat{\mathbf{w}}^T \hat{\mathbf{x}},$$



优化方法



- 经验风险最小化（最小二乘法）
- 结构风险最小化（岭回归）
- 最大似然估计
- 最大后验估计

矩阵微积分



- 标量关于向量的偏导数

$$\frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_M} \right]^\top$$

- 向量关于向量的偏导数

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_N}{\partial x_1} \\ \vdots & \vdots & \vdots \\ \frac{\partial y_1}{\partial x_M} & \dots & \frac{\partial y_N}{\partial x_M} \end{bmatrix}$$

- 向量函数及其导数

$$\begin{aligned} \frac{\partial \mathbf{x}}{\partial \mathbf{x}} &= \mathbf{I}, \\ \frac{\partial \mathbf{A}\mathbf{x}}{\partial \mathbf{x}} &= \mathbf{A}^\top, \\ \frac{\partial \mathbf{x}^\top \mathbf{A}}{\partial \mathbf{x}} &= \mathbf{A} \end{aligned}$$

经验风险最小化



- 模型

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \mathbf{x}$$

- 学习准则

$$\begin{aligned}\mathcal{R}(\mathbf{w}) &= \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}; \mathbf{w})) \\ &= \frac{1}{2} \sum_{n=1}^N \left(y^{(n)} - \mathbf{w}^T \mathbf{x}^{(n)} \right)^2 \\ &= \frac{1}{2} \|\mathbf{y} - X^T \mathbf{w}\|^2,\end{aligned}$$

- 优化

$$\begin{aligned}\frac{\partial}{\partial \mathbf{w}} \mathcal{R}(\mathbf{w}) &= 0 \quad \frac{\partial \mathcal{R}(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{2} \frac{\partial \|\mathbf{y} - X^T \mathbf{w}\|^2}{\partial \mathbf{w}} \\ &= -X(\mathbf{y} - X^T \mathbf{w}),\end{aligned}$$

结构风险最小化



- 结构风险最小化准则

$$\mathcal{R}(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}^\top \mathbf{w}\|^2 + \frac{1}{2} \lambda \|\mathbf{w}\|^2,$$

- 得到

$$\mathbf{w}^* = (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I})^{-1} \mathbf{X}\mathbf{y},$$

岭回归 (Ridge Regression)

关于概率的一些基本概念



概率 (Probability)

- 一个随机事件发生的可能性大小，为0到1之间的实数。

随机变量 (Random Variable)

- 比如随机掷一个骰子，得到的点数就可以看成一个随机变量 X ，其取值为 $\{1, 2, 3, 4, 5, 6\}$ 。

概率分布 (Probability Distribution)

- 一个随机变量 X 取每种可能值的概率

$$P(X = x_i) = p(x_i), \quad \forall i \in \{1, \dots, n\}.$$

并且满足

$$\sum_{i=1}^n p(x_i) = 1,$$

$$p(x_i) \geq 0, \quad \forall i \in \{1, \dots, n\}.$$

概率的一些基本概念



伯努利分布 (Bernoulli Distribution)

- 在一次试验中，事件A出现的概率为 μ ，不出现的概率为 $1 - \mu$ 。若用变量X表示事件A出现的次数，则X的取值为0和1，其相应的分布为

$$p(x) = \mu^x (1 - \mu)^{(1-x)}$$

二项分布 (Binomial Distribution)

- 在n次伯努利分布中，若以变量X表示事件A出现的次数，则X的取值为 $\{0, \dots, n\}$ ，其相应的分布

$$P(X = k) = \binom{n}{k} \mu^k (1 - \mu)^{n-k}, \quad k = 1 \dots, n$$

二项式系数，表示从n个元素中取出k个元素而不考虑其顺序的组合的总数。

概率的一些基本概念



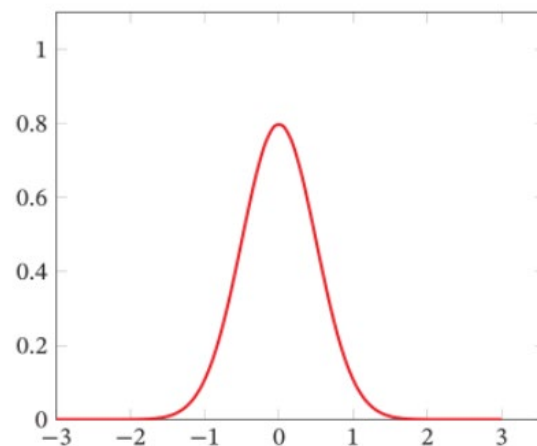
连续随机变量Y的概率分布一般用概率密度函数（Probability Density Function, PDF） $p(x)$ 来描述。

$$\int_{-\infty}^{+\infty} p(x)dx = 1$$

高斯分布（Gaussian Distribution）

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$



概率的一些基本概念



条件概率 (Conditional Probability)

- 对于离散随机向量 (X, Y) ，已知 $X = x$ 的条件下，随机变量 $Y = y$ 的条件概率为：

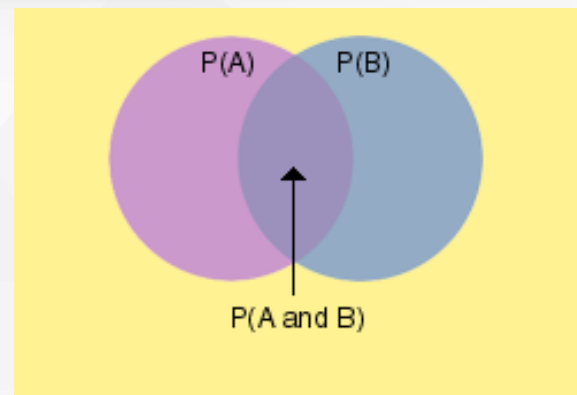
$$p(y|x) = P(Y = y|X = x) = \frac{p(x, y)}{p(x)}$$

贝叶斯公式

- 两个条件概率 $p(y|x)$ 和 $p(x|y)$ 之间的关系

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}$$

例子



Marginal Probability

性别\行业	计算机	教育	
男	0.4	0.1	0.5
女	0.1	0.4	0.5
	0.5	0.5	

Marginal Probability

$p(\text{男}|\text{计算机})=$

似然 (Likelihood)



贝叶斯公式

$$p(Y|X) = \frac{p(X|Y)p(Y)}{p(X)}$$

后验
posterior

似然
likelihood

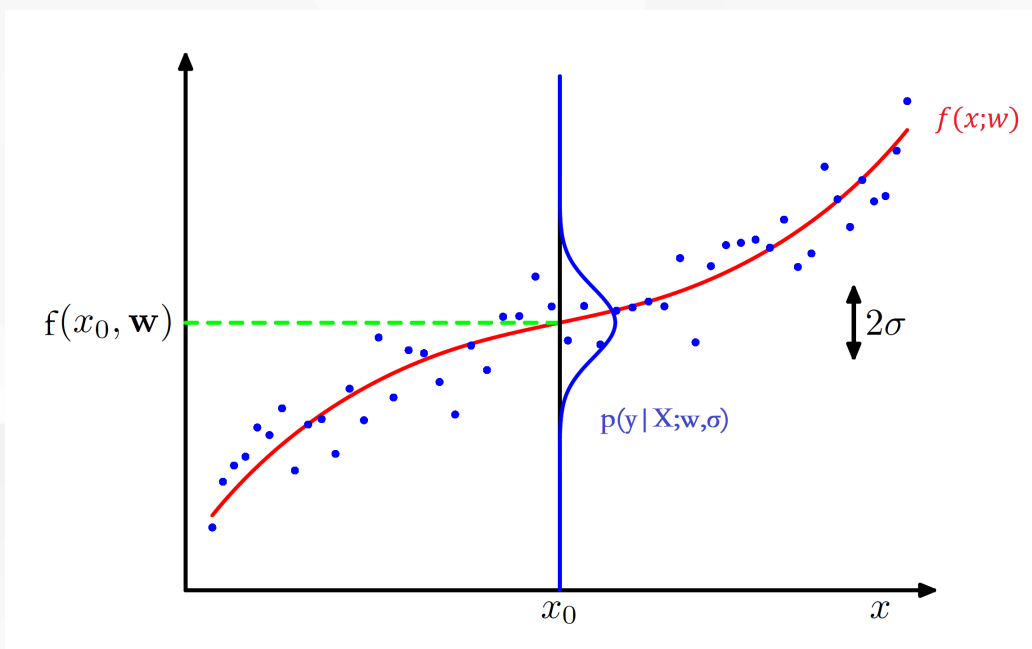
先验
prior

从概率角度来看线性回归



- 假设标签 y 为一个随机变量，其服从以均值为 $f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \mathbf{x}$ ，方差为 σ^2 的高斯分布。

$$\begin{aligned} p(y|\mathbf{x}; \mathbf{w}, \sigma) &= \mathcal{N}(y; \mathbf{w}^T \mathbf{x}, \sigma^2) \\ &= \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - \mathbf{w}^T \mathbf{x})^2}{2\sigma^2}\right). \end{aligned}$$



线性回归中的似然函数



- 参数 $\mathbf{w}$ 在训练集 D 上的似然函数 (Likelihood) 为

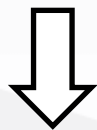
$$\begin{aligned} p(\mathbf{y}|X; \mathbf{w}, \sigma) &= \prod_{n=1}^N p(y^{(n)} | \mathbf{x}^{(n)}; \mathbf{w}, \sigma) \\ &= \prod_{n=1}^N \mathcal{N}(y^{(n)}; \mathbf{w}^T \mathbf{x}^{(n)}, \sigma^2) \end{aligned}$$

最大似然估计



- 最大似然估计 (Maximum Likelihood Estimate, MLE)
是指找到一组参数 $\mathbf{w}$ 使得似然函数 $p(\mathbf{y}|\mathbf{X}; \mathbf{w}, \sigma)$ 最大

$$\text{令 } \frac{\partial \log p(\mathbf{y}|\mathbf{X}; \mathbf{w}, \sigma)}{\partial \mathbf{w}} = 0$$



$$\mathbf{w}^{ML} = (X X^T)^{-1} X \mathbf{y}.$$

最大后验估计



$$p(\mathbf{w}|\mathbf{X}, \mathbf{y}; \nu, \sigma) = \frac{p(\mathbf{w}, \mathbf{y}|\mathbf{X}; \nu, \sigma)}{\sum_{\mathbf{w}} p(\mathbf{w}, \mathbf{y}|\mathbf{X}; \nu, \sigma)} \\ \propto \underbrace{p(\mathbf{y}|\mathbf{X}, \mathbf{w}; \sigma)}_{\text{似然}} p(\mathbf{w}; \nu),$$

后验
posterior

似然
likelihood

先验
prior

$$p(\mathbf{w}; \nu) = \mathcal{N}(\mathbf{w}; \mathbf{0}, \nu^2 I)$$

$$\log p(\mathbf{w}|\mathbf{X}, \mathbf{y}; \nu, \sigma) \propto \log p(\mathbf{y}|\mathbf{X}, \mathbf{w}; \sigma) + \log p(\mathbf{w}; \nu) \\ \propto -\frac{1}{2\sigma^2} \sum_{n=1}^N \left(y^{(n)} - \mathbf{w}^\top \mathbf{x}^{(n)} \right)^2 - \frac{1}{2\nu^2} \mathbf{w}^\top \mathbf{w}, \\ = -\frac{1}{2\sigma^2} \underbrace{\|\mathbf{y} - \mathbf{X}^\top \mathbf{w}\|^2}_{\text{正则化系数}} - \frac{1}{2\nu^2} \mathbf{w}^\top \mathbf{w}.$$

正则化系数

$$\lambda = \sigma^2 / \nu^2$$

总结



	无先验	引入先验
平方误差	经验风险 最小化	结构风险 最小化
概率	最大似然估计	最大后验估计

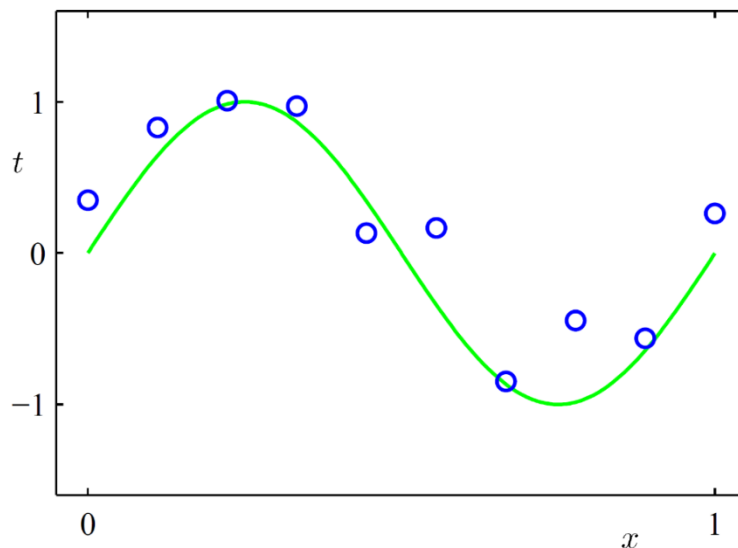
$$\mathbf{w}^{ML} = (\mathbf{X}\mathbf{X}^\top)^{-1}\mathbf{X}\mathbf{y}$$

$$\mathbf{w}^* = (\mathbf{X}\mathbf{X}^\top + \lambda\mathbf{I})^{-1}\mathbf{X}\mathbf{y}$$

多项式回归



- 一个例子：Polynomial Curve Fitting



模型

$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M$$

损失函数

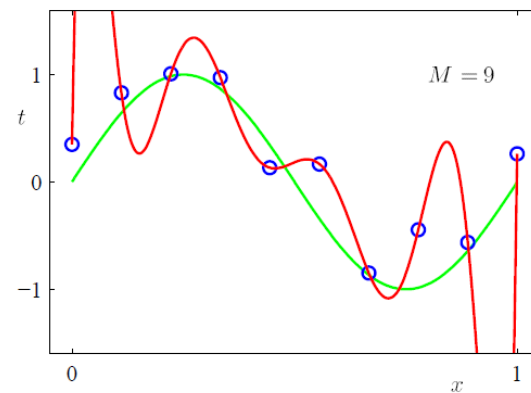
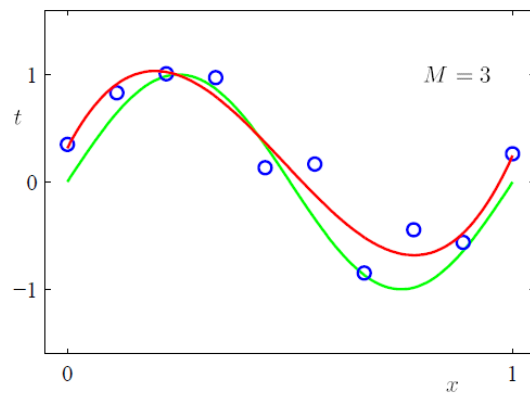
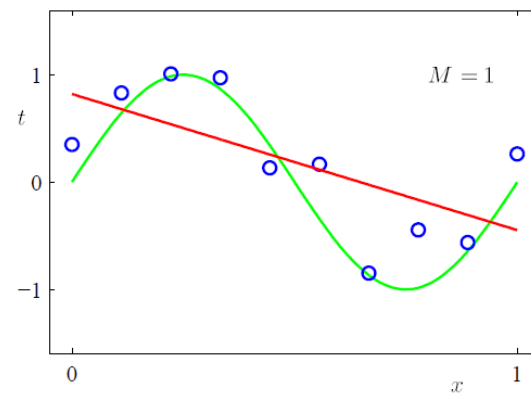
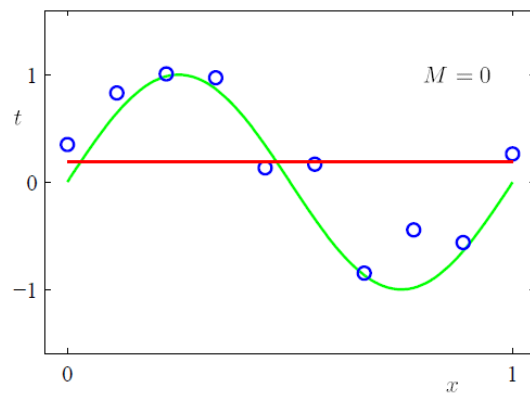
$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

多项式回归



- Which Degree of Polynomial?

A **model selection** problem

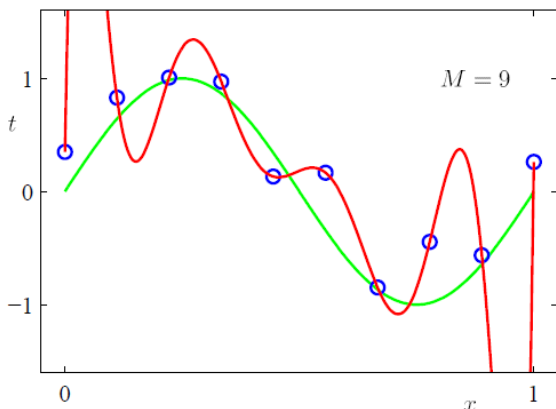


$M = 9 \rightarrow E(w) = 0$: This is **overfitting**

多项式回归



- Controlling Overfitting: Regularization



	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

As order of polynomial M increases, so do coefficient magnitudes!

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

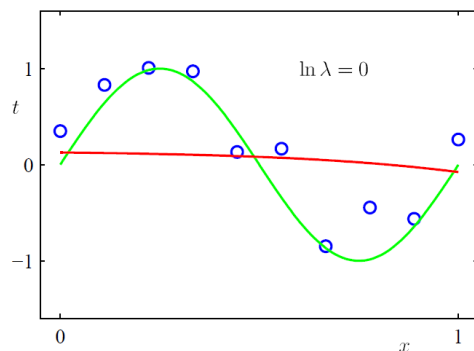
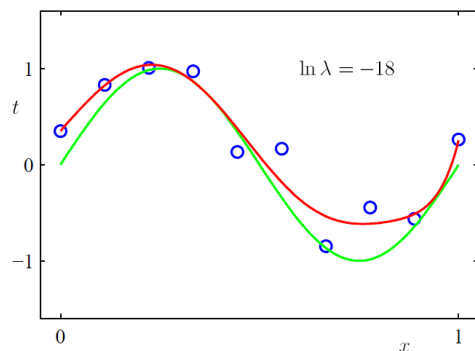
对大的系数进行惩罚

多项式回归



- Controlling Overfitting: Regularization

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

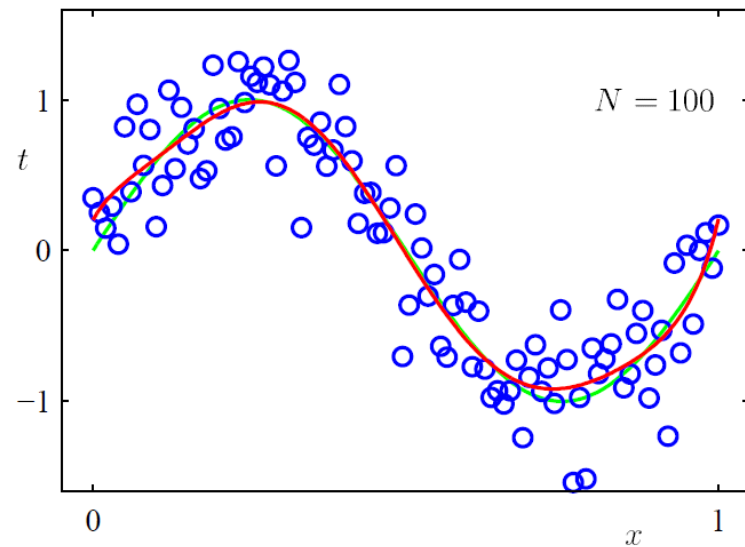
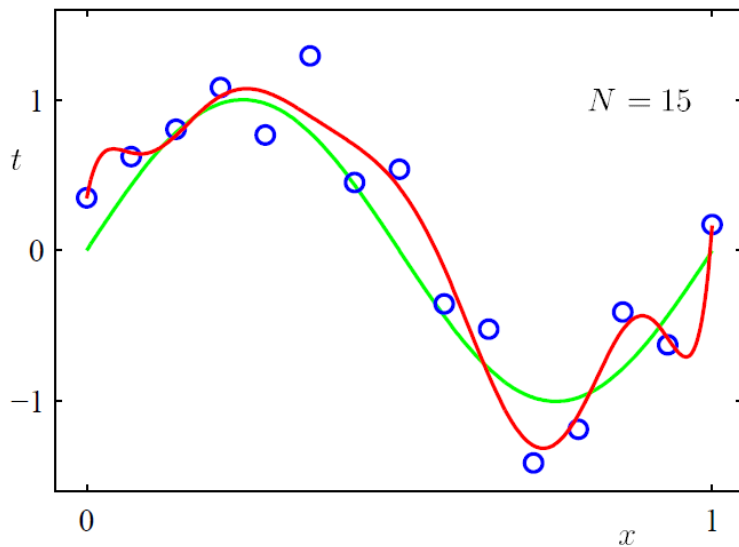


	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
w_0^*	0.35	0.35	0.13
w_1^*	232.37	4.74	-0.05
w_2^*	-5321.83	-0.77	-0.06
w_3^*	48568.31	-31.97	-0.05
w_4^*	-231639.30	-3.89	-0.03
w_5^*	640042.26	55.28	-0.02
w_6^*	-1061800.52	41.32	-0.01
w_7^*	1042400.18	-45.95	-0.00
w_8^*	-557682.99	-91.53	0.00
w_9^*	125201.43	72.68	0.01

多项式回归



- Controlling Overfitting: Dataset size



常见的机器学习类型



	监督学习	无监督学习	强化学习
训练样本	训练集 $\{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$	训练集 $\{\mathbf{x}^n\}_{n=1}^N$	智能体和环境交互的 轨迹 τ 和累积奖励 G_τ
优化目标	$y = f(\mathbf{x})$ 或 $p(y \mathbf{x})$	$p(\mathbf{x})$ 或带隐变量 $\mathbf{z}$ 的 $p(\mathbf{x} \mathbf{z})$	期望总回报 $\mathbb{E}_\tau[G_\tau]$
学习准则	期望风险最小化 最大似然估计	最大似然估计 最小重构错误	策略评估 策略改进

如何选择一个合适模型？



- 模型选择

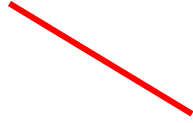
- 拟合能力强的模型一般复杂度会比较高，容易过拟合。
- 如果限制模型复杂度，降低拟合能力，可能会欠拟合。

- 偏差与方差分解

- 期望错误可以分解为

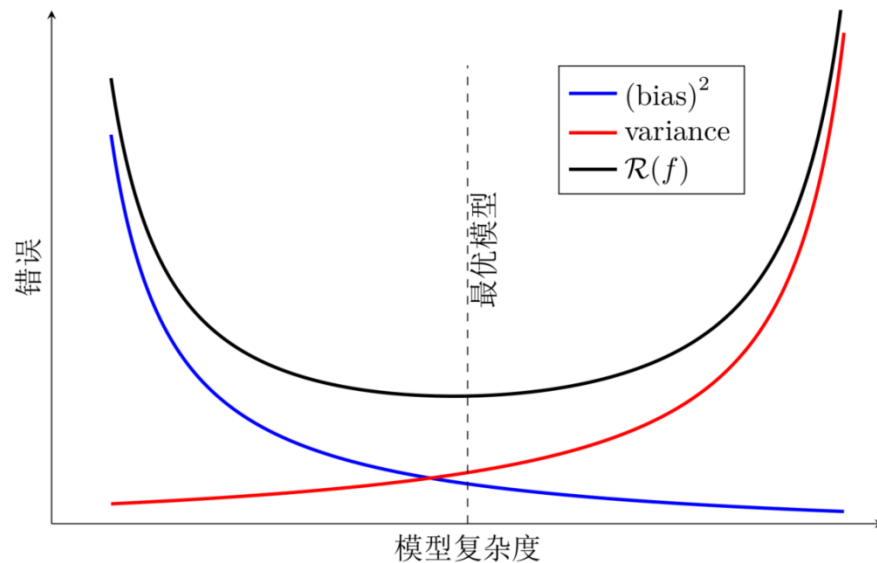
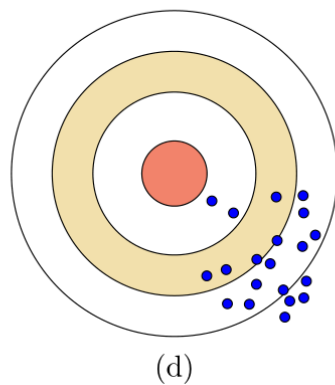
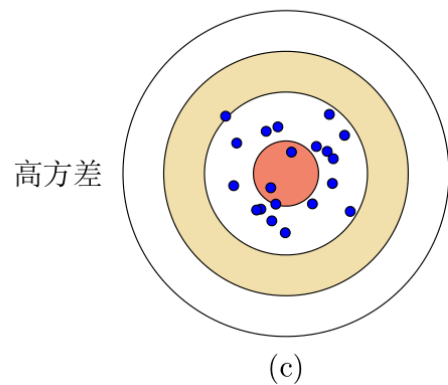
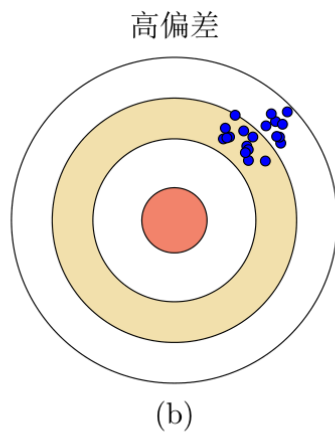
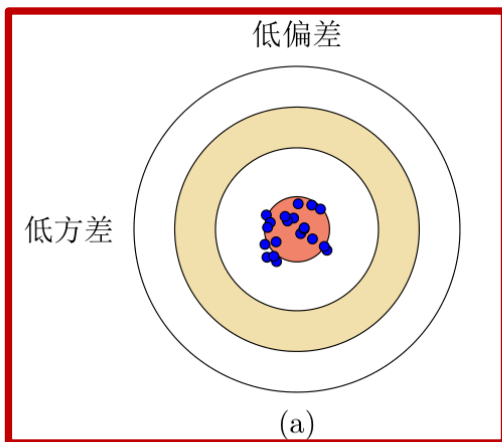
$$\mathcal{R}(f) = (\text{bias})^2 + \text{variance} + \varepsilon.$$


$$\mathbb{E}_{\mathbf{x}} \left[\left(\mathbb{E}_{\mathcal{D}} [f_{\mathcal{D}}(\mathbf{x})] - f^*(\mathbf{x}) \right)^2 \right]$$


$$\mathbb{E}_{\mathbf{x}} \left[\mathbb{E}_{\mathcal{D}} \left[\left(f_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} [f_{\mathcal{D}}(\mathbf{x})] \right)^2 \right] \right]$$


$$\mathbb{E}_{\mathbf{x}} \left[\mathbb{E}_{\mathcal{D}} \left[\left(f_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} [f_{\mathcal{D}}(\mathbf{x})] \right)^2 \right] \right]$$

模型选择：偏差与方差



集成模型：有效的降低方差的方法



PAC: Probably Approximately Correct

根据大数定律，当训练集大小 $|D|$ 趋向无穷大时，泛化错误趋向于0，即经验风险趋近于期望风险。

$$\lim_{|D| \rightarrow \infty} \mathcal{R}(f) - \mathcal{R}_D^{emp}(f) = 0$$

PAC学习

$$P\left(\underbrace{(\mathcal{R}(f) - \mathcal{R}_D^{emp}(f)) \leq \epsilon}_{\text{近似正确, } 0 < \epsilon < 0.5}\right) \geq 1 - \delta$$

近似正确, $0 < \epsilon < 0.5$

可能, $0 < \delta < 0.5$

样本复杂度



- 如果固定 ϵ , δ , 可以反过来计算出样本复杂度为

$$n(\epsilon, \delta) \geq \frac{1}{2\epsilon^2} (\ln |\mathcal{F}| + \ln \frac{2}{\delta})$$

- 其中 $|\mathcal{F}|$ 为假设空间的大小, 可以用Rademacher复杂性或VC维来衡量。
- PAC学习理论可以帮助分析一个机器学习方法在什么条件下可以学习到一个近似正确的分类器。
- 如果希望模型的假设空间越大, 泛化错误越小, 其需要的样本数量越多。

常用的定理



没有免费午餐定理 (No Free Lunch Theorem, NFL)

对于基于迭代的最优化算法，不存在某种算法对所有问题（有限的搜索空间内）都有效。如果一个算法对某些问题有效，那么它一定在另外一些问题上比纯随机搜索算法更差。



常用的定理



丑小鸭定理(Ugly Duckling Theorem)

丑小鸭与白天鹅之间的区别和两只白天鹅之间的区别一样大。



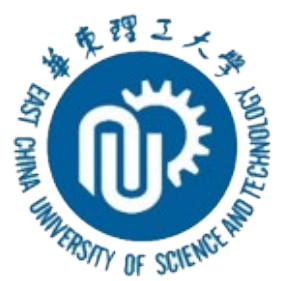
常用的定理



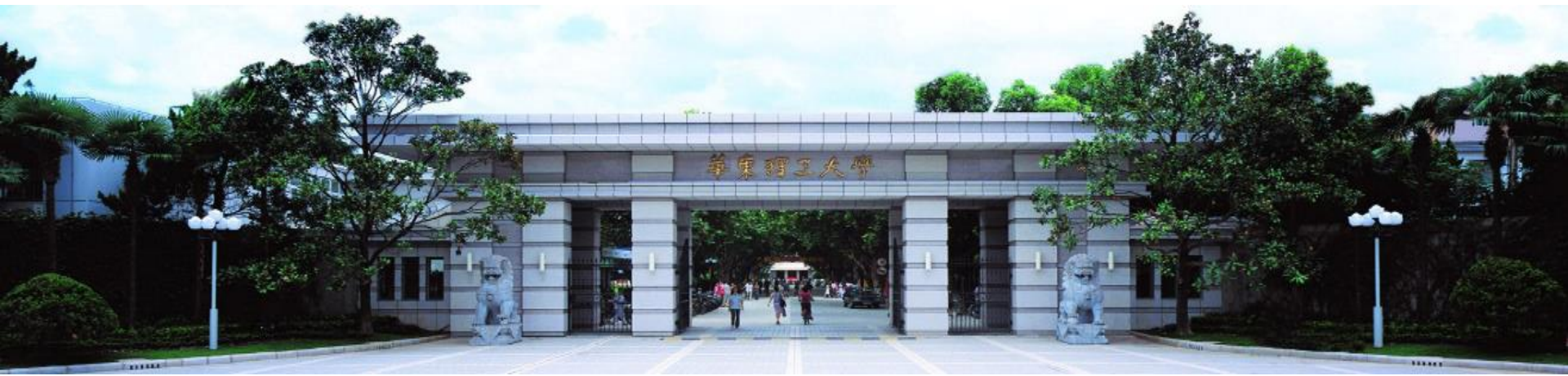
奥卡姆剃刀原理(Occam's Razor)

如无必要，勿增实体。





前馈神经网络 (课后自行复习)



内容



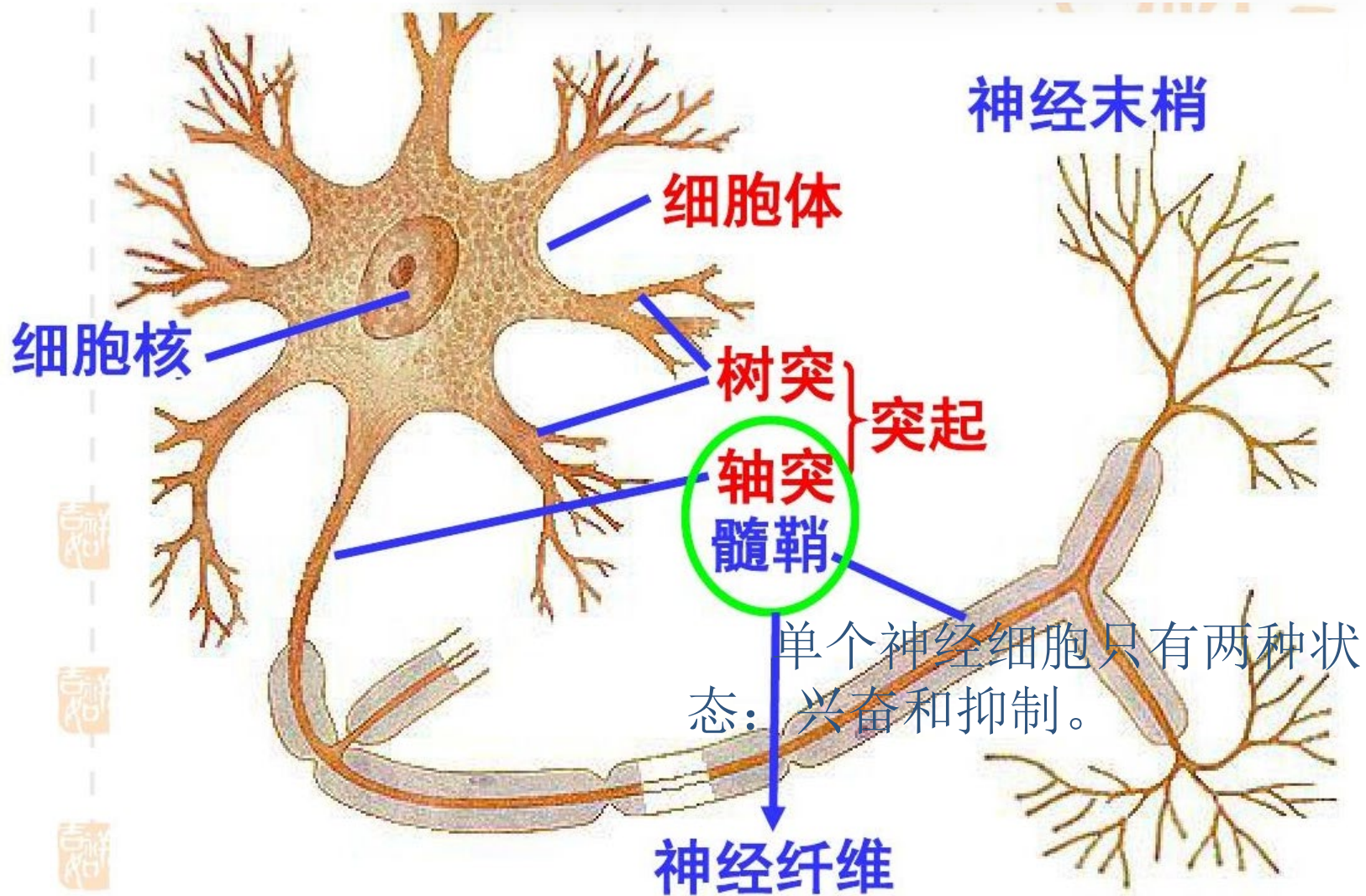
- 神经网络
 - 神经元
 - 网络结构
- 前馈神经网络
 - 参数学习
 - 计算图与自动微分
 - 优化问题



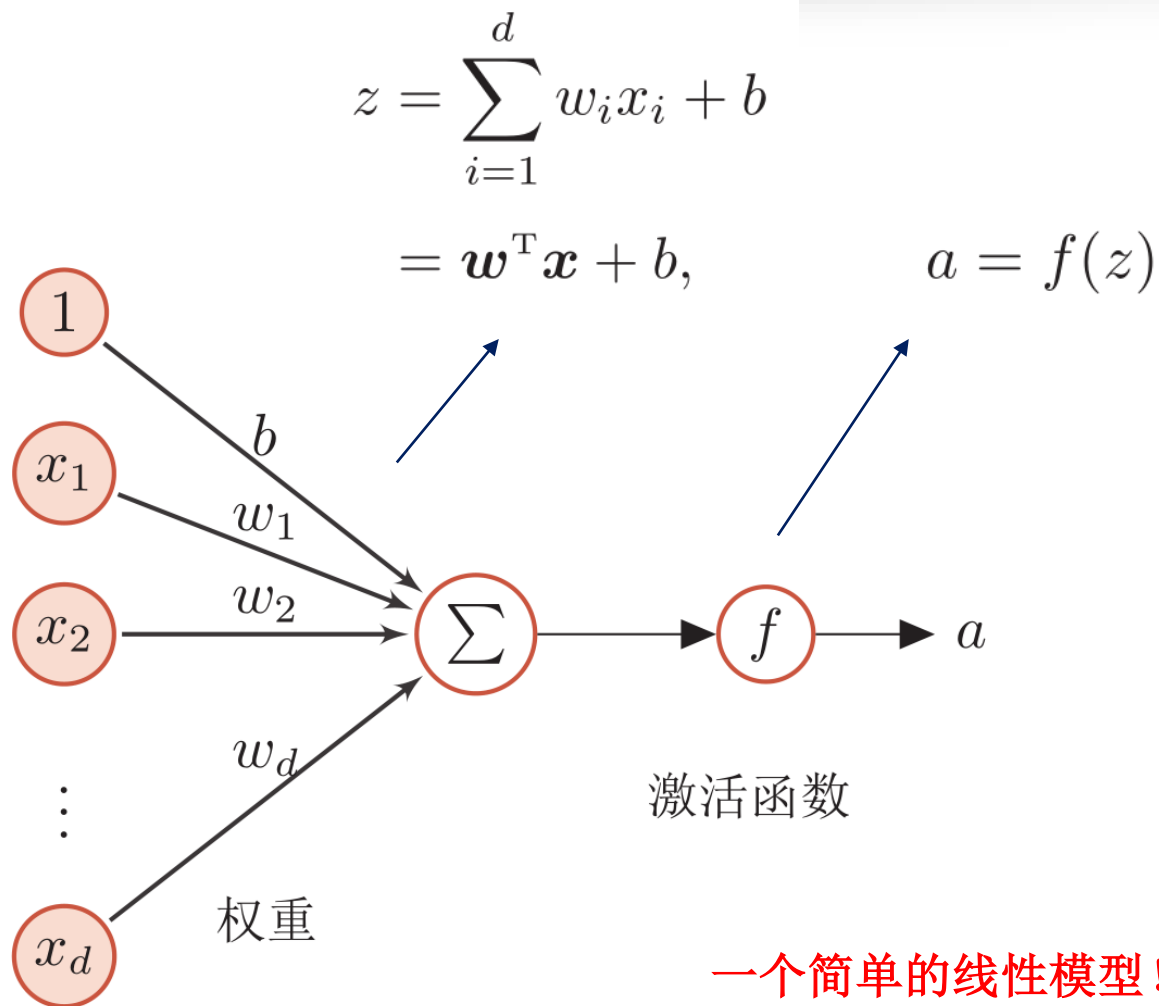
神经网络最早是作为一种主要的连接主义模型。

- 20世纪80年代后期，最流行的一种连接主义模型是分布式并行处理（Parallel Distributed Processing, PDP）网络，其有3个主要特性：
 - 1) 信息表示是分布式的（非局部的）；
 - 2) 记忆和知识是存储在单元之间的连接上；
 - 3) 通过逐渐改变单元之间的连接强度来学习新的知识。
- 引入误差反向传播来改进其学习能力之后，神经网络也越来越多地应用在各种机器学习任务上。

生物神经元



人工神经元

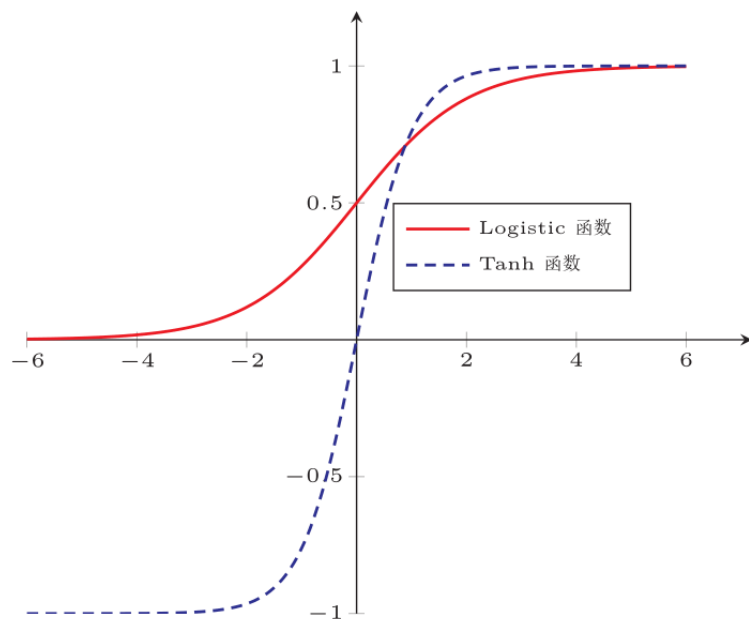


激活函数的性质



- 连续并可导（允许少数点上不可导）的非线性函数。
可导的激活函数可以直接利用数值优化的方法来学习网络参数。
- 激活函数及其导函数要尽可能的简单。
有利于提高网络计算效率。
- 激活函数的导函数的值域要在一个合适的区间内。
不能太大也不能太小，否则会影响训练的效率和稳定性。
- 单调递增

常见激活函数



$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

非零中心化的输出会使得其下一层的神经元的输入发生偏置偏移（bias shift），并进一步使得梯度下降的收敛速度变慢。

性质：

（1）饱和函数

（2）Tanh函数是零中心化的，而logistic函数的输出恒大于0

常见激活函数



$$\text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$= \max(0, x).$$

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$

$$= \max(0, x) + \gamma \min(0, x)$$

$$\text{PReLU}_i(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma_i x & \text{if } x \leq 0 \end{cases}$$

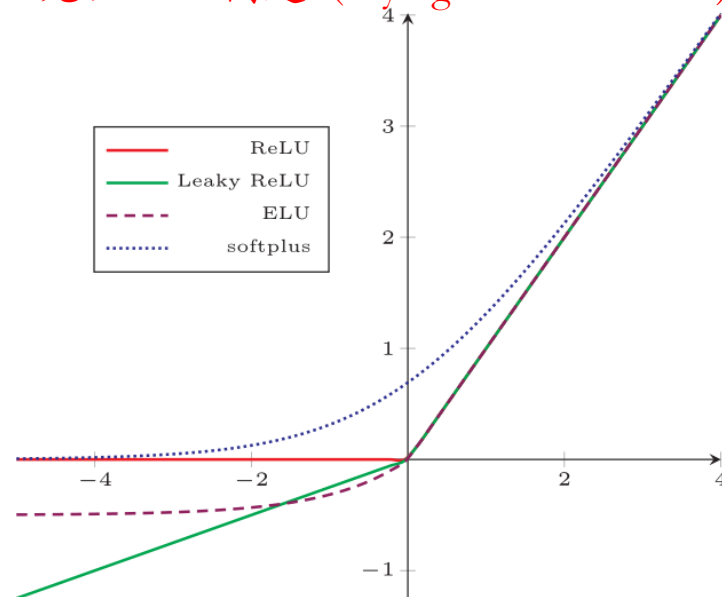
$$= \max(0, x) + \gamma_i \min(0, x)$$

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

$$\text{softplus}(x) = \log(1 + \exp(x))$$

死亡ReLU问题 (Dying ReLU Problem)



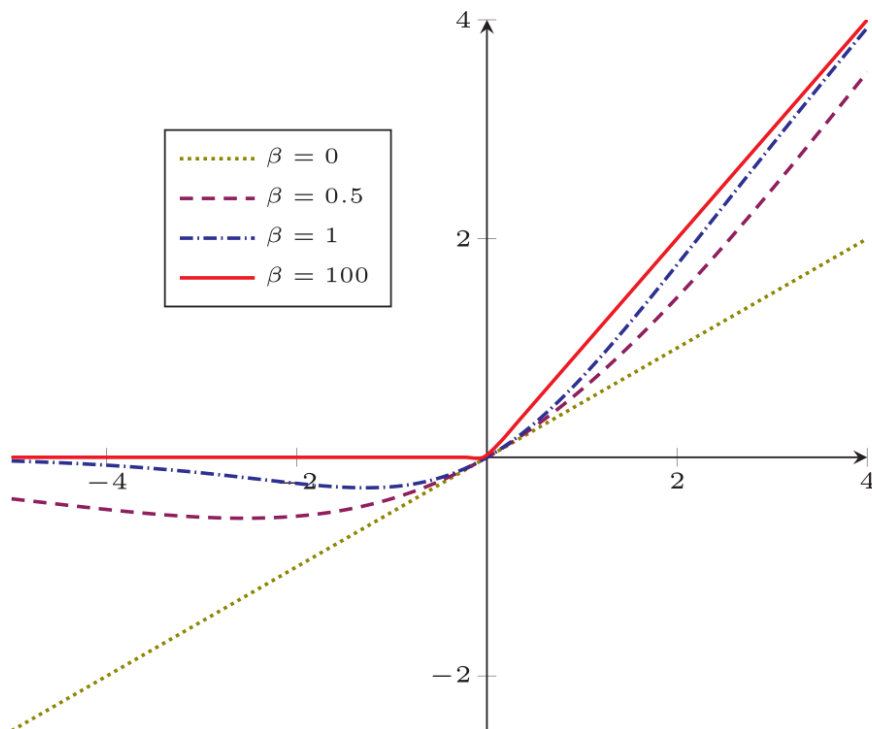
- ▶ 计算上更加高效
- ▶ 生物学合理性 单侧抑制、宽兴奋边界
- ▶ 在一定程度上缓解梯度消失问题

常见激活函数



Swish函数

$$\text{swish}(x) = x\sigma(\beta x)$$



常见激活函数



- 高斯误差线性单元 (Gaussian Error Linear Unit, GELU)

$$\text{GELU}(x) = xP(X \leq x)$$

- 其中 $P(X \leq x)$ 是高斯分布 $N(\mu, \sigma^2)$ 的累积分布函数，其中 μ, σ 为超参数，一般设 $\mu = 0, \sigma = 1$ 即可
- 由于高斯分布的累积分布函数为S型函数，因此GELU可以用Tanh函数或Logistic函数来近似

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$$

或 $\text{GELU}(x) \approx x \sigma(1.702x).$

常见激活函数及其导数



激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)(1 - f(x))$
Tanh 函数	$f(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU 函数	$f(x) = \max(0, x) + \min(0, \gamma(\exp(x) - 1))$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log(1 + \exp(x))$	$f'(x) = \frac{1}{1+\exp(-x)}$

人工神经网络



人工神经网络主要由大量的神经元以及它们之间的有向连接构成。因此考虑三方面：

01 神经元的激活规则

主要是指神经元输入到输出之间的映射关系，一般为非线性函数。

02 网络的拓扑结构

不同神经元之间的连接关系。

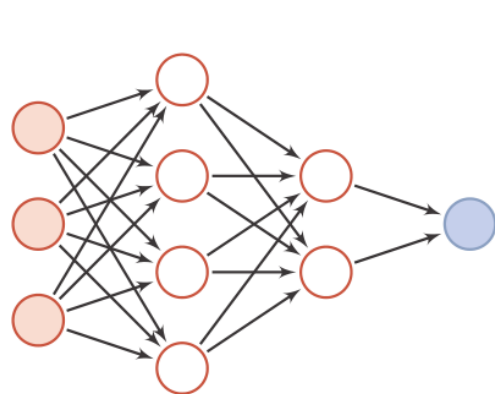
03 学习算法

通过训练数据来学习神经网络的参数。

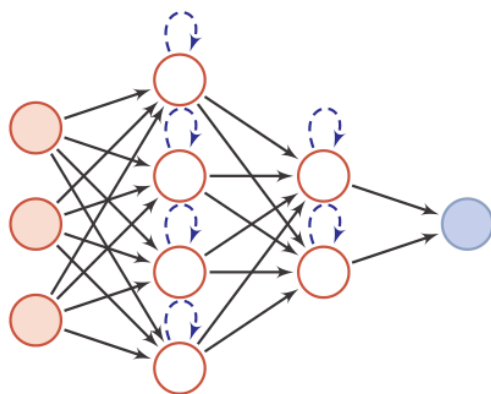
网络结构



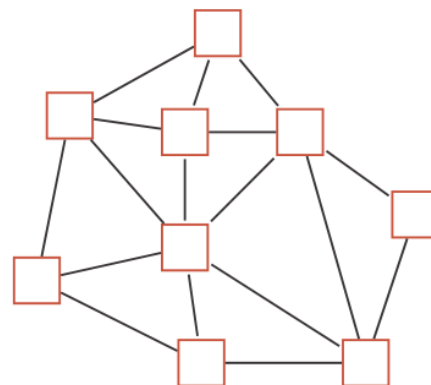
- 人工神经网络由神经元模型构成，这种由许多神经元组成的信息处理网络具有并行分布结构。



(a) 前馈网络



(b) 记忆网络



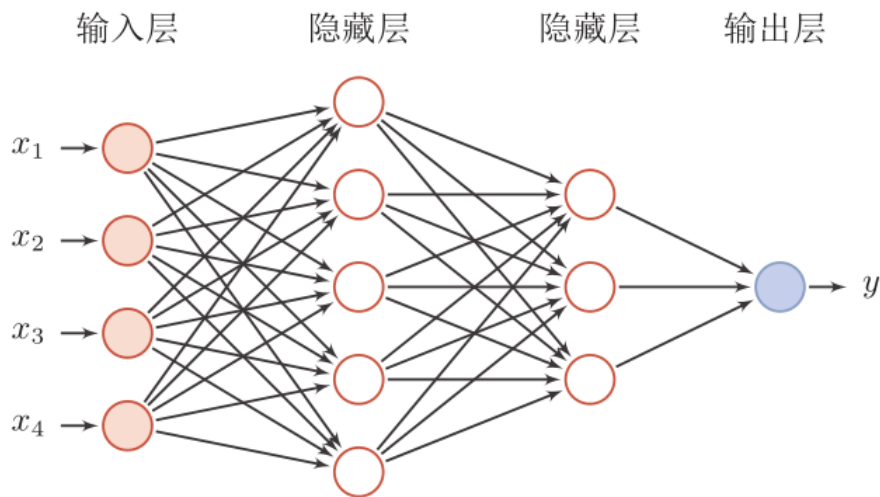
(c) 图网络

圆形节点表示一个神经元，方形节点表示一组神经元。

前馈网络



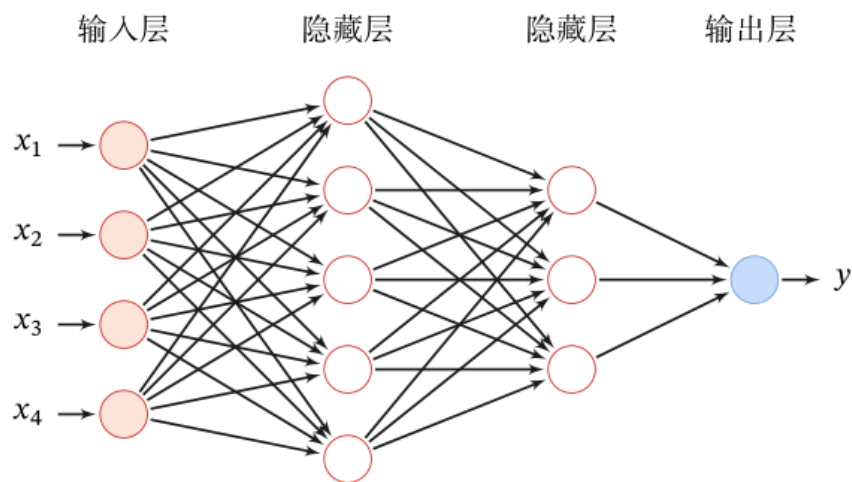
- 前馈神经网络（全连接神经网络、多层感知器）
 - 各神经元分别属于不同的层，层内无连接。
 - 相邻两层之间的神经元全部两两连接。
 - 整个网络中无反馈，信号从输入层向输出层单向传播，可用一个有向无环图表示。



前馈网络



给定一个前馈神经网络，用下面的记号来描述这样网络：



记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\mathbf{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\mathbf{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$\mathbf{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入 (净活性值)
$\mathbf{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出 (活性值)

- 前馈神经网络通过下面公式进行信息传播。

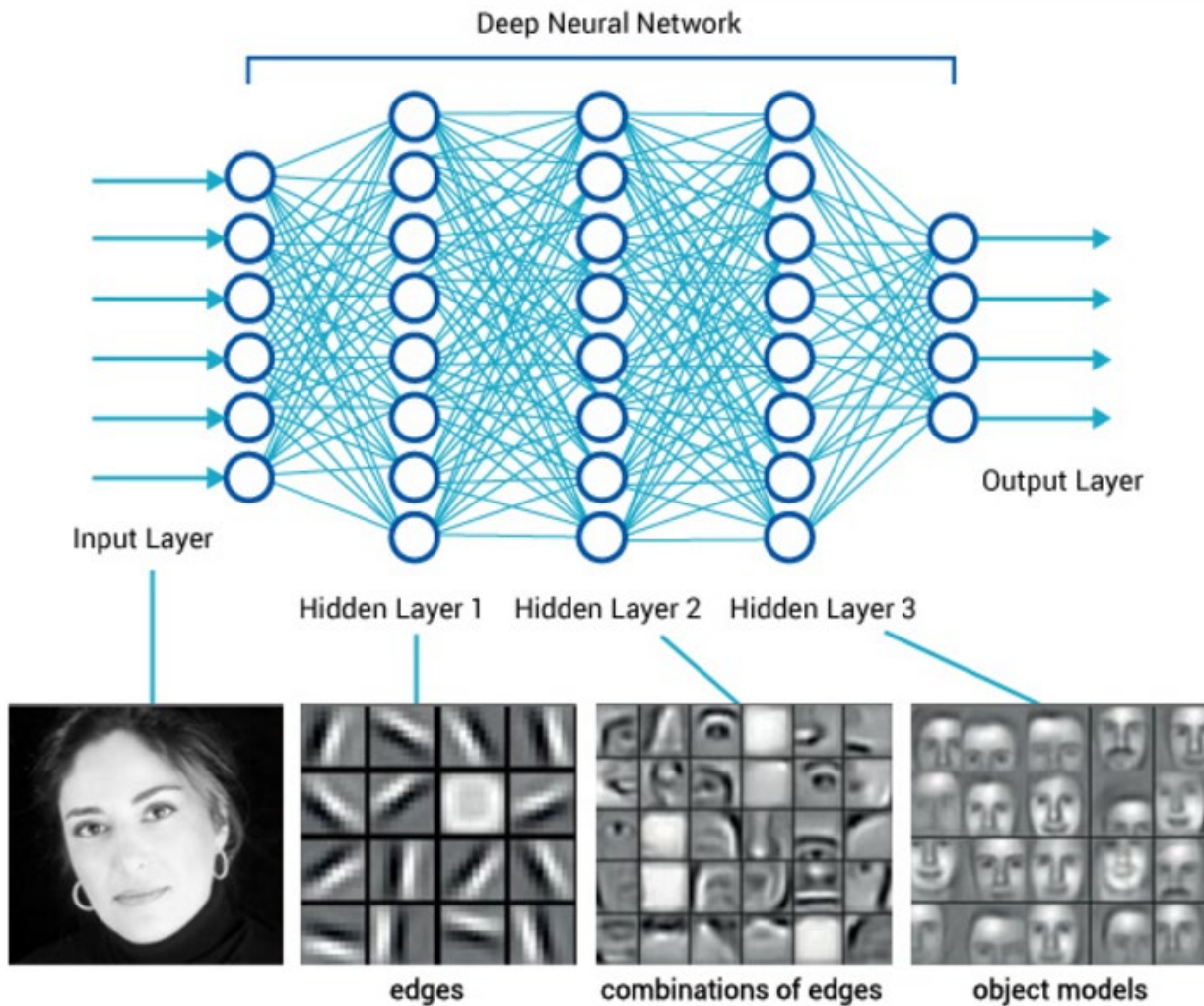
$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)},$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}).$$

- 前馈计算：

$$\mathbf{x} = \mathbf{a}^{(0)} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \dots \rightarrow \mathbf{a}^{(L-1)} \rightarrow \mathbf{z}^{(L)} \rightarrow \mathbf{a}^{(L)} = \phi(\mathbf{x}; \mathbf{W}, \mathbf{b})$$

深层前馈神经网络



通用近似定理



定理 4.1 – 通用近似定理 (Universal Approximation Theorem)

[Cybenko, 1989, Hornik et al., 1989]: 令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数, $\mathcal{I}_d$ 是一个 d 维的单位超立方体 $[0, 1]^d$, $C(\mathcal{I}_d)$ 是定义在 $\mathcal{I}_d$ 上的连续函数集合。对于任何一个函数 $f \in C(\mathcal{I}_d)$, 存在一个整数 m , 和一组实数 $v_i, b_i \in \mathbb{R}$ 以及实数向量 $\mathbf{w}_i \in \mathbb{R}^d$, $i = 1, \dots, m$, 以至于我们可以定义函数

$$F(\mathbf{x}) = \sum_{i=1}^m v_i \varphi(\mathbf{w}_i^T \mathbf{x} + b_i), \quad (4.33)$$

作为函数 f 的近似实现, 即

$$|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon, \forall \mathbf{x} \in \mathcal{I}_d. \quad (4.34)$$

其中 $\epsilon > 0$ 是一个很小的正数。

根据通用近似定理, 对于具有线性输出层和至少一个使用“挤压”性质的激活函数的隐藏层组成的前馈神经网络, 只要其隐藏层神经元的数量足够, 它可以以任意的精度来近似任何从一个定义在实数空间中的有界闭集函数。

应用到机器学习



- 神经网络可以作为一个“万能”函数来使用，可以用来进行复杂的特征转换，或逼近一个复杂的条件分布。

$$\hat{y} = g(\varphi(\mathbf{x}), \theta)$$

分类器

神经网络

- 如果 $g(\cdot)$ 为Logistic回归，那么Logistic回归分类器可以看成神经网络的最后一层。



对于多分类问题

- 如果使用Softmax回归分类器，相当于网络最后一层设置C个神经元，其输出经过Softmax函数进行归一化后可以作为每个类的条件概率。

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(L)})$$

- 采用交叉熵损失函数，对于样本(x,y)，其损失函数为

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

参数学习



- 给定训练集 $D = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^N$ ，将每个样本 $\mathbf{x}^{(n)}$ 输入给前馈神经网络，得到网络输出为 $\mathbf{y}^{(n)}$ ，其在数据集 D 上的结构化风险函数为：

$$\mathcal{R}(W, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2} \lambda \|W\|_F^2$$

- 梯度下降

$$W^{(l)} \leftarrow W^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial W^{(l)}}$$

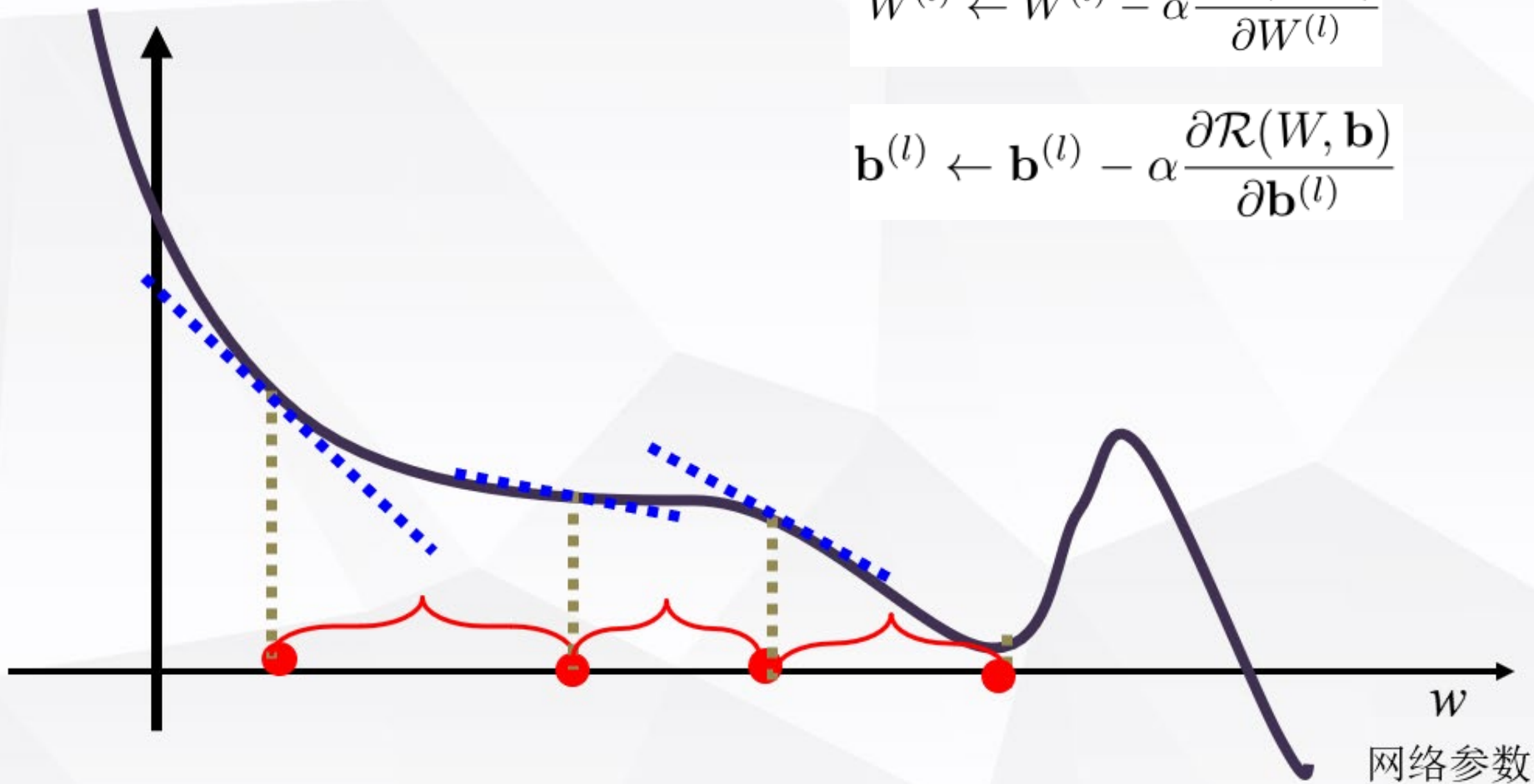
$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial \mathbf{b}^{(l)}}$$

梯度下降



$$W^{(l)} \leftarrow W^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial W^{(l)}}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial \mathbf{b}^{(l)}}$$



如何计算梯度？



- 神经网络为一个复杂的复合函数
链式法则

$$y = f^5(f^4(f^3(f^2(f^1(x))))) \rightarrow \frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$

- 反向传播算法
根据前馈网络的特点而设计的高效方法
- 一个更加通用的计算方法
自动微分 (Automatic Differentiation, AD)

矩阵微积分



- 矩阵微积分（Matrix Calculus）是多元微积分的一种表达方式，即使用矩阵和向量来表示因变量每个成分关于自变量每个成分的偏导数。
- 分母布局

标量关于向量的偏导数

$$\frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_p} \right]^T$$

向量关于向量的偏导数

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_q}{\partial x_1} \\ \vdots & \vdots & \vdots \\ \frac{\partial y_1}{\partial x_p} & \dots & \frac{\partial y_q}{\partial x_p} \end{bmatrix} \in \mathbb{R}^{p \times q}$$

链式法则



- 链式法则 (Chain Rule) 是在微积分中求复合函数导数的一种常用方法。

(1) 若 $x \in \mathbb{R}$, $\mathbf{u} = u(x) \in \mathbb{R}^s$, $\mathbf{g} = g(\mathbf{u}) \in \mathbb{R}^t$, 则

$$\frac{\partial \mathbf{g}}{\partial x} = \frac{\partial \mathbf{u}}{\partial x} \frac{\partial \mathbf{g}}{\partial \mathbf{u}} \in \mathbb{R}^{1 \times t}.$$

(2) 若 $\mathbf{x} \in \mathbb{R}^p$, $\mathbf{y} = g(\mathbf{x}) \in \mathbb{R}^s$, $\mathbf{z} = f(\mathbf{y}) \in \mathbb{R}^t$, 则

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \in \mathbb{R}^{p \times t}.$$

(3) 若 $X \in \mathbb{R}^{p \times q}$ 为矩阵, $\mathbf{y} = g(X) \in \mathbb{R}^s$, $z = f(\mathbf{y}) \in \mathbb{R}$, 则

$$\frac{\partial z}{\partial X_{ij}} = \frac{\partial \mathbf{y}}{\partial X_{ij}} \frac{\partial z}{\partial \mathbf{y}} \in \mathbb{R}.$$

反向传播算法



- 假设采用随机梯度下降进行神经网络参数学习，给定一个样本 (x, y) ，将其输入到神经网络模型中，得到网络输出为 $\hat{y}$ ，假设损失函数为 $L(y, \hat{y})$ ，要进行参数学习就需要计算损失函数关于每个参数的导数。
- 计算 $\frac{\partial \mathcal{L}(y, \hat{y})}{\partial W^{(l)}}$ 的过程涉及矩阵微分，比较繁琐，因此先计算 $L(y, \hat{y})$ 关于参数矩阵中每个元素的偏导数 $\frac{\partial \mathcal{L}(y, \hat{y})}{\partial w_{ij}^{(l)}}$ ，根据链式法则：

$$\frac{\partial \mathcal{L}(y, \hat{y})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \mathbf{z}^{(l)}},$$
$$\frac{\partial \mathcal{L}(y, \hat{y})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \mathbf{z}^{(l)}}.$$

- 分别计算 $\frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}}$ 、 $\frac{\partial \mathcal{L}(y, \hat{y})}{\partial \mathbf{z}^{(l)}}$ 、 $\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}}$

反向传播算法



$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\begin{aligned} \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_m^{(l)}}{\partial w_{ij}^{(l)}} \right] \\ &= \left[0, \dots, \frac{\partial (\mathbf{w}_i^{(l)} \mathbf{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}, \dots, 0 \right] \\ &= \left[0, \dots, a_j^{(l-1)}, \dots, 0 \right] \\ &\triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{m^{(l)}}, \end{aligned}$$

$$\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}}$$

误差项

$$\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{m^{(l)}} \in \mathbb{R}^{m^{(l)} \times m^{(l)}}$$

计算

$$\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}}$$



$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$$

$$\mathbf{z}^{(l+1)} = W^{(l+1)} \mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$$

$$\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} = (W^{(l+1)})^T$$

$$\begin{aligned} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} &= \frac{\partial f_l(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(l)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \end{aligned}$$

$$\begin{aligned} \delta^{(l)} &\triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \\ &= \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l+1)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \cdot (W^{(l+1)})^T \cdot \delta^{(l+1)} \\ &= f'_l(\mathbf{z}^{(l)}) \odot ((W^{(l+1)})^T \delta^{(l+1)}), \end{aligned}$$

反向传播算法



在计算出上面三个偏导数之后, 公式 (4.49) 可以写为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \mathbb{I}_i(a_j^{(l-1)}) \delta^{(l)} = \delta_i^{(l)} a_j^{(l-1)}.$$

进一步, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层权重 $W^{(l)}$ 的梯度为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial W^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top.$$

同理, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层偏置 $\mathbf{b}^{(l)}$ 的梯度为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}.$$

反向传播算法



算法 4.1 使用反向传播算法的随机梯度下降训练过程

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α , 正则化系数 λ , 网络层数 L , 神经元数量 M_l , $1 \leq l \leq L$.

```
1 随机初始化  $\mathbf{W}, \mathbf{b}$ ;  
2 repeat  
3   对训练集  $\mathcal{D}$  中的样本随机重排序;  
4   for  $n = 1 \cdots N$  do  
5     从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;  
6     前馈计算每一层的净输入  $\mathbf{z}^{(l)}$  和激活值  $\mathbf{a}^{(l)}$ , 直到最后一层;  
7     反向传播计算每一层的误差  $\delta^{(l)}$ ; // 公式 (4.63)  
      // 计算每一层参数的导数  
8      $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$ ; // 公式 (4.68)  
9      $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$ ; // 公式 (4.69)  
      // 更新参数  
10     $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha (\delta^{(l)} (\mathbf{a}^{(l-1)})^\top + \lambda \mathbf{W}^{(l)})$ ;  
11     $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \delta^{(l)}$ ;  
12  end  
13 until 神经网络模型在验证集  $\mathcal{V}$  上的错误率不再下降;  
    输出:  $\mathbf{W}, \mathbf{b}$ 
```

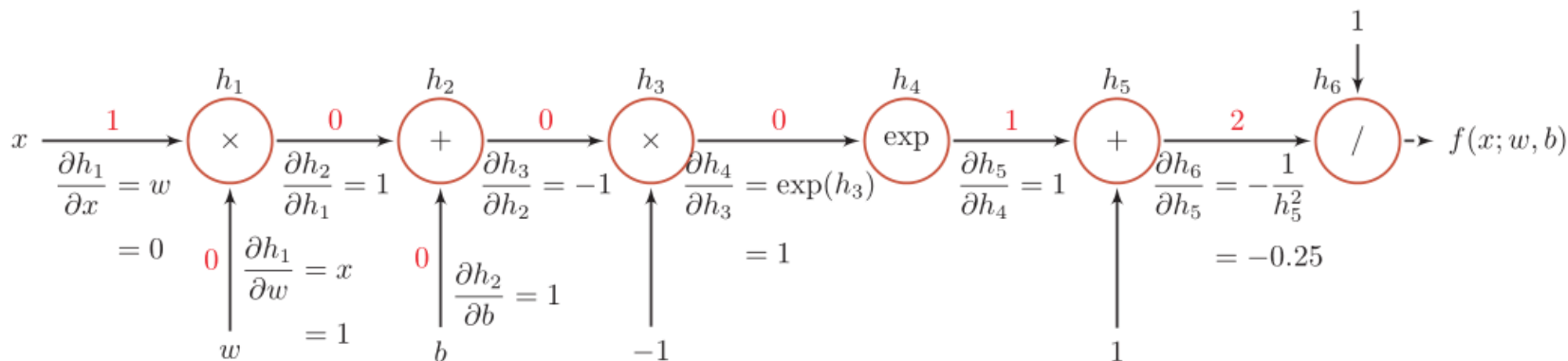
计算图与自动微分



- 自动微分是利用链式法则来自动计算一个复合函数的梯度。

$$f(x; w, b) = \frac{1}{\exp(-(wx + b)) + 1}.$$

- 计算图



计算图



函数	导数	
$h_1 = x \times w$	$\frac{\partial h_1}{\partial w} = x$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$	$\frac{\partial h_2}{\partial b} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$	
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$	
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$	
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$	

当 $x = 1, w = 0, b = 0$ 时，可以得到

$$\begin{aligned}\frac{\partial f(x; w, b)}{\partial w} \Big|_{x=1, w=0, b=0} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} \\ &= 1 \times -0.25 \times 1 \times 1 \times -1 \times 1 \times 1 \\ &= 0.25.\end{aligned}$$

自动微分



- 前向模式和反向模式
反向模式和反向传播的计算梯度的方式相同
- 如果函数和参数之间有多条路径，可以将这多条路径上的导数再进行相加，得到最终的梯度。

反向传播算法

(自动微分的反向模式)



- 前馈神经网络的训练过程可以分为以下三步

Step1 前向计算每一层的状态和激活值，直到最后一层

Step2 反向计算每一层的参数的偏导数

Step3 更新参数

如何实现？



Deep Learning



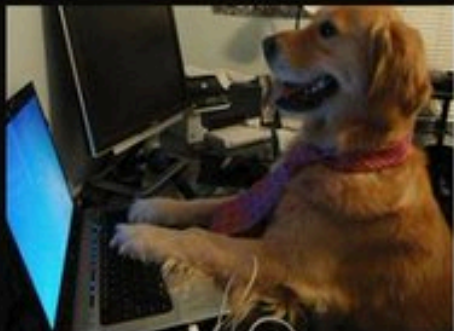
What society thinks I do



What my friends think I do



What other computer scientists think I do



What mathematicians think I do



What I think I do

```
from theano import *
```

What I actually do

Getting started: 30 seconds to Keras

```
from keras.models import Sequential
from keras.layers import Dense, Activation
from keras.optimizers import SGD

model = Sequential()
model.add(Dense(output_dim=64, input_dim=100))
model.add(Activation("relu"))
model.add(Dense(output_dim=10))
model.add(Activation("softmax"))

model.compile(loss='categorical_crossentropy',
              optimizer='sgd', metrics=['accuracy'])

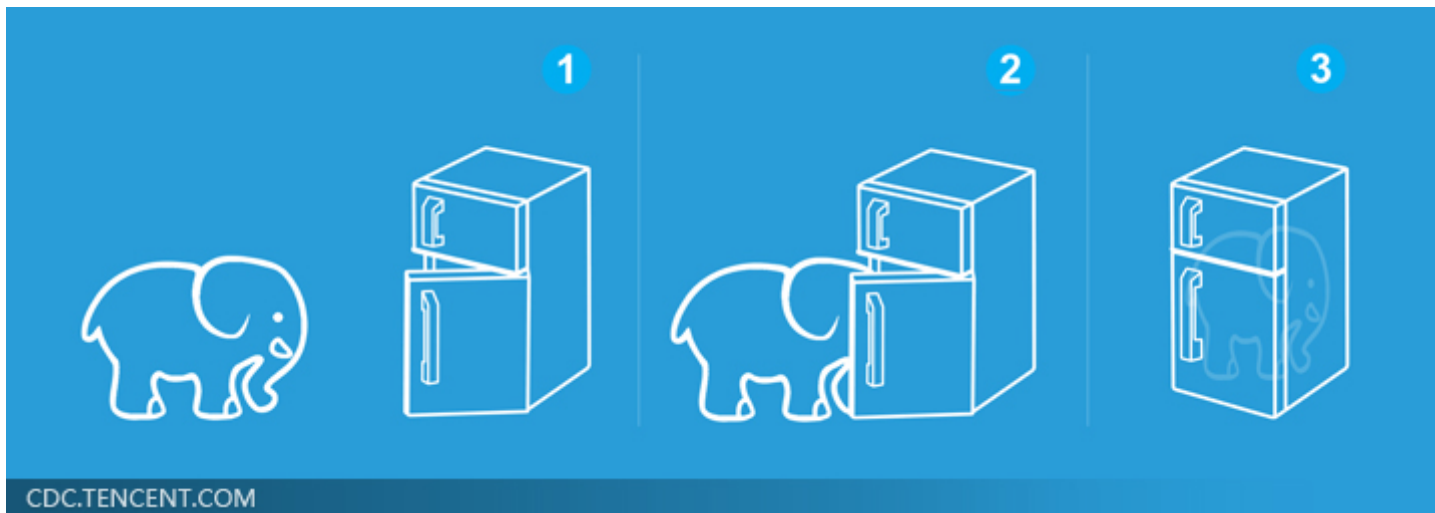
model.fit(X_train, Y_train, nb_epoch=5, batch_size=32)

loss = model.evaluate(X_test, Y_test, batch_size=32)
```

深度学习的三个步骤



Deep Learning is so simple



优化问题



难点

- 参数过多，影响训练
- 非凸优化问题：即存在局部最优而非全局最优解，影响迭代
- 梯度消失问题，下层参数比较难调
- 参数解释起来比较困难

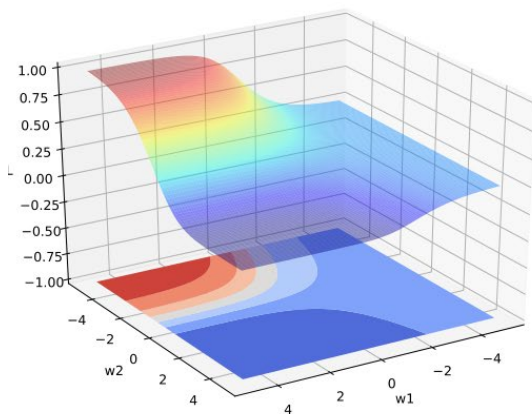
需求

- 计算资源要大
- 数据要多
- 算法效率要好：即收敛快

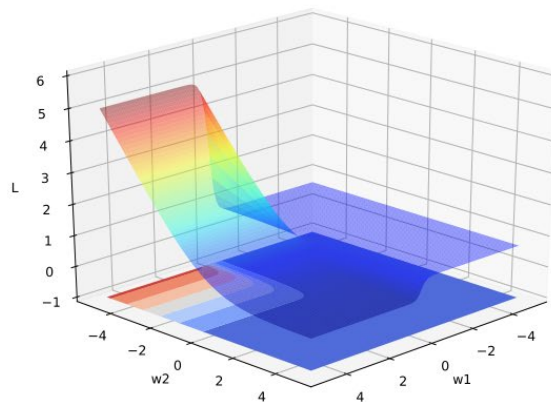
优化问题



- 非凸优化问题



(a) 平方误差损失



(b) 交叉熵损失

图 4.9 神经网络 $y = \sigma(w_2\sigma(w_1x))$ 的损失函数

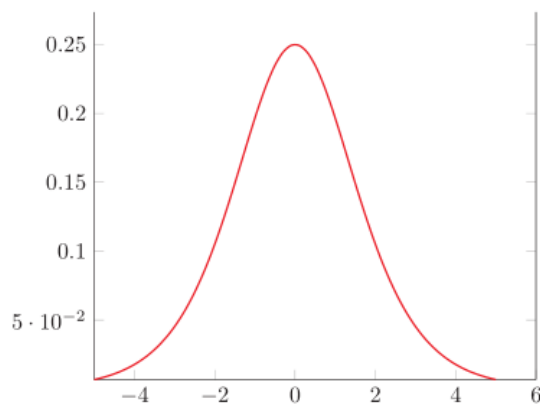
优化问题



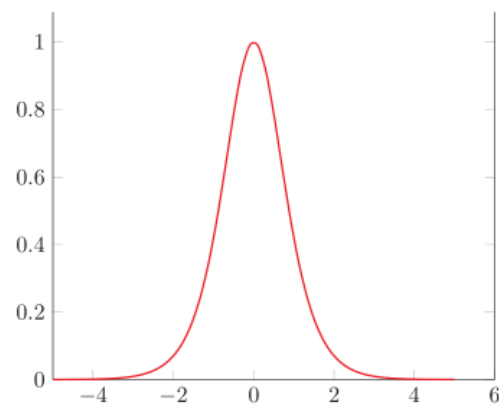
- 梯度消失问题 (Vanishing Gradient Problem)

$$y = f^5(f^4(f^3(f^2(f^1(x)))))$$

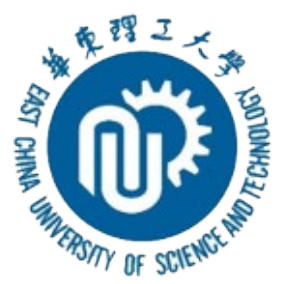
$$\frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$



(a) logistic 函数的导数

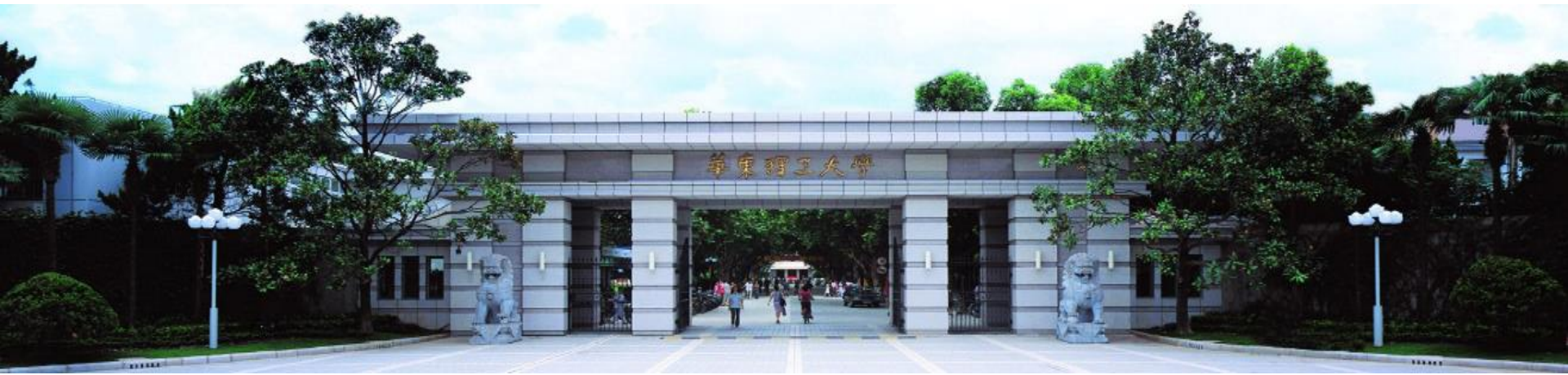


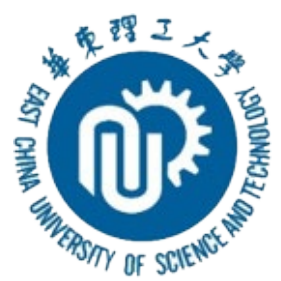
(b) tanh 函数的导数



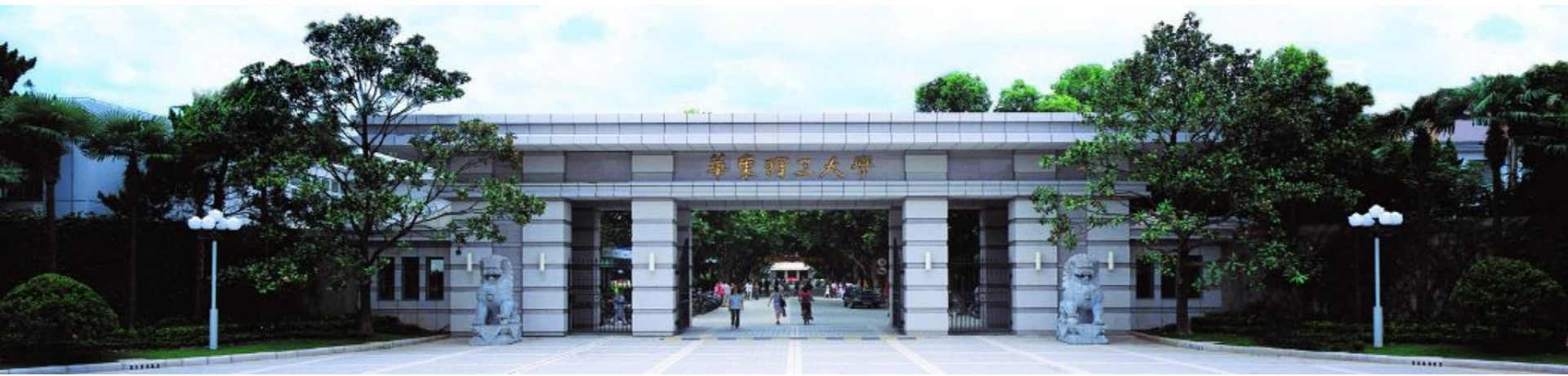
谢谢！

THANK YOU FOR LISTENING





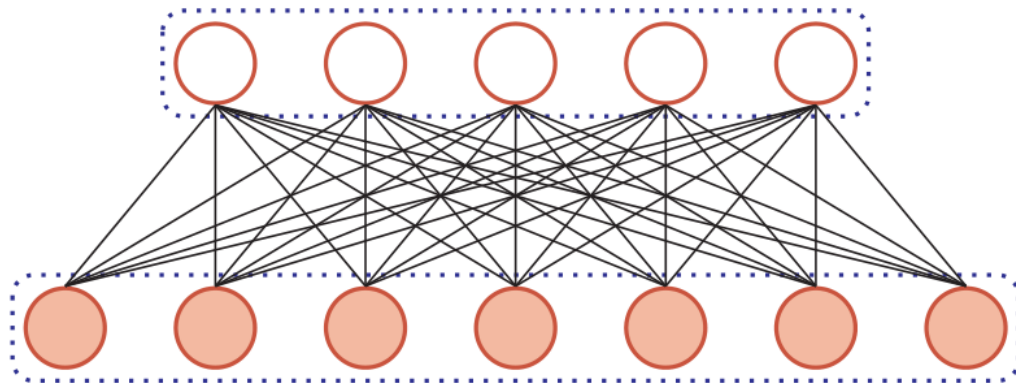
卷积神经网络 (课后自行复习)



全连接前馈神经网络



- 权重矩阵的参数非常多



- 局部不变性特征

自然图像中的物体都具有局部不变性特征，尺度缩放、平移、旋转等操作不影响其语义信息。

全连接前馈网络很难提取这些局部不变特征。



卷积神经网络：一种前馈神经网络

- 受生物学上感受野（Receptive Field）的机制而提出的
在视觉神经系统中，一个神经元的感受野是指视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元。

卷积神经网络有三个结构上的特性：

★ 局部连接

★ 权重共享

★ 空间或时间上的次采样

卷积



- 卷积经常用在信号处理中，用于计算信号的延迟累积。
- 假设一个信号发生器每个时刻 t 产生一个信号 x_t ，其信息的衰减率为 w_k ，即在 $k-1$ 个时间步长后，信息为原来的 w_k 倍
 - 假设 $w_1 = 1, w_2 = 1/2, w_3 = 1/4$
- 时刻 t 收到的信号 y_t 为当前时刻产生的信息和以前时刻延迟信息的叠加。

$$y_t = 1 \times x_t + 1/2 \times x_{t-1} + 1/4 \times x_{t-2}$$

$$= w_1 \times x_t + w_2 \times x_{t-1} + w_3 \times x_{t-2}$$

$$= \sum_{k=1}^3 w_k \cdot x_{t-k+1}$$



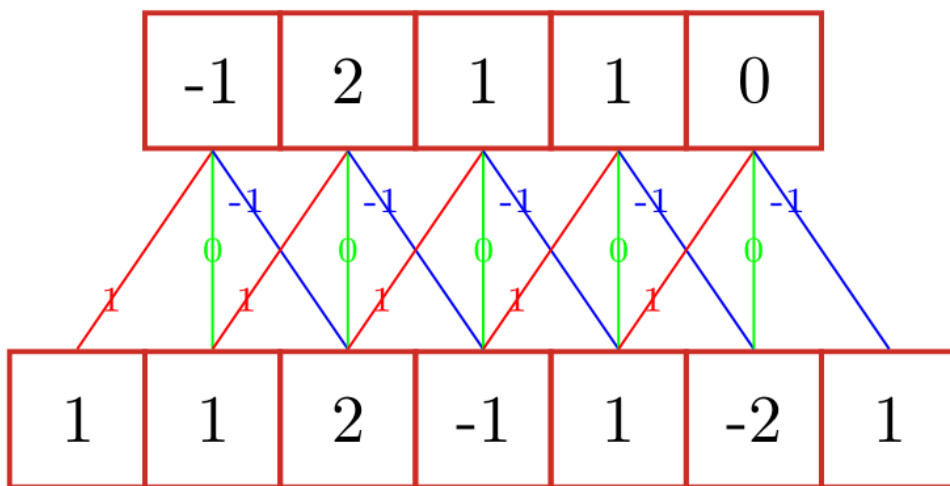
滤波器 (filter) 或
卷积核 (convolution kernel)

卷积



- 给定一个输入信号序列 x 和滤波器 w ，卷积的输出为：

$$y_t = \sum_{k=1}^K w_k x_{t-k+1}$$

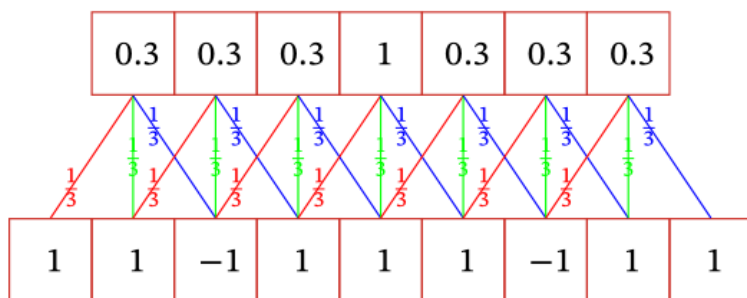


滤波器: $[-1, 0, 1]$

卷积

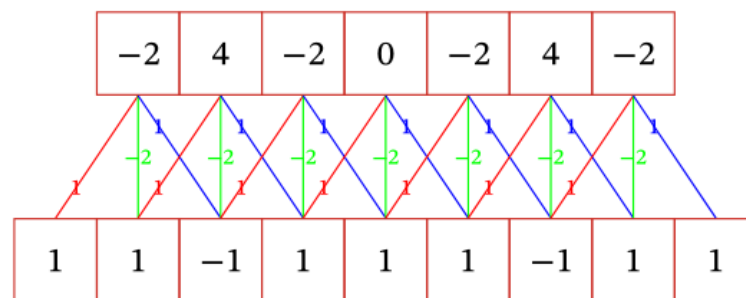


- 不同的滤波器来提取信号序列中的不同特征



(a) 滤波器 $[1/3, 1/3, 1/3]$

低频信息



(b) 滤波器 $[1, -2, 1]$

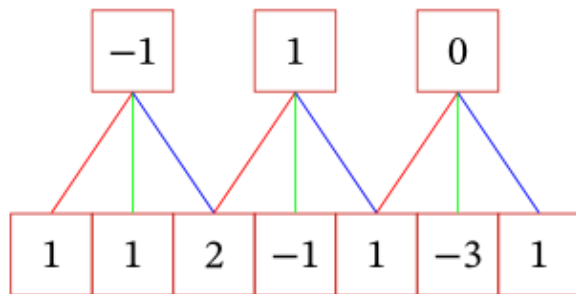
高频信息

近似信号序列的二阶微分

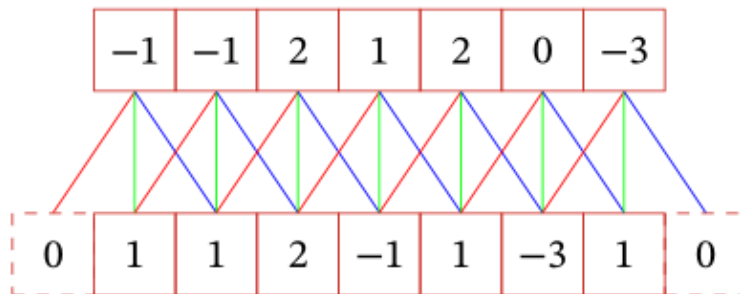
卷积扩展



- 引入滤波器的滑动步长 S 和零填充 P



(a) 步长 $S = 2$



(b) 零填充 $P = 1$

卷积类型



卷积的结果按输出长度不同可以分为三类：

1. **窄卷积**：步长 $T = 1$ ，两端不补零 $P = 0$ ，卷积后输出长度为 $M - K + 1$
2. **宽卷积**：步长 $T = 1$ ，两端补零 $P = K - 1$ ，卷积后输出长度 $M + K - 1$
3. **等宽卷积**：步长 $T = 1$ ，两端补零 $P = (K - 1)/2$ ，卷积后输出长度 M

在早期的文献中，卷积一般默认为窄卷积。
而目前的文献中，卷积一般默认为等宽卷积。

二维卷积



- 在图像处理中，图像是以二维矩阵的形式输入到神经网络中，因此我们需要二维卷积。

一个输入信息 $\mathbf{X}$ 和滤波器 $\mathbf{W}$ 的二维卷积定义为

$$\mathbf{Y} = \mathbf{W} * \mathbf{X},$$

$$y_{ij} = \sum_{u=1}^U \sum_{v=1}^V w_{uv} x_{i-u+1, j-v+1}$$

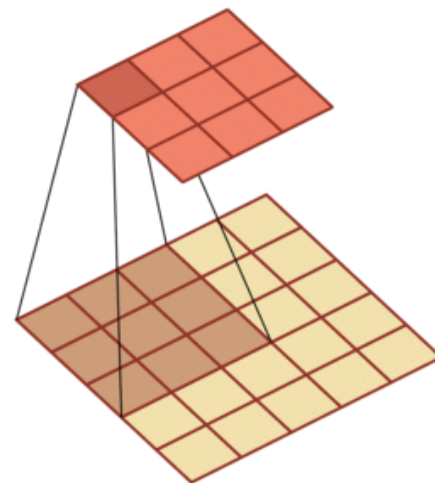
1	1	1	1	1
-1	0	-3	0	1
2	1	1	-1	0
0	-1	1	2	1
1	2	1	1	1

$\otimes$

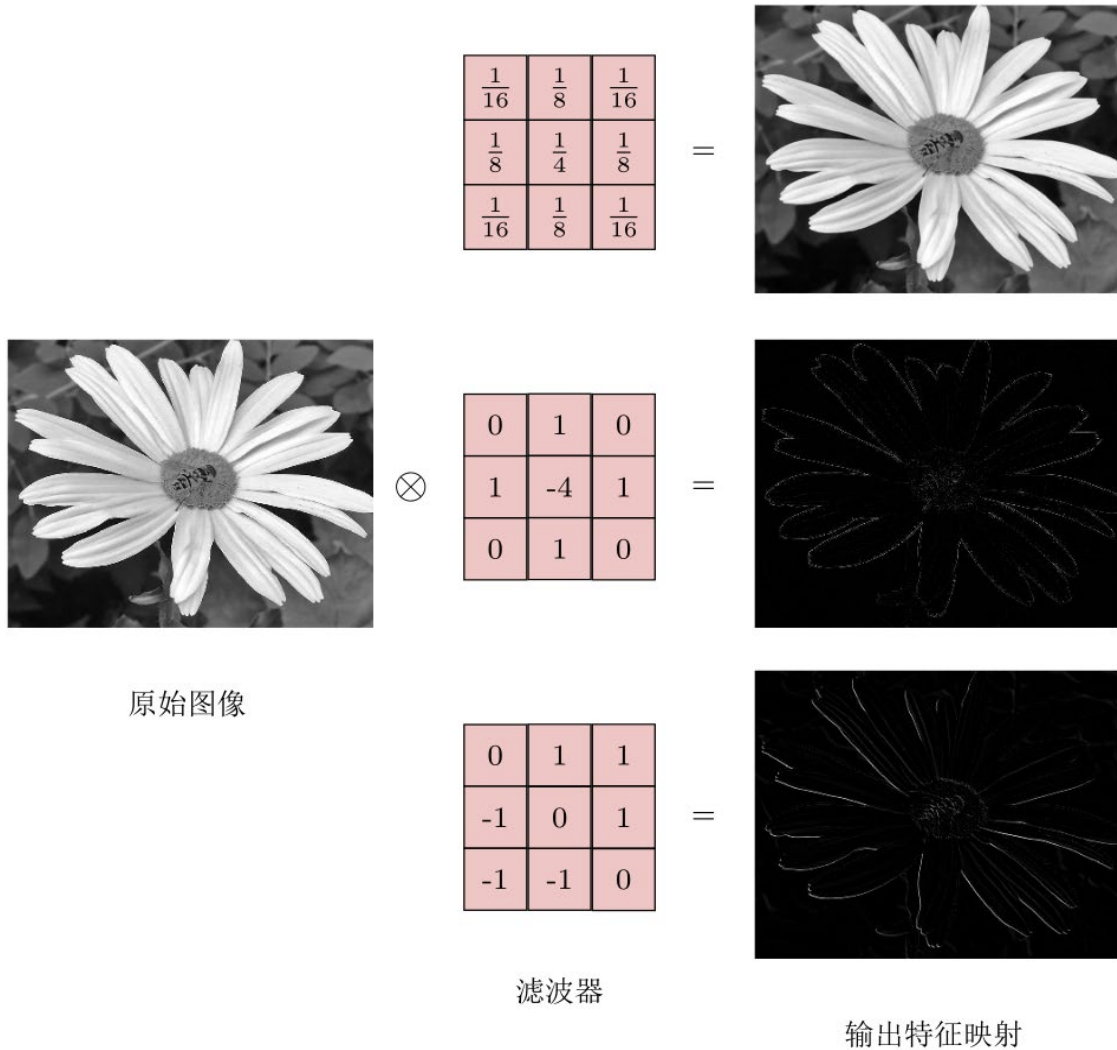
1	0	0
0	0	0
0	0	-1

$=$

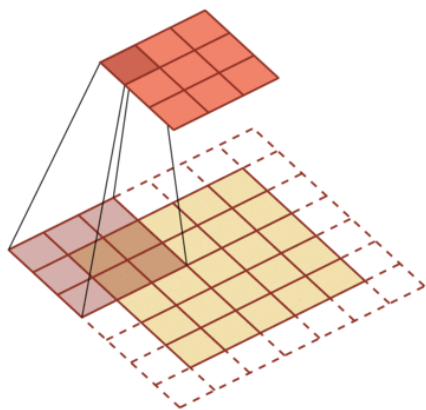
0	-2	-1
2	2	4
-1	0	0



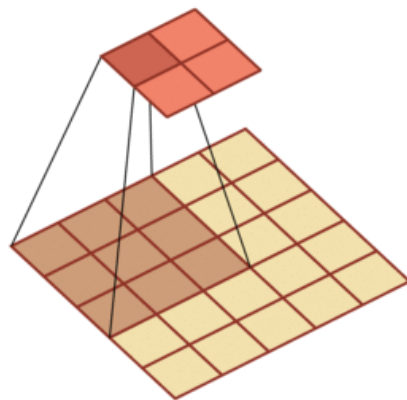
卷积作为特征提取器



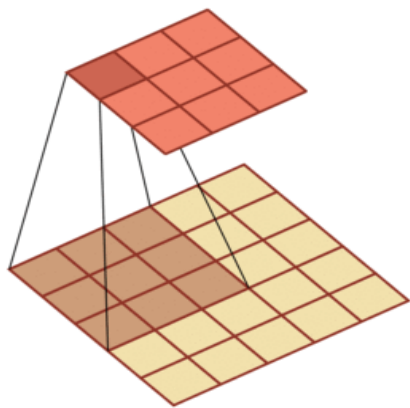
两维卷积



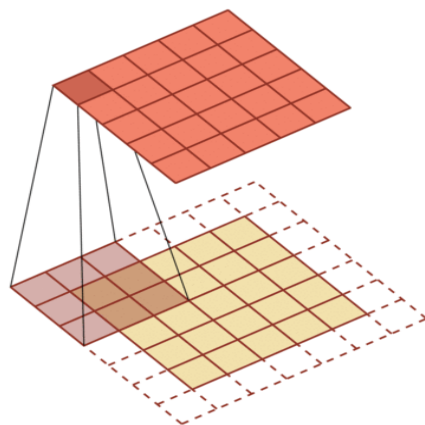
步长1，零填充0



步长2，零填充0

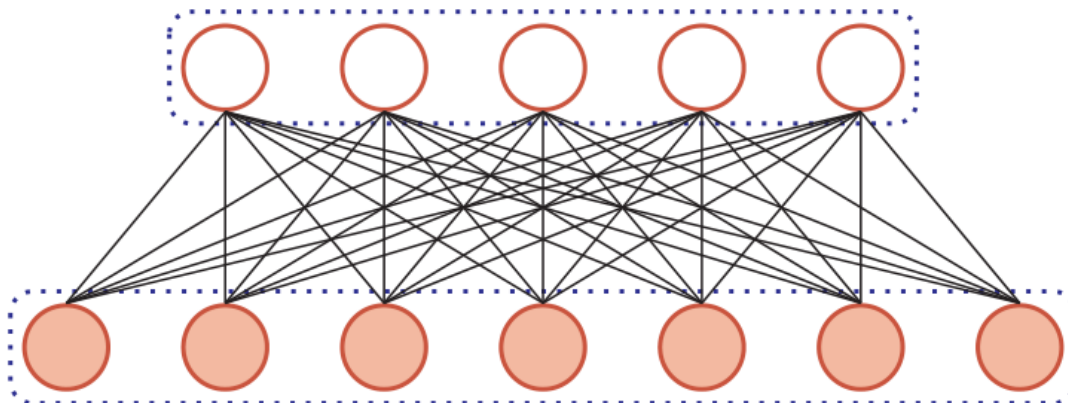


步长1，零填充1

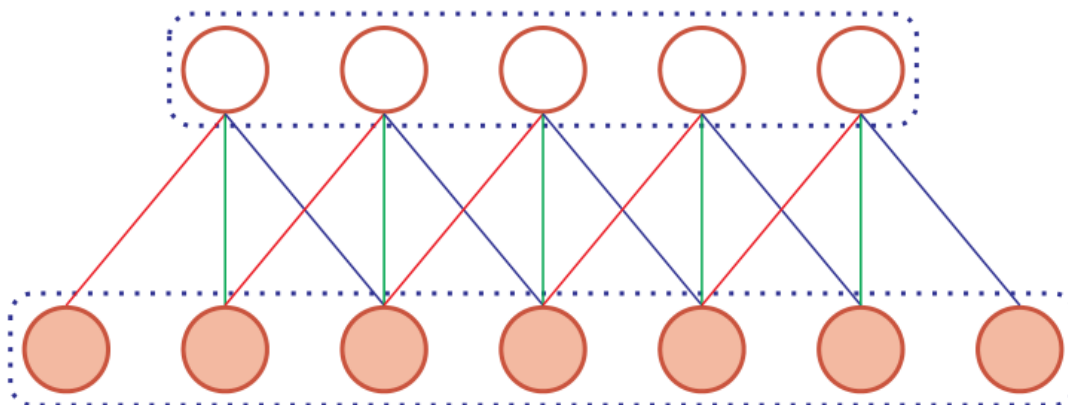


步长2，零填充1

用卷积来代替全连接



(a) 全连接层



(b) 卷积层

互相关



- 计算卷积需要进行卷积核**翻转**。

a, b 的下标相加都为1, 1

$$f = \begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} \\ a_{1,0} & a_{1,1} & a_{1,2} \\ a_{2,0} & a_{2,1} & a_{2,2} \end{bmatrix} \quad g = \begin{bmatrix} b_{-1,-1} & b_{-1,0} & b_{-1,1} \\ b_{0,-1} & b_{0,0} & b_{0,1} \\ b_{1,-1} & b_{1,0} & b_{1,1} \end{bmatrix}$$

- 卷积操作的目标：**提取特征**。

$$c_{1,1} = a_{0,0}b_{1,1}$$

翻转是不必要的！

- 互相关

输入		核函数		输出																	
<table border="1"><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table border="1"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table border="1"><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

图中的输入数组的高和宽分别为 2，其中的 4 个元素由二维互相关运算得出：

$$\begin{aligned} 0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 &= 19 \\ 1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 &= 25 \\ 3 \times 0 + 4 \times 1 + 6 \times 2 + 7 \times 3 &= 37 \\ 4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 &= 43 \end{aligned}$$

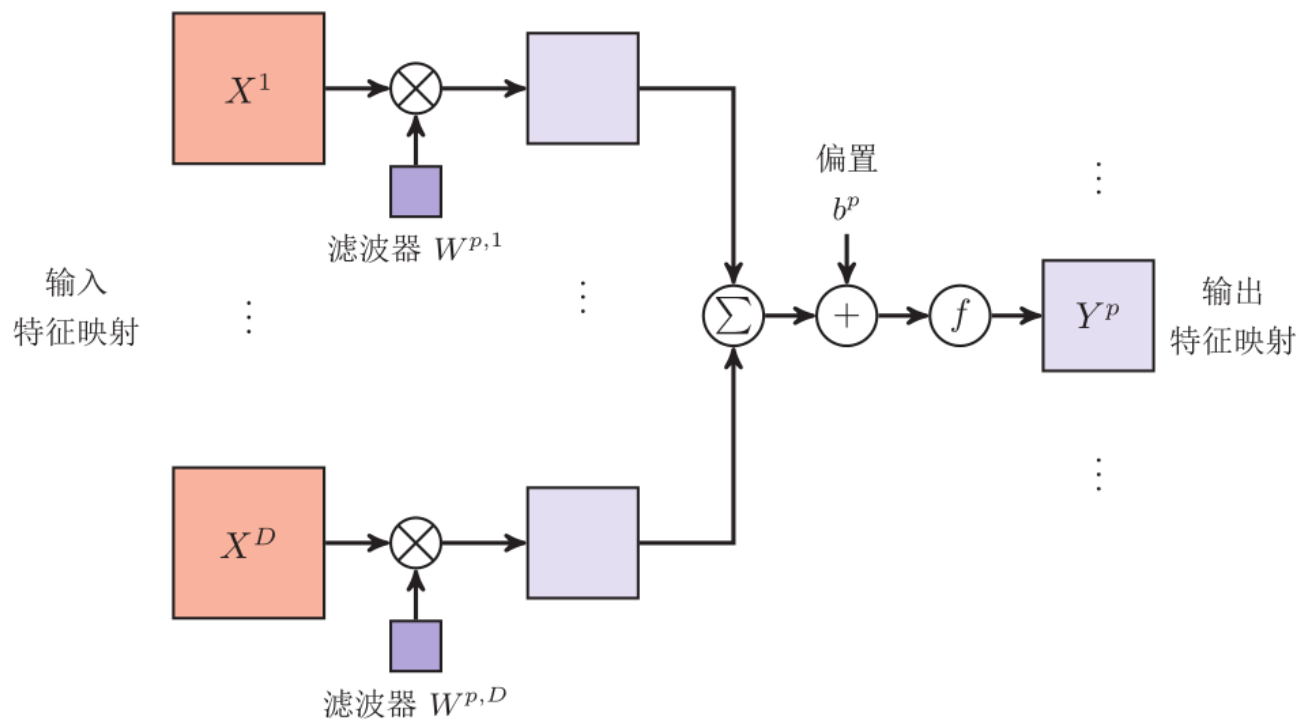
除非特别声明，卷积一般指“互相关”。

多个卷积核



- 卷积核看成一个特征提取器。
 - 如何增强卷积层的能力？
 - 引入多个卷积核
- 特征映射（Feature Map）：图像经过卷积后得到的特征。
- 卷积层
 - 输入：D个特征映射 $M \times N \times D$
 - 输出：P个特征映射 $M' \times N' \times P$

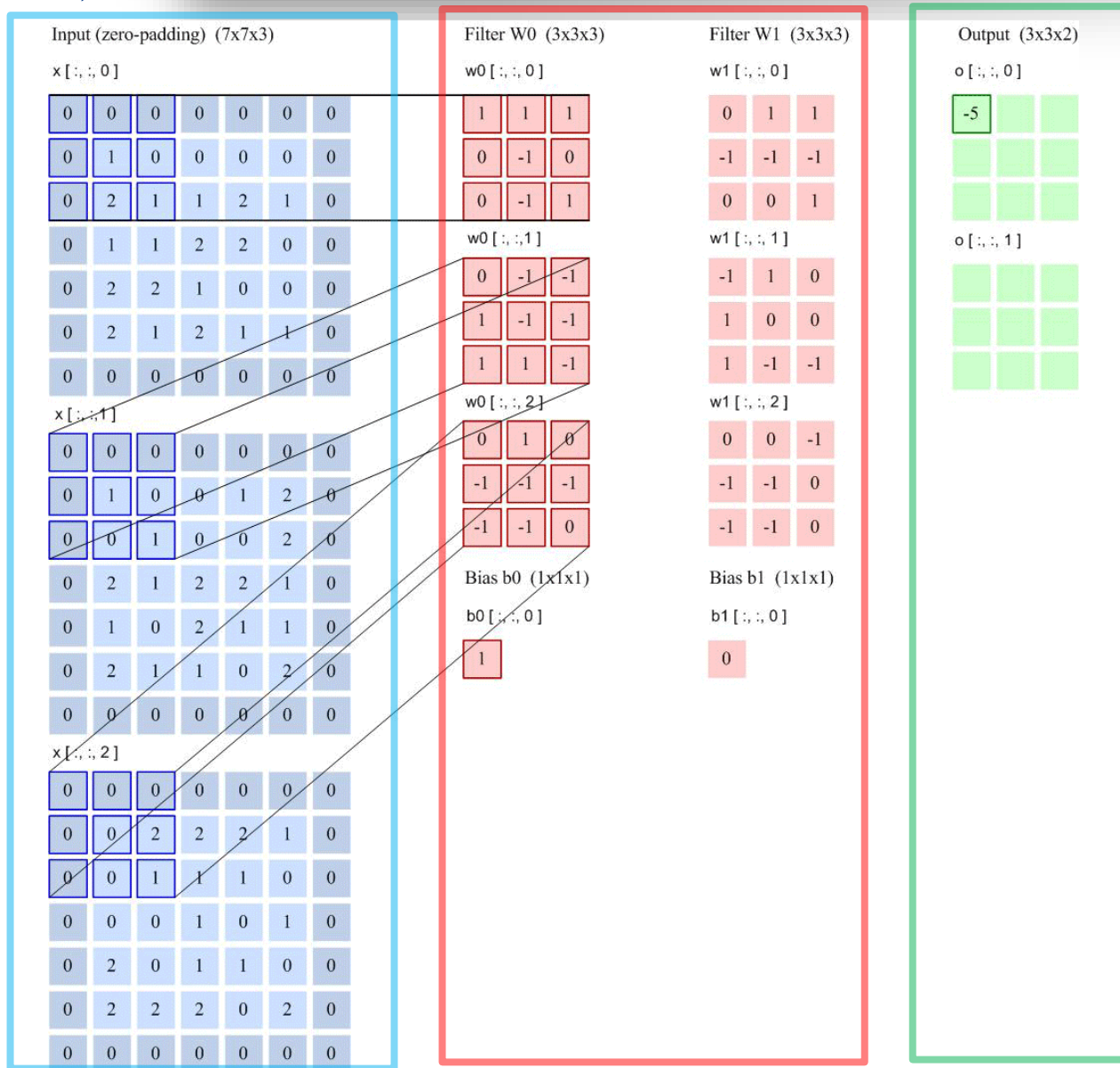
卷积层的映射关系



$$\mathbf{Z}^p = \mathbf{W}^p \otimes \mathbf{X} + b^p = \sum_{d=1}^D \mathbf{W}^{p,d} \otimes \mathbf{X}^d + b^p,$$

$$\mathbf{Y}^p = f(\mathbf{Z}^p).$$

卷积

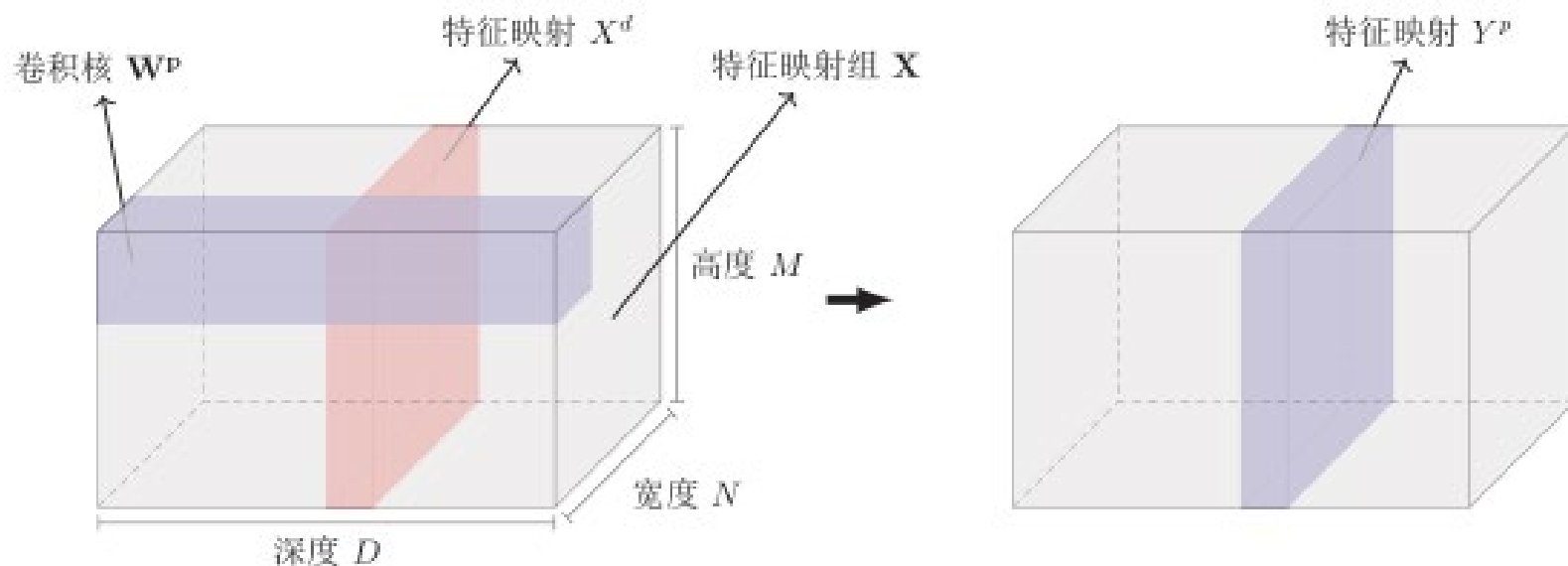


步长 2
卷积核 3*3
卷积核个数 6
零填充 1

卷积层



◆ 典型的卷积层为3维结构



$$Z^p = \mathbf{W}^p \otimes \mathbf{X} + b^p = \sum_{d=1}^D W^{p,d} \otimes X^d + b^p,$$

$$Y^p = f(Z^p).$$

汇聚层



- 卷积层虽然可以显著减少连接的个数，但是每一个特征映射的神经元个数并没有显著减少。



输入特征映射组 $\mathbf{X}$

1	1	2	0
0	1	1	2
0	0	1	3
0	0	1	1

输入特征映射 X^d

max pooling



输出特征映射组 $\mathbf{Y}$

1	2
0	3

输出特征映射 Y^d

卷积网络结构

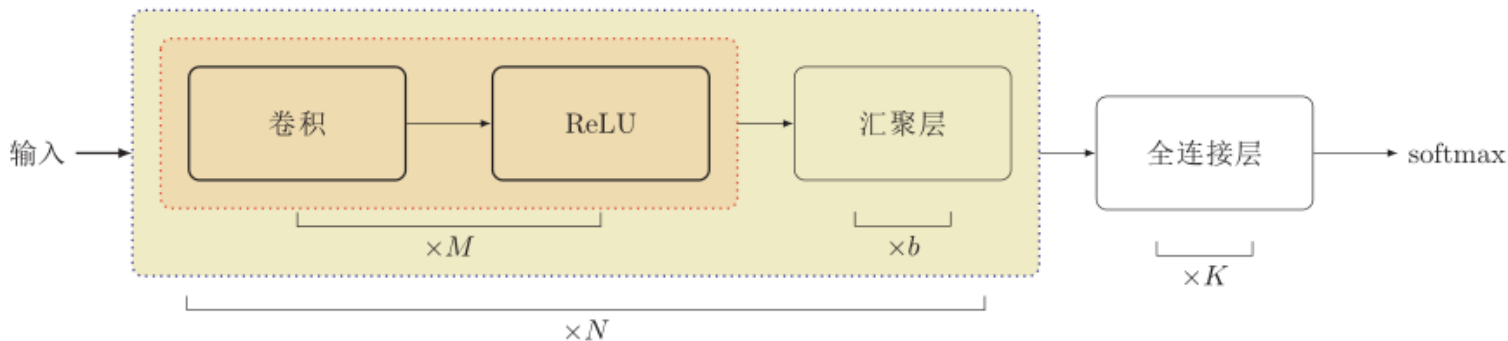


卷积网络是由卷积层、子采样层、全连接层交叉堆叠而成。

趋向于小卷积、大深度

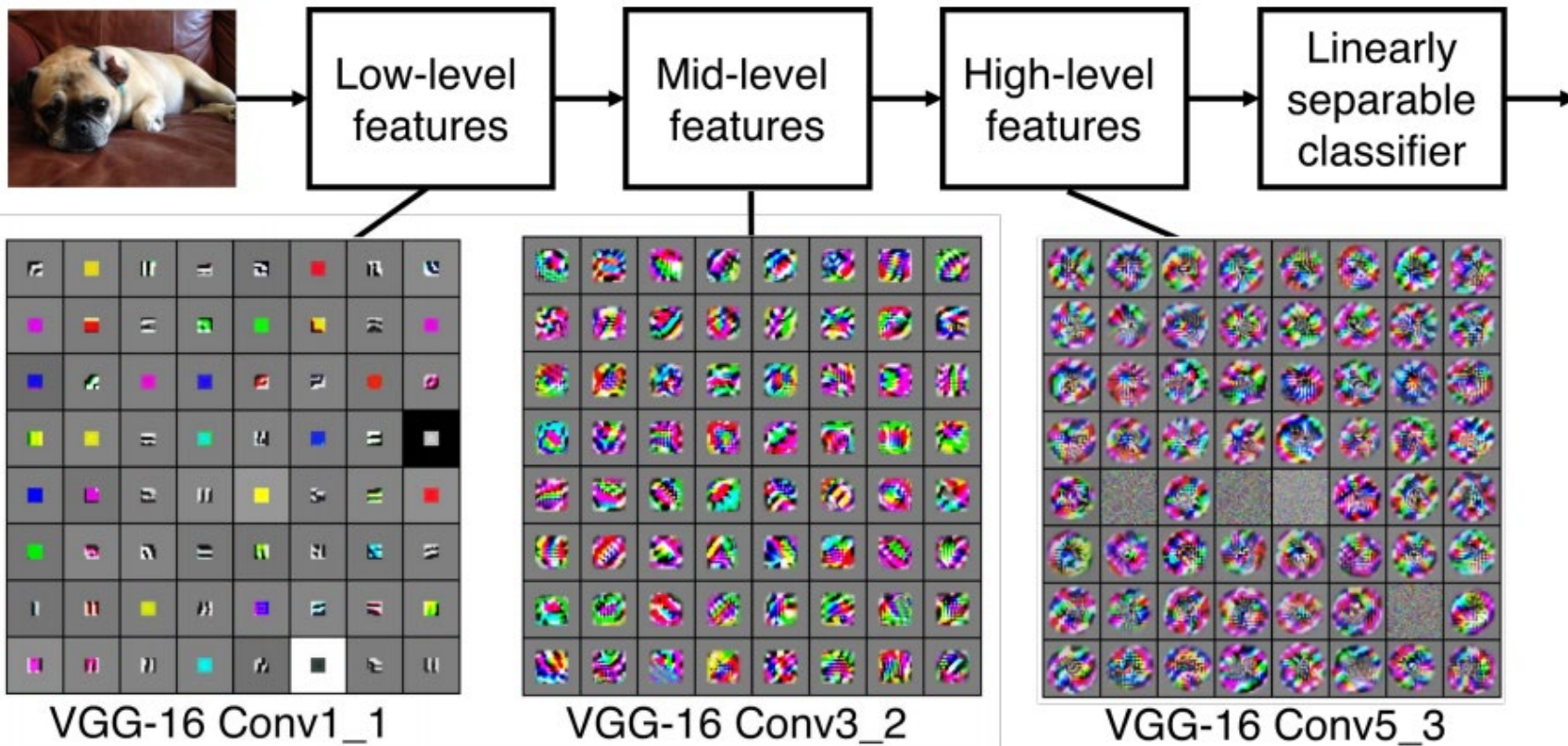
趋向于全卷积

● 典型结构

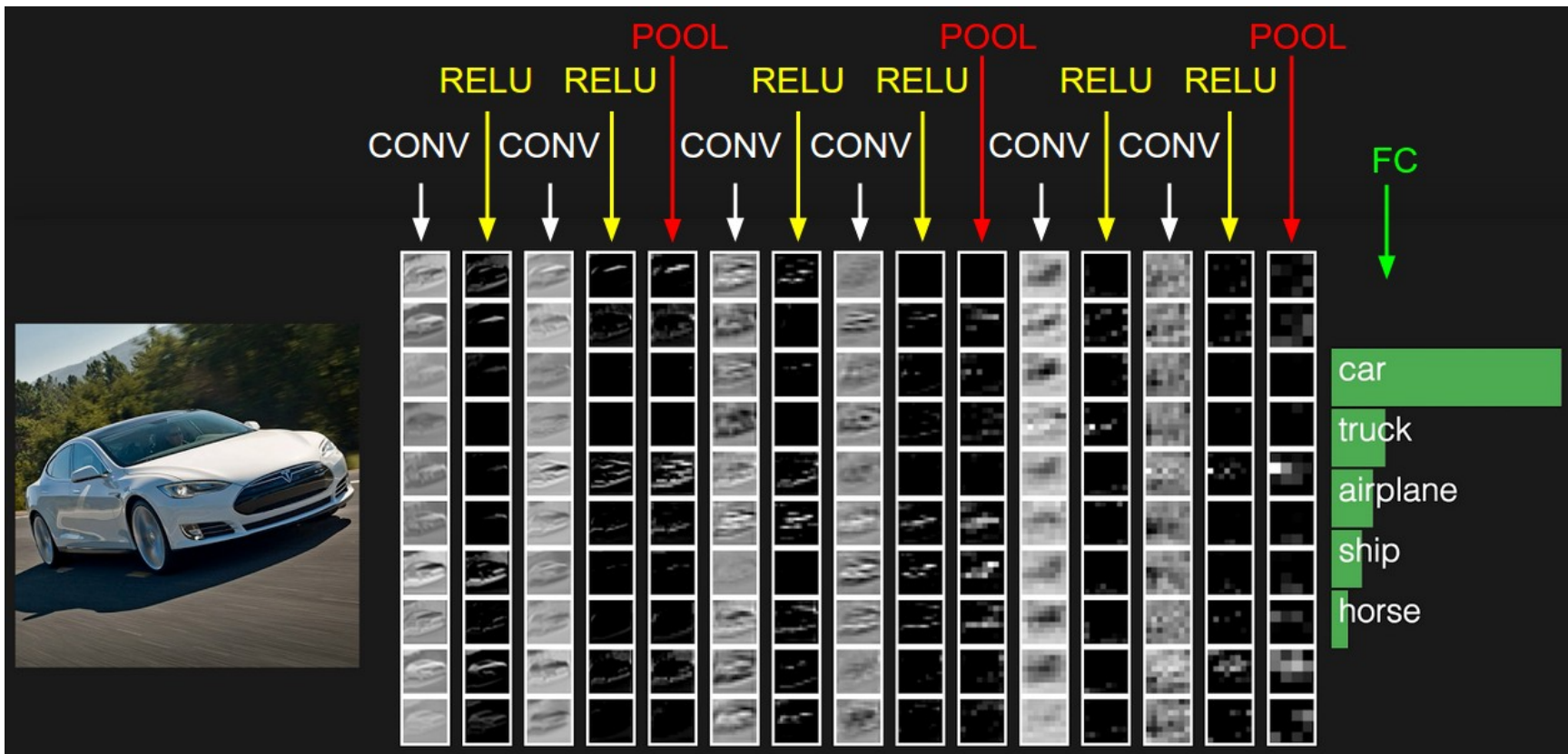


一个卷积块为连续 M 个卷积层和 b 个汇聚层（ M 通常设置为2~5， b 为0或1）。一个卷积网络中可以堆叠 N 个连续的卷积块，然后在接着 K 个全连接层（ N 的取值区间比较大，比如1~100或者更大； K 一般为0~2）。

表示学习



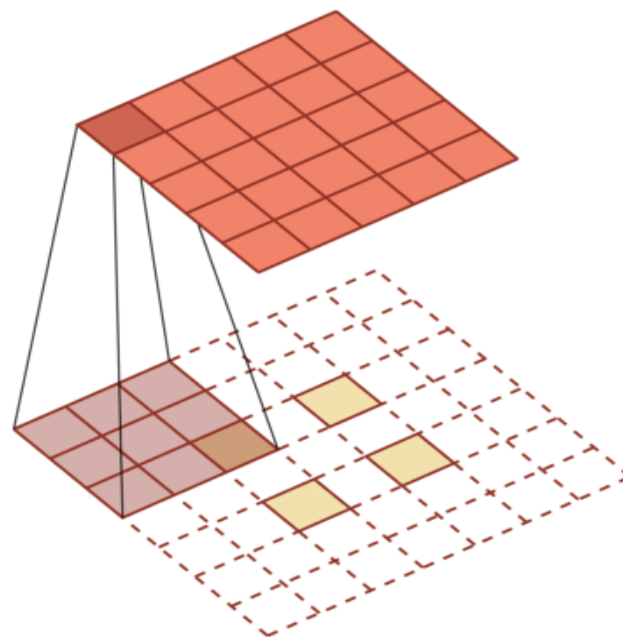
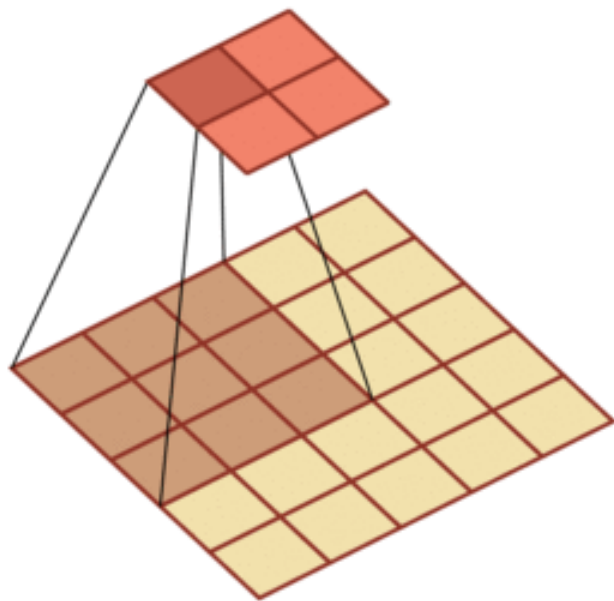
表示学习



转置卷积/微步卷积



- 低维特征映射到高维特征

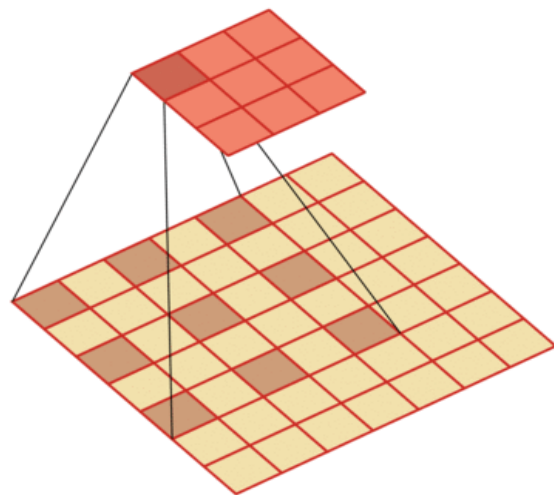


空洞卷积



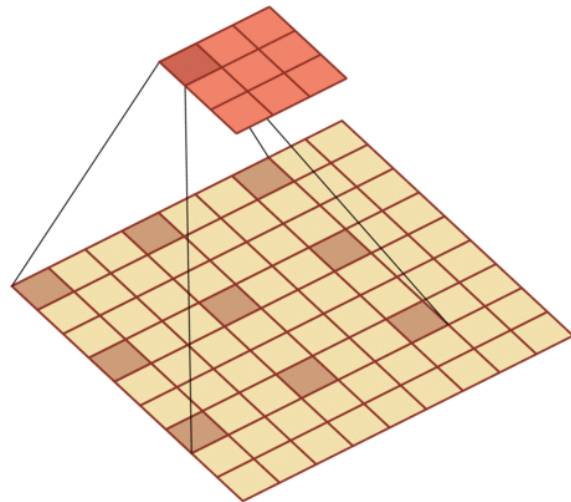
- 如何增加输出单元的感受野

- ① 增加卷积核的大小
- ② 增加层数来实现
- ③ 在卷积之前进行汇聚操作



- 空洞卷积

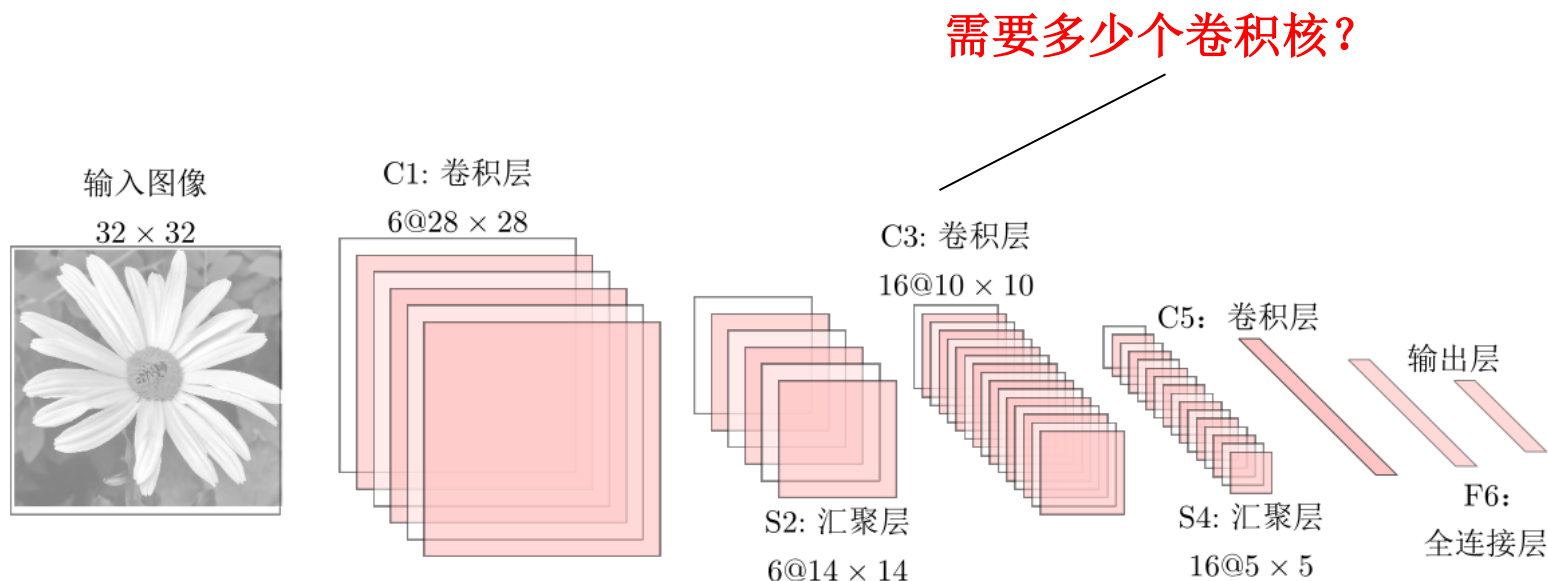
通过给卷积核插入“空洞”来变相地增加其大小。



LeNet-5



- LeNet-5 是一个非常成功的神经网络模型。
 - 基于 LeNet-5 的手写数字识别系统在 90 年代被美国很多银行使用，用来识别支票上面的手写数字。
 - LeNet-5 共有 7 层。



Large Scale Visual Recognition Challenge

2012 Teams	%error	2013 Teams	%error	2014 Teams	%error
Supervision (Toronto)	15.3	Clarifai (NYU spinoff)	11.7	GoogLeNet	6.6
ISI (Tokyo)	26.1	NUS (singapore)	12.9	VGG (Oxford)	7.3
VGG (Oxford)	26.9	Zeiler-Fergus (NYU)	13.5	MSRA	8.0
XRCE/INRIA	27.0	A. Howard	13.5	A. Howard	8.1
UvA (Amsterdam)	29.6	OverFeat (NYU)	14.1	DeeperVision	9.5
INRIA/LEAR	33.4	UvA (Amsterdam)	14.2	NUS-BST	9.7
		Adobe	15.2	TTIC-ECP	10.2
		VGG (Oxford)	15.2	XYZ	11.2
		VGG (Oxford)	23.0	UvA	12.1





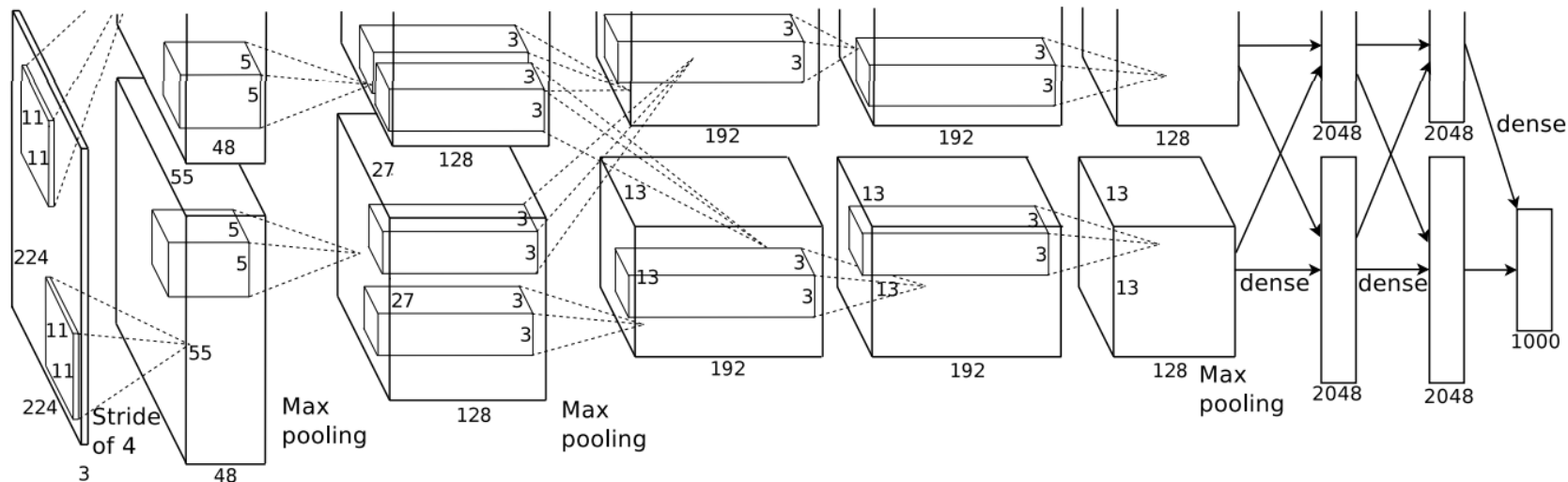
- 2012 ILSVRC winner

(top 5 error of 16\% compared to runner-up with 26\% error)

首次使用了很多现代深度卷积网络的技术方法

- 使用GPU进行并行训练，采用了ReLU作为非线性激活函数，使用Dropout防止过拟合，使用数据增强

共有8层，其中前5层卷积层，后边3层全连接层



Inception网络



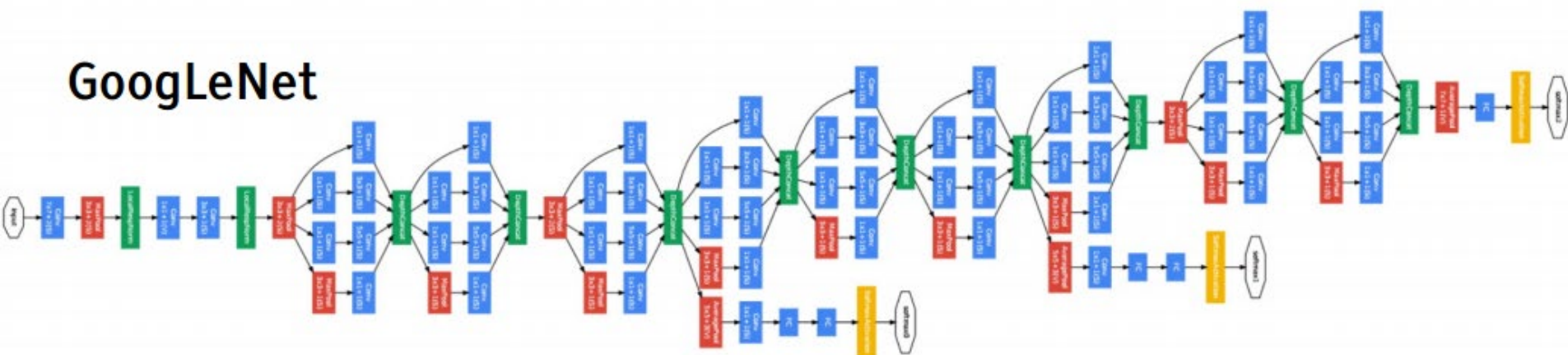
- 2014 ILSVRC winner (22层)

参数: GoogLeNet: 4M **VS** AlexNet: 60M

错误率: 6.7%

Inception网络由多个inception模块和少量的汇聚层堆叠而成

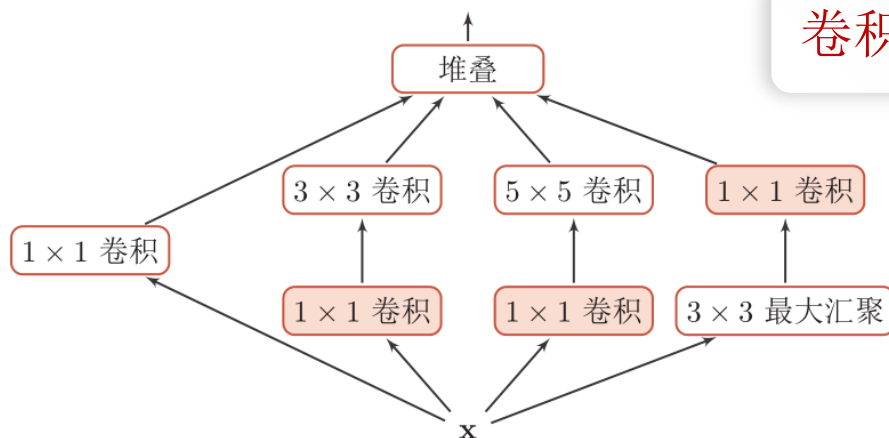
GoogLeNet



Inception模块 v1



- 在卷积网络中，如何设置卷积层的卷积核大小是一个十分关键的问题。
 - 在Inception网络中，一个卷积层包含多个不同大小的卷积操作，称为Inception模块。
 - Inception模块同时使用 1×1 、 3×3 、 5×5 等不同大小的卷积核，并将得到的特征映射在深度上拼接（堆叠）起来作为输出特征映射。



卷积和最大汇聚都是等宽的。

Inception模块 v3



- 用多层的小卷积核来替换大的卷积核，以减少计算量和参数量。
 - 使用两层 3×3 的卷积来替换v1中的 5×5 的卷积
 - 使用连续的 $n \times 1$ 和 $1 \times n$ 来替换 $n \times n$ 的卷积。

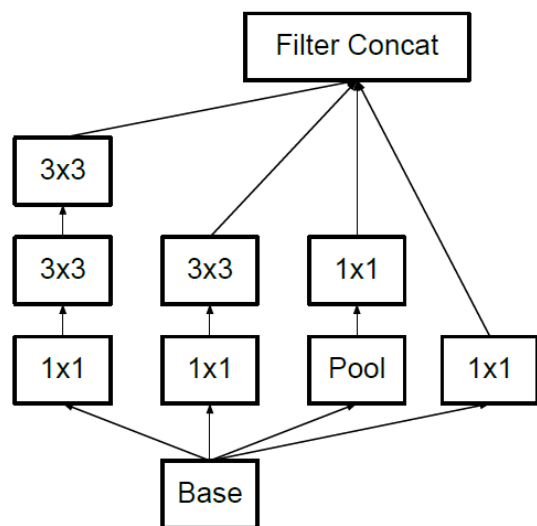


Figure 5. Inception modules where each 5×5 convolution is replaced by two 3×3 convolution, as suggested by principle 3 of Section 2.

<http://blog.csdn.net/xbinworld>

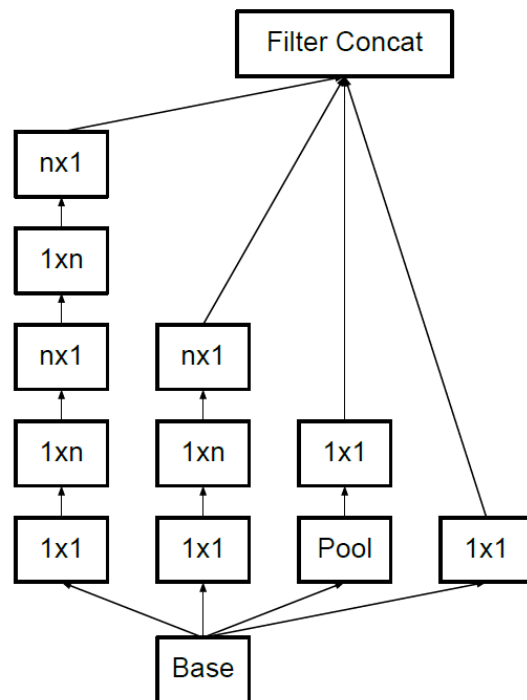


Figure 6. Inception modules after the factorization of the $n \times n$ convolutions. In our proposed architecture, we chose $n = 7$ for the 17×17 grid. (The filter sizes are picked using principle 3)

<http://blog.csdn.net/xbinworld>

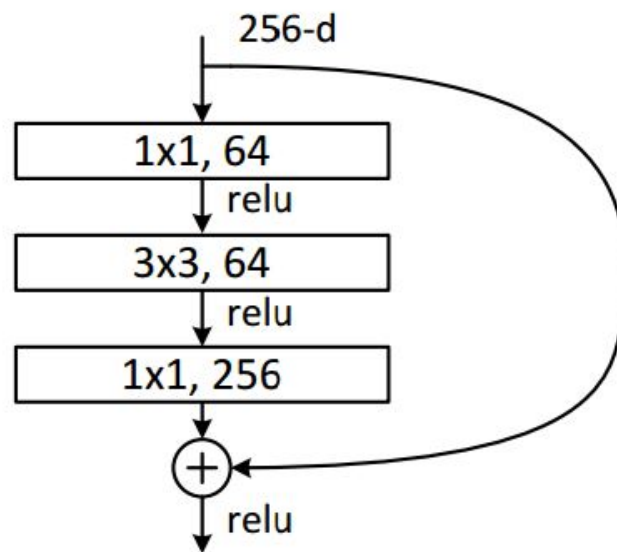
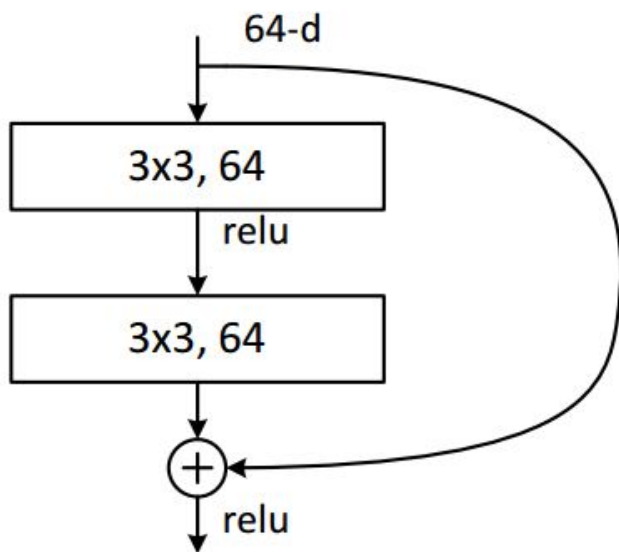
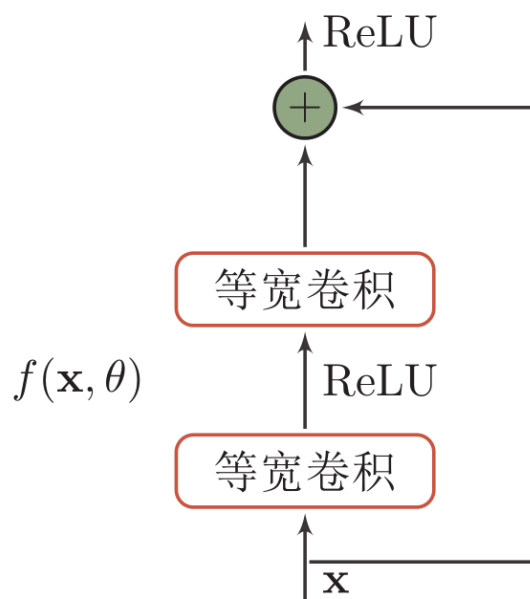
残差网络



- 残差网络（Residual Network, ResNet）是通过给非线性的卷积层增加直连边的方式来提高信息的传播效率。
- 假设在一个深度网络中，我们期望一个非线性单元（可以为一层或多层的卷积层） $f(x, \theta)$ 去逼近一个目标函数为 $h(x)$ 。
- 将目标函数拆分成两部分：恒等函数和残差函数

$$h(\mathbf{x}) = \underbrace{\mathbf{x}}_{\text{恒等函数}} + \underbrace{(h(\mathbf{x}) - \mathbf{x})}_{\text{残差函数}} \rightarrow f(\mathbf{x}, \theta)$$

残差单元

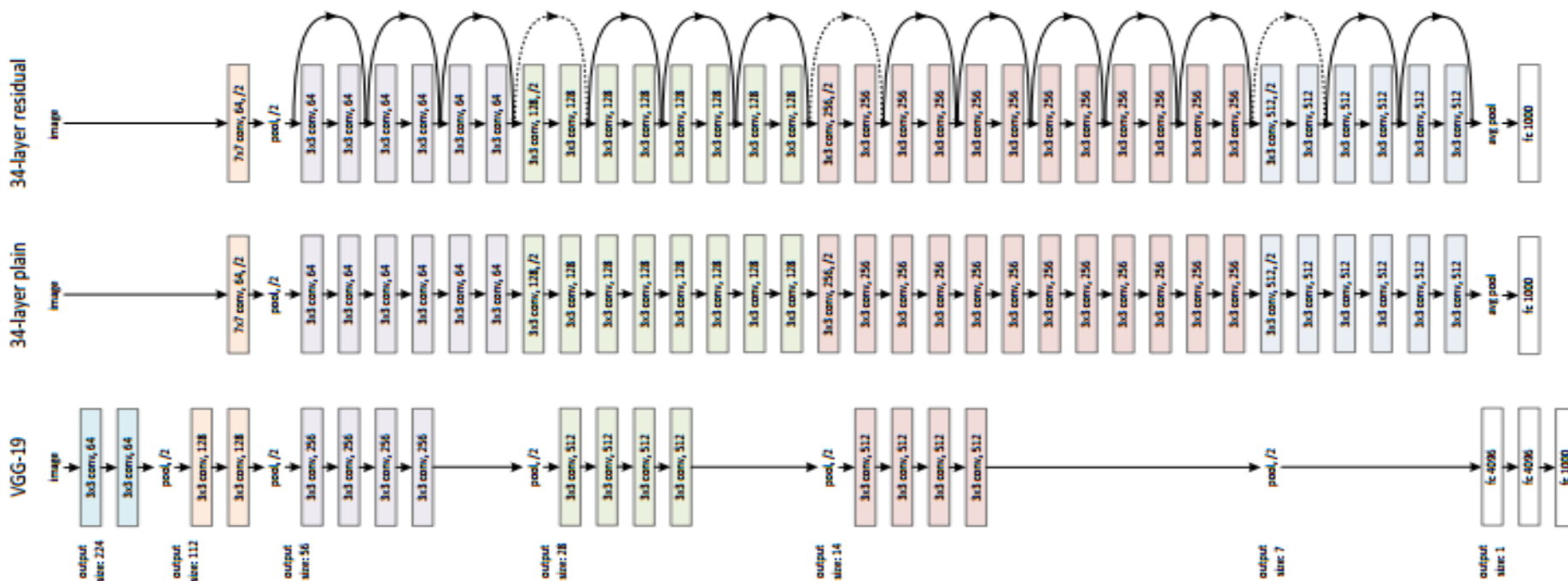


ResNet

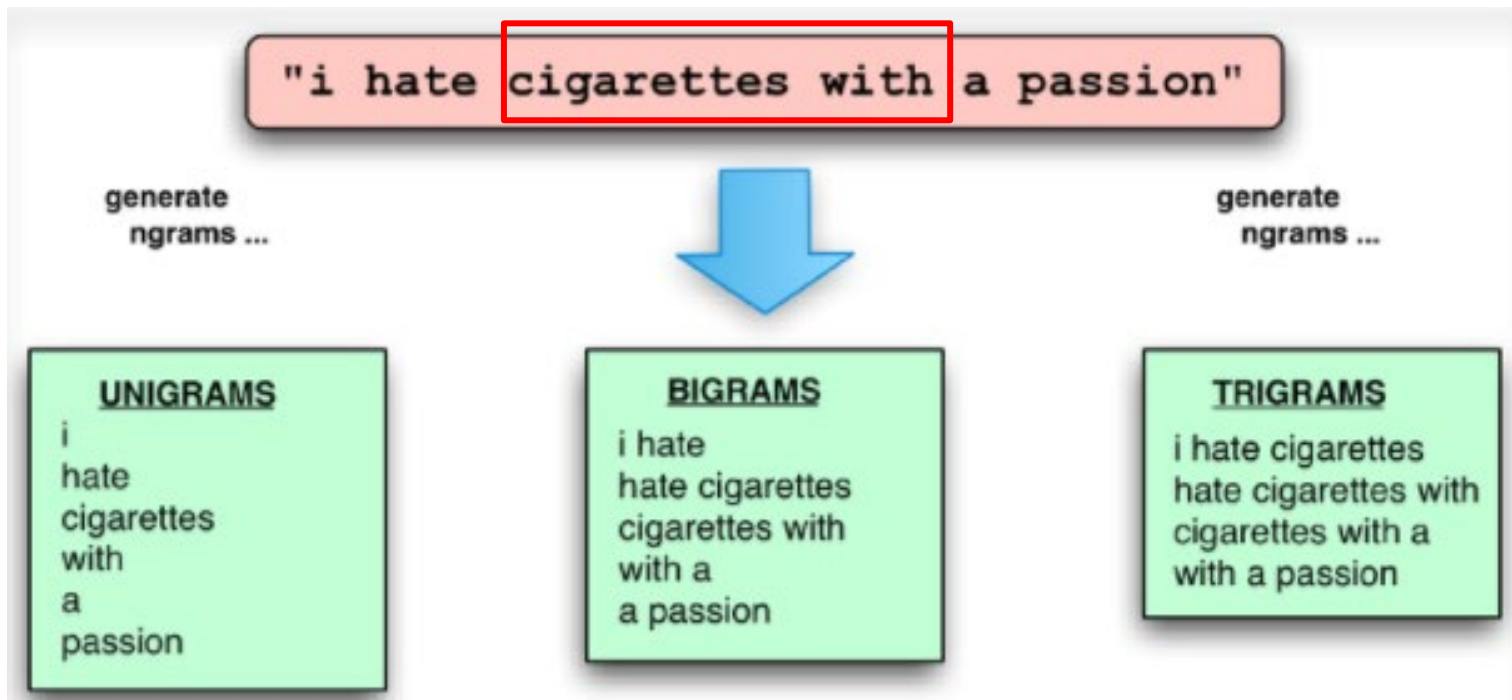


- 2015 ILSVRC winner (152层)

错误率: 3.57%

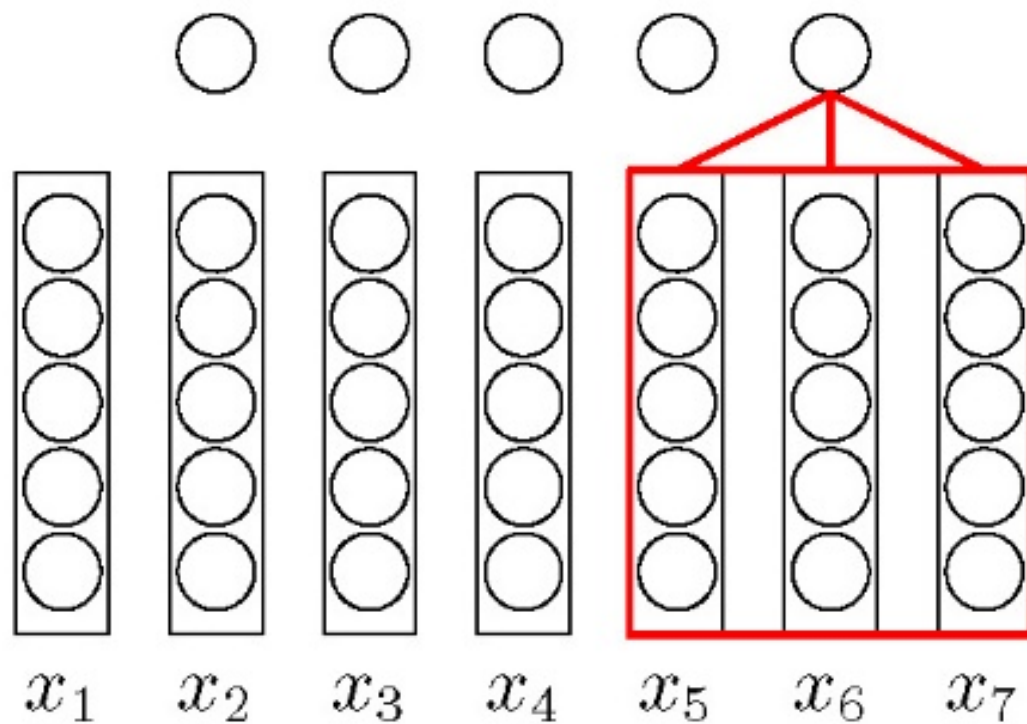


Ngram特征与卷积

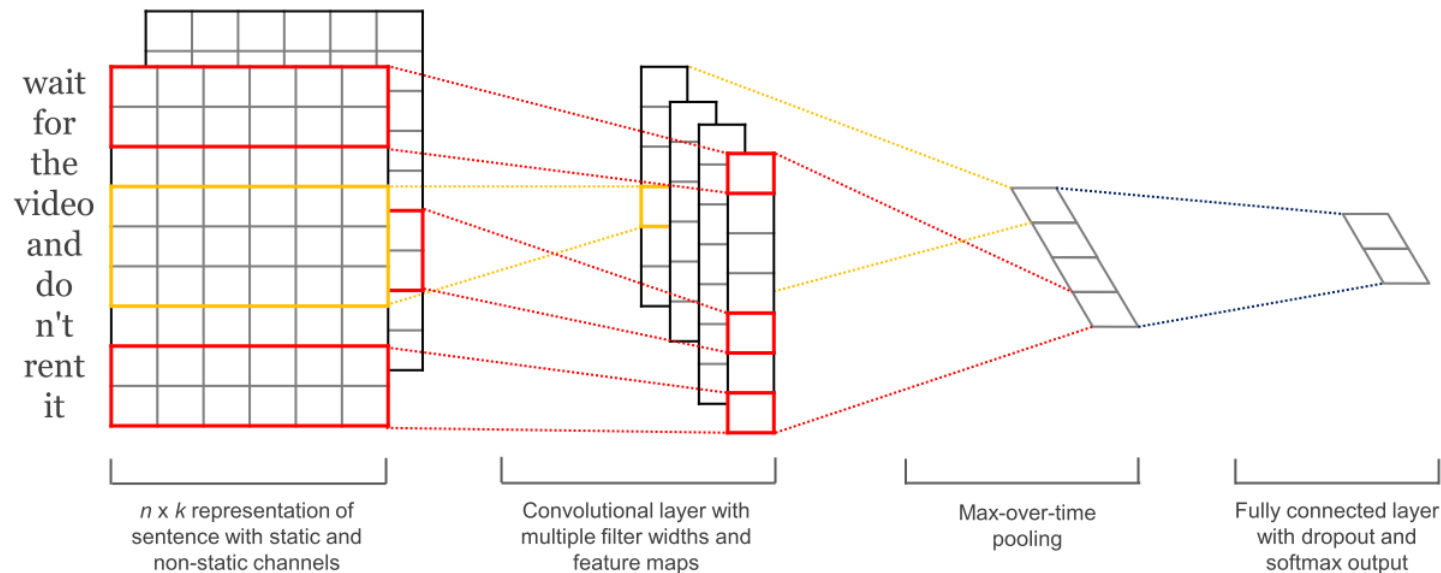


如何用卷积操作来实现？

文本序列的卷积

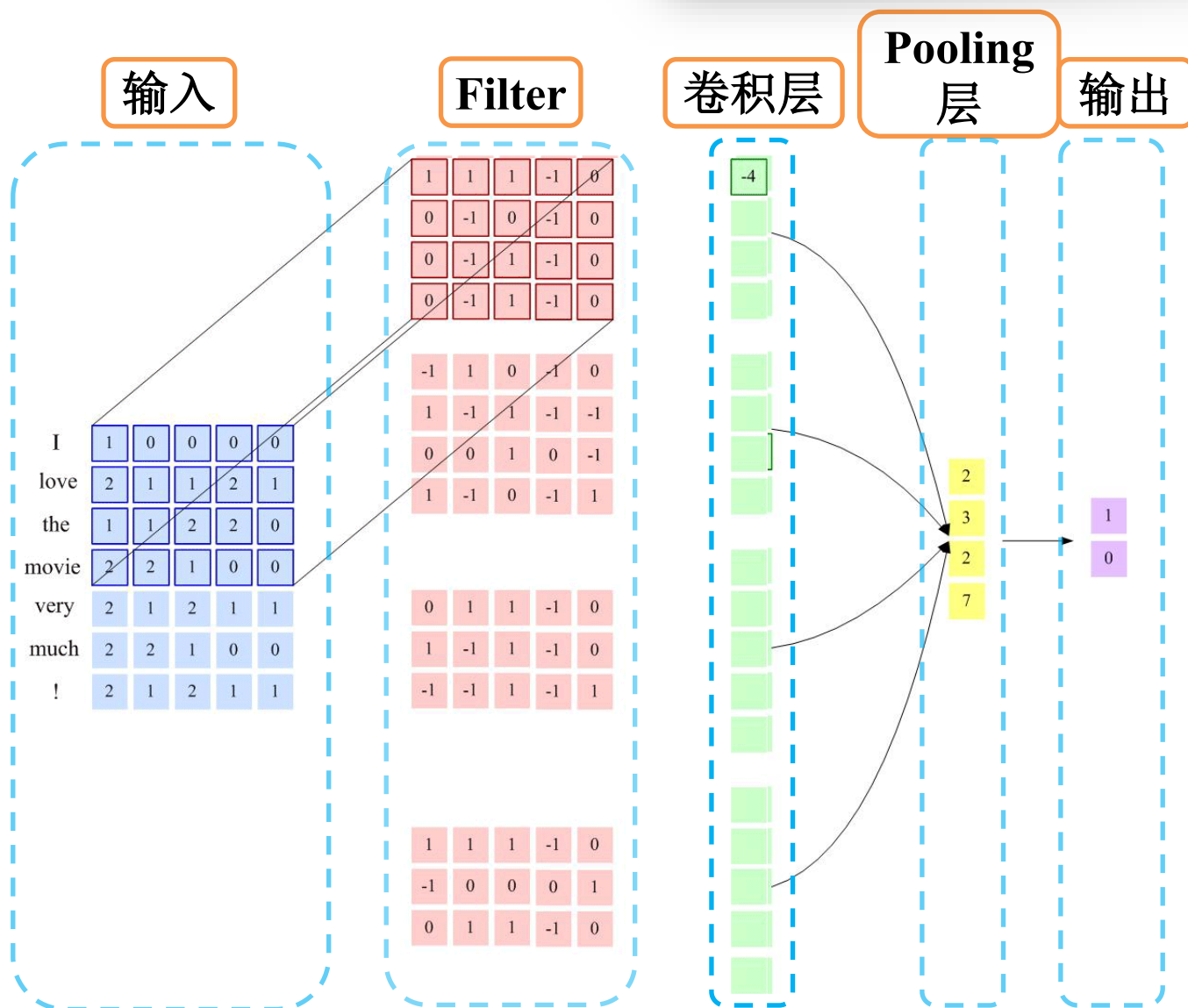


基于卷积模型的句子表示



Y. Kim. "Convolutional neural networks for sentence classification". In: *arXiv preprint arXiv:1408.5882* (2014).

文本序列的卷积模型



CNN可视化：滤波器



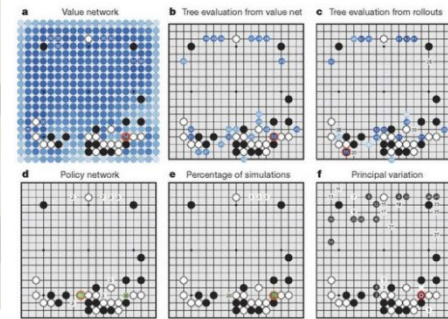
- AlexNet中的滤波器（96 filters [11x11x3]）



AlphaGo



- 分布式系统：1202 个CPU 和176 块GPU
- 单机版：48 个CPU 和8 块GPU
- 走子速度：3 毫秒-2 微秒



The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image, then convolves k filters of kernel size 5×5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21×21 image, then convolves k filters of kernel size 3×3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1×1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used $k = 192$ filters; Fig. 2b and Extended Data Table 3 additionally show the results of training with $k = 128, 256$ and 384 filters.

policy network:

[19x19x48] Input

CONV1: 192 5x5 filters , stride 1, pad 2 => [19x19x192]

CONV2..12: 192 3x3 filters, stride 1, pad 1 => [19x19x192]

CONV: 1 1x1 filter, stride 1, pad 0 => [19x19] (*probability map of promising moves*)

目标检测

Object Detection

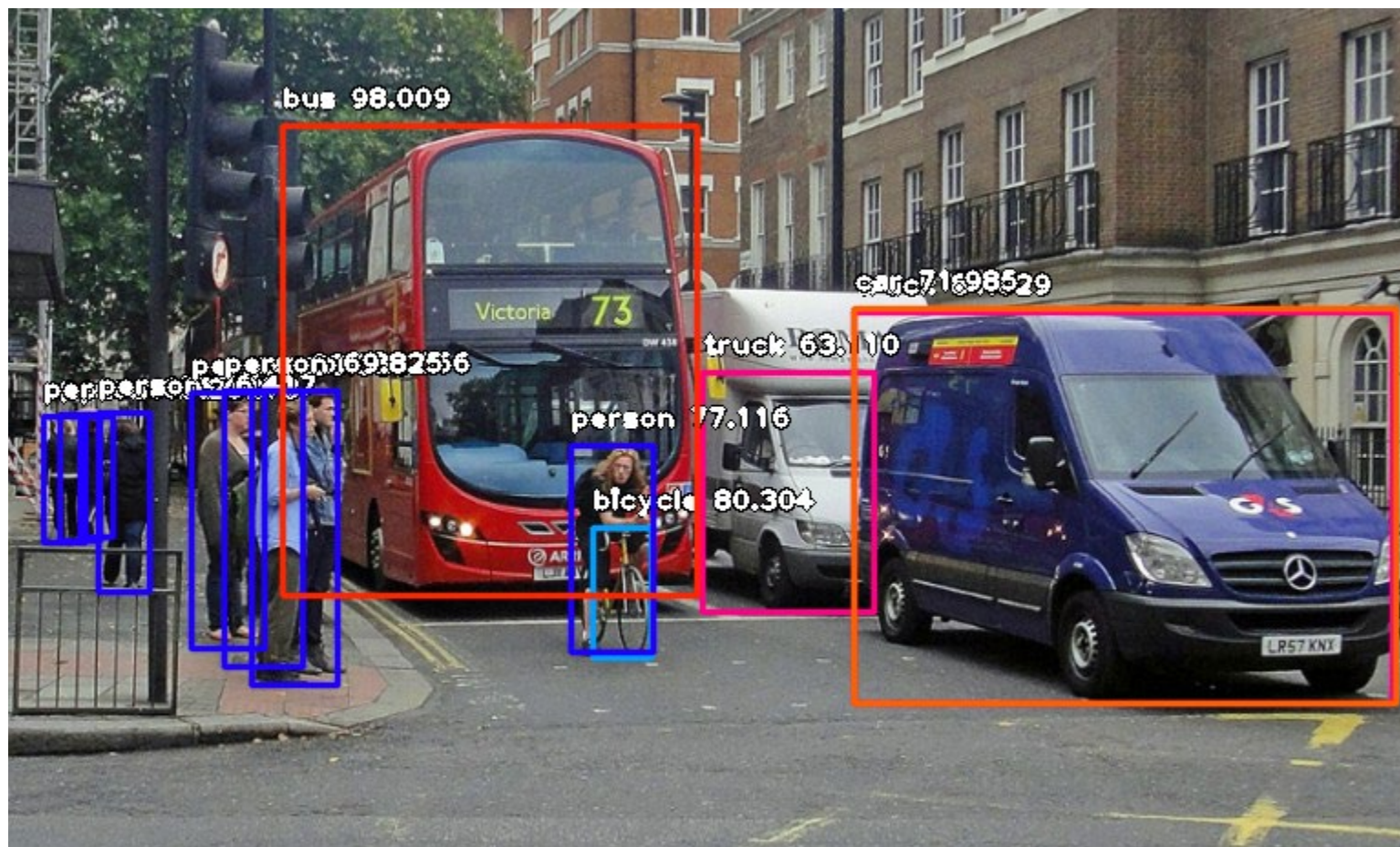




Figure 4. More results of **Mask R-CNN** on COCO test images, using ResNet-101-FPN and running at 5 fps, with 35.7 mask AP (Table 1).

OCR



图像生成



	是	在	中	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
is	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
at	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
in	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
not	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
(past tense)	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
country	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
on	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
have	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
large	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
for	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
year	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
this	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
(individual)	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
out	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
time	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
minute	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
people	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
city	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
do	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到
to	是	在	是	不	了	国	上	有	大	为	年	这	个	出	时	分	人	市	行	到

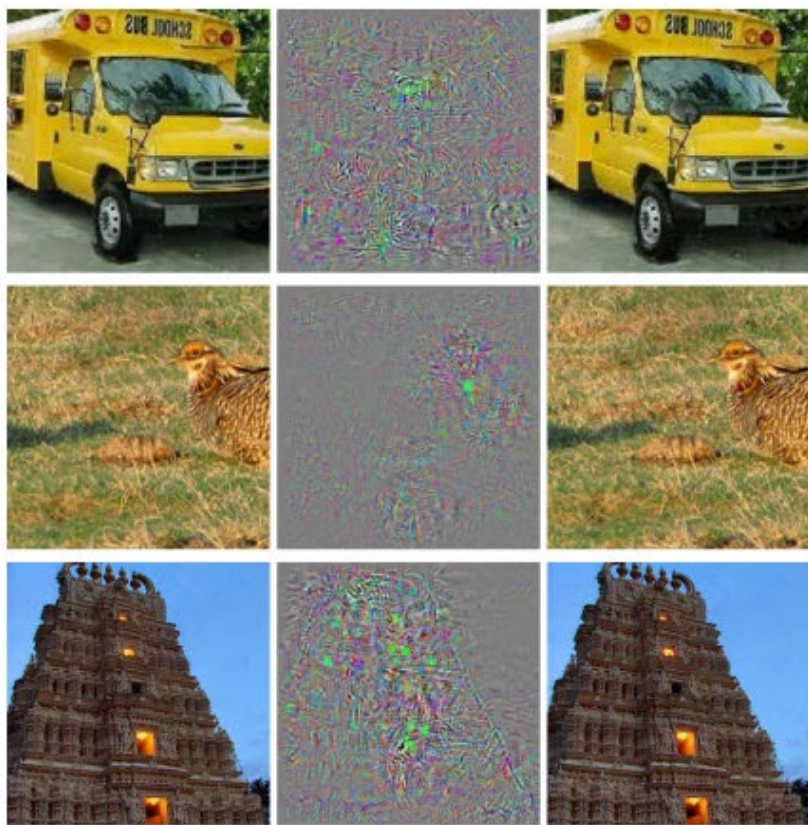
Deep Dream



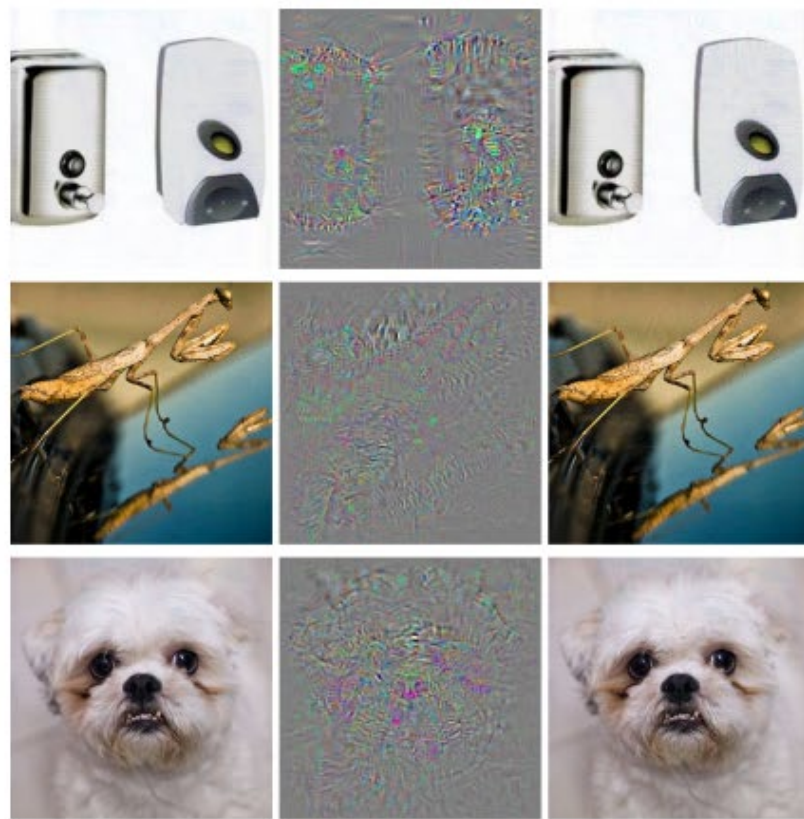
画风迁移



对抗样本



(a)

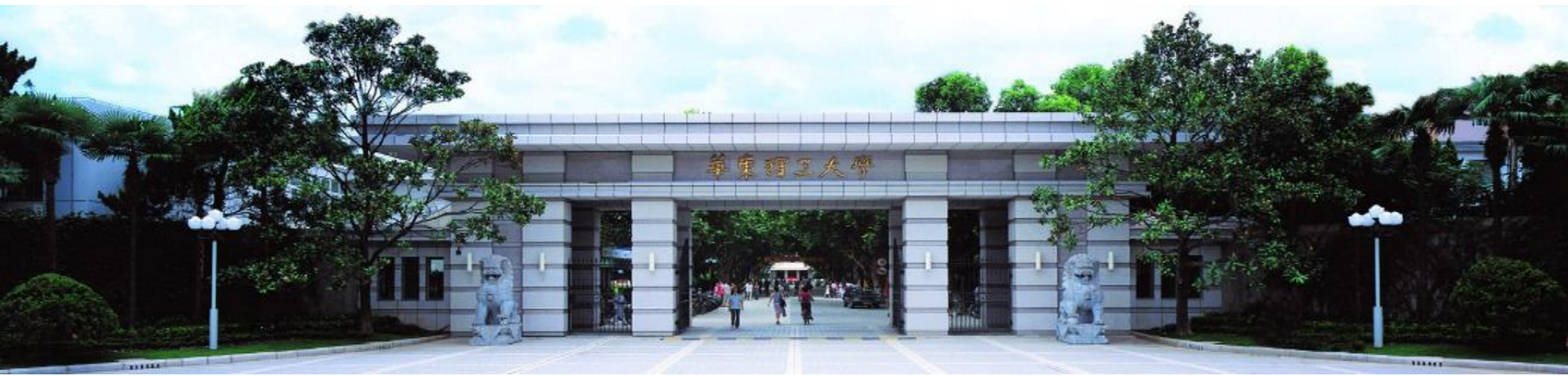


(b)



循环神经网络

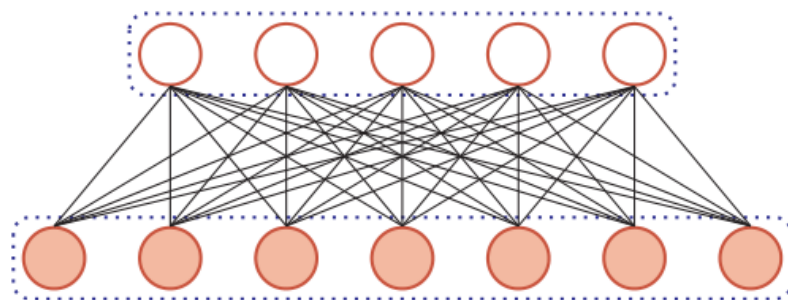
(课后自行复习)



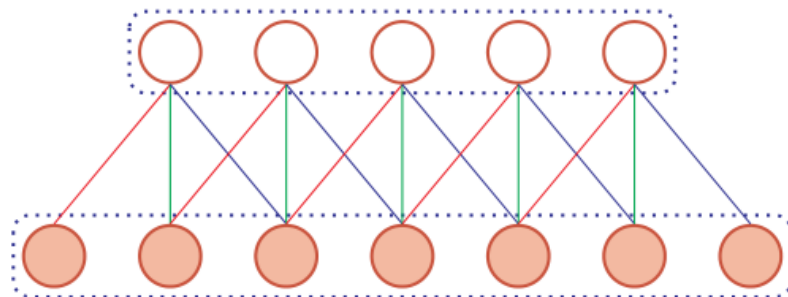
前馈网络



- 连接仅存在于层与层之间，同一层的节点之间是无连接的。（无循环）
- 输入和输出的维数都是固定的，不能任意改变，因此无法处理变长的序列数据。



(a) 全连接层

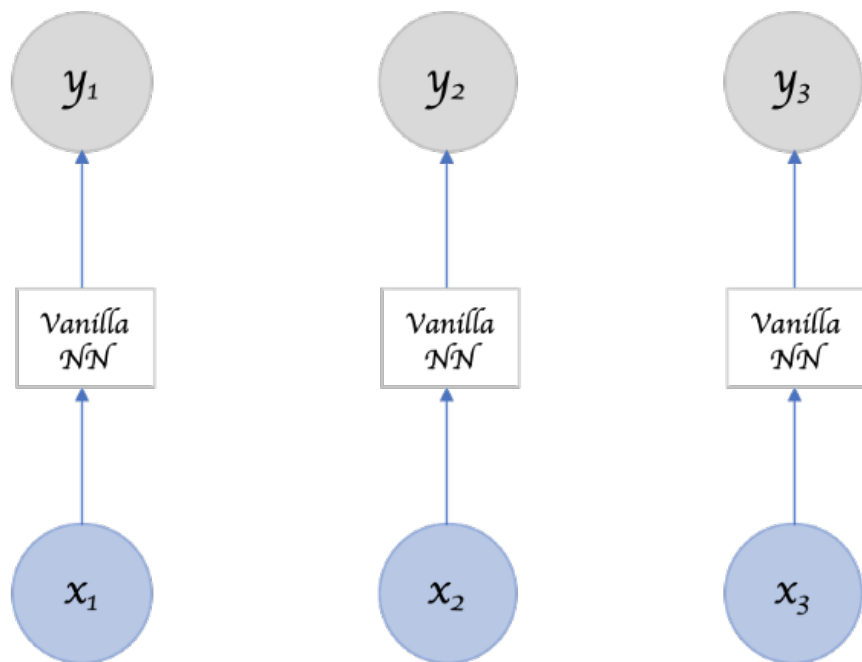


(b) 卷积层

前馈网络



- 假设每次的输入都是独立的，也就是说每次网络的输出只依赖于当前的输入，如下图所示。

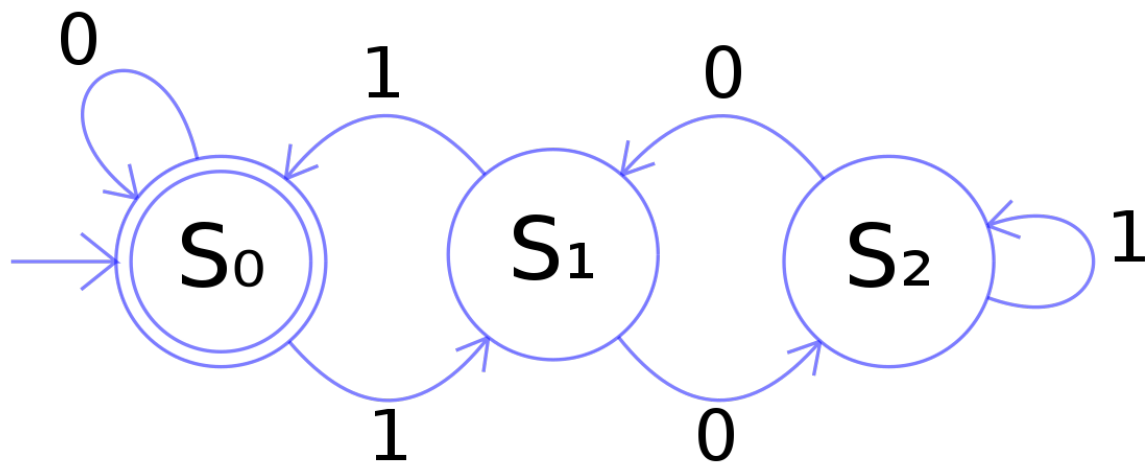


有限状态自动机

Finite Automata



- 有限状态自动机拥有有限数量的状态，每个状态可以迁移到零个或多个状态，输入字符串决定执行哪个状态的迁移。

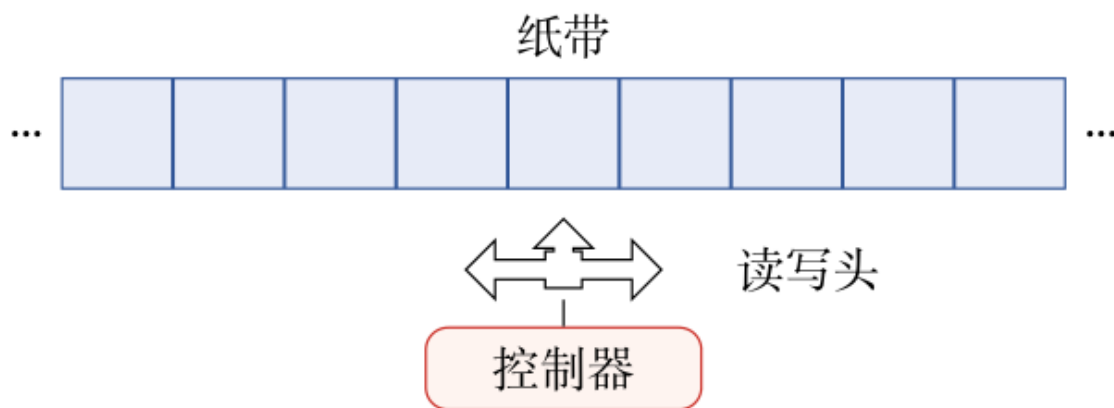


如何用FNN去模拟一个有限状态自动机？

图灵机



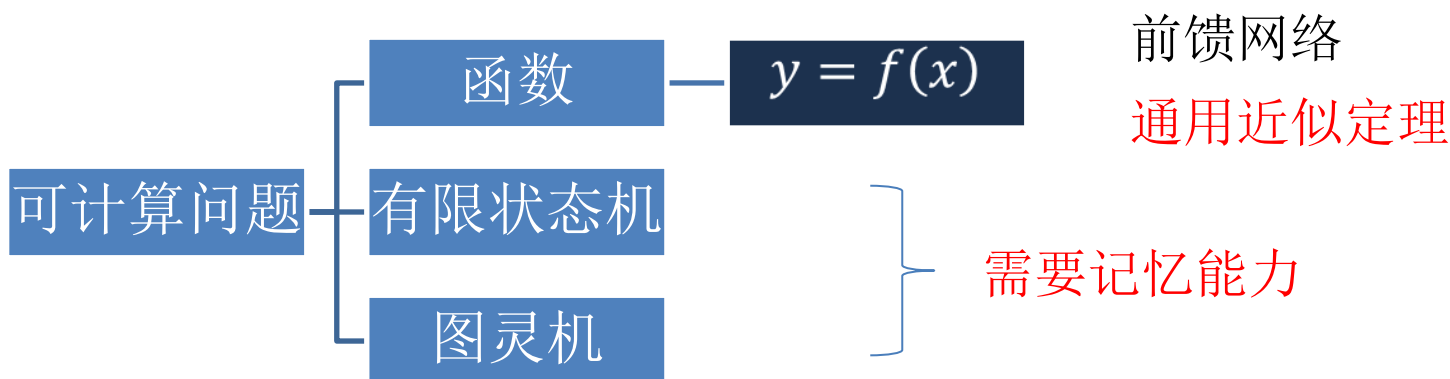
- 一种抽象数学模型，即将人们使用纸笔进行数学运算的过程进行抽象，由一个虚拟的机器替代人类进行数学运算，可以用来模拟任何可计算问题。



可计算问题



- 假设每次的输入都是独立的，也就是说每次网络的输出只依赖于当前的输入。



如何给网络增加记忆能力？

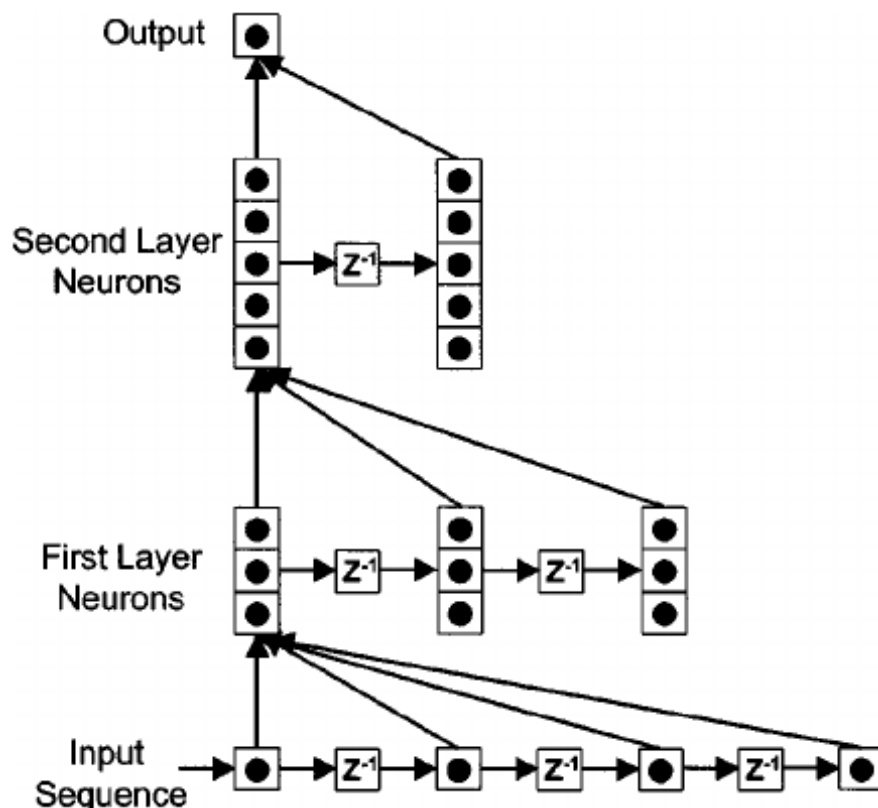
如何给网络增加记忆能力？



● 延时神经网络（Time Delay Neural Network, TDNN）

建立一个额外的延时单元，用来存储网络的历史信息（可以包括输入、输出、隐状态等）

$$\mathbf{h}_t^{(l)} = f\left(\mathbf{h}_t^{(l-1)}, \mathbf{h}_{t-1}^{(l-1)}, \dots, \mathbf{h}_{t-K}^{(l-1)}\right)$$



这样，前馈网络就具有了短期记忆的能力。

如何给网络增加记忆能力?



自回归模型 (Autoregressive Model, AR)

- 一类时间序列模型，用变量 y_t 的历史信息来预测自己

$$y_t = w_0 + \sum_{k=1}^K w_k y_{t-k} + \epsilon_t$$

- $\epsilon_t \sim N(0, \sigma^2)$ 为第 t 个时刻的噪声，方差与时间无关

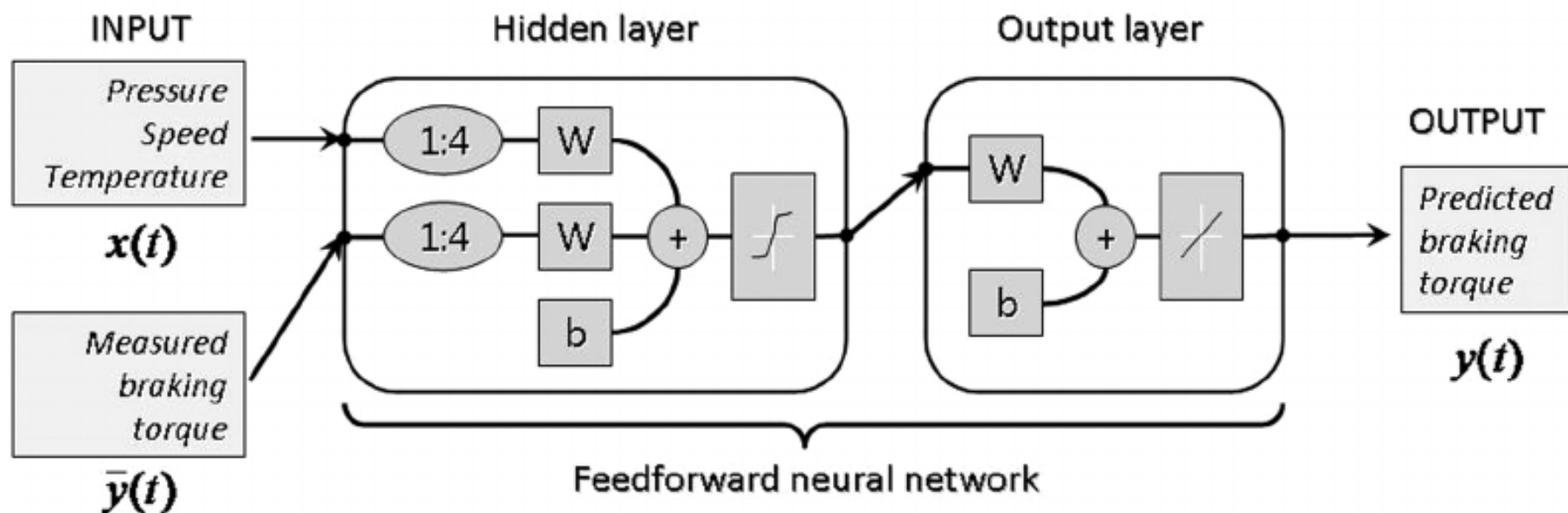
有外部输入的非线性自回归模型

(Nonlinear Autoregressive with Exogenous Inputs Model, NARX)

$$y_t = f(x_t, x_{t-1}, \dots, x_{t-K_x}, y_{t-1}, y_{t-2}, \dots, y_{t-K_y})$$

- 其中 $f(\cdot)$ 表示非线性函数，可以是一个前馈网络， K_x 和 K_y 为超参数

非线性自回归模型

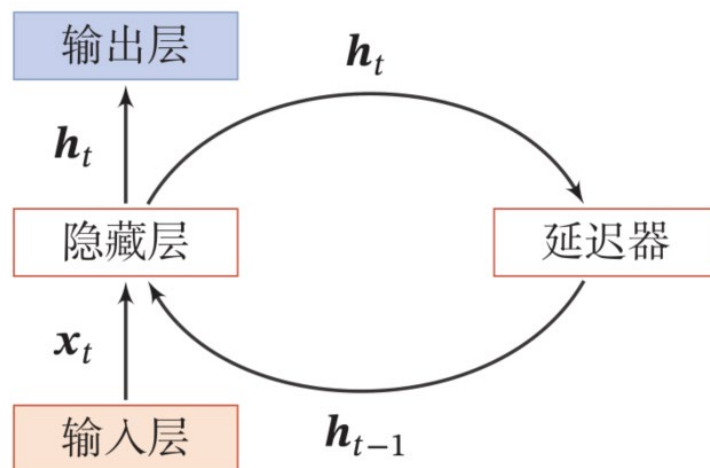




- 循环神经网络通过使用带自反馈的神经元，能够处理任意长度的时序数据。

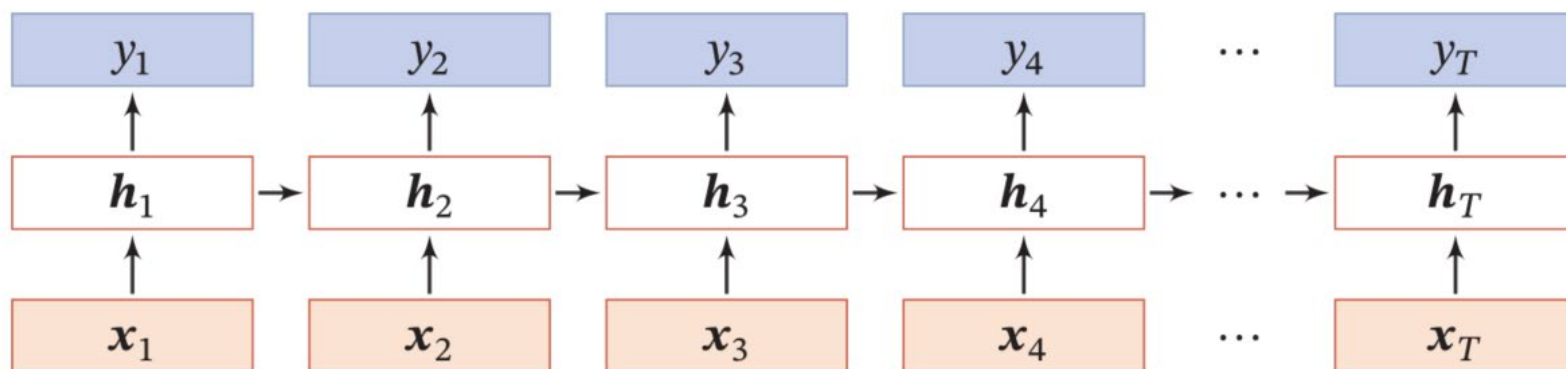
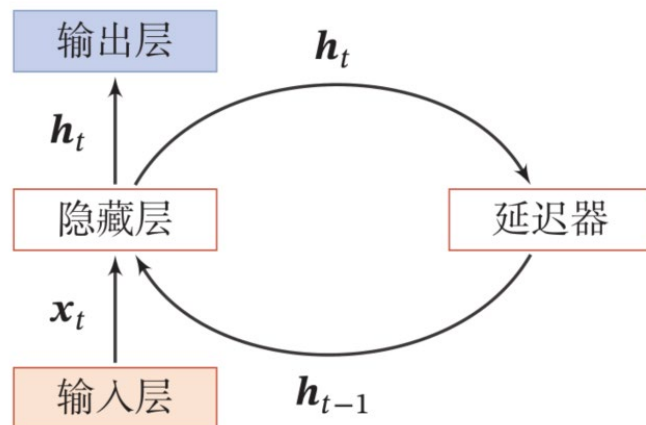
$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

活性值状态



- 循环神经网络比前馈神经网络更加符合生物神经网络的结构。
- 循环神经网络已经被广泛应用于语音识别、语言模型以及自然语言生成等任务上。

按时间展开





- 状态更新:

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b})$$

- 一个完全连接的循环网络是任何非线性动力系统的近似器。

定理 6.1 – 循环神经网络的通用近似定理 [Haykin, 2009]: 如果一个完全连接的循环神经网络有足够数量的 sigmoid 型隐藏神经元, 它可以以任意的准确率去近似任何一个非线性动力系统

$$\mathbf{s}_t = g(\mathbf{s}_{t-1}, \mathbf{x}_t), \quad (6.10)$$

$$\mathbf{y}_t = o(\mathbf{s}_t), \quad (6.11)$$

其中 $\mathbf{s}_t$ 为每个时刻的隐状态, $\mathbf{x}_t$ 是外部输入, $g(\cdot)$ 是可测的状态转换函数, $o(\cdot)$ 是连续输出函数, 并且对状态空间的紧致性没有限制.

图灵完备



- 图灵完备（Turing Completeness）是指一种数据操作规则，比如一种计算机编程语言，可以实现图灵机的所有功能，解决所有的可计算问题。

定理 6.2 – 图灵完备 [Siegelmann et al., 1991]: 所有的图灵机都可以被一个由使用 Sigmoid 型激活函数的神经元构成的全连接循环网络来进行模拟。

- 一个完全连接的循环神经网络可以近似解决所有的可计算问题。

循环神经网络



- 作用

- 输入-输出映射：机器学习模型（本节主要关注这种情况）
- 存储器：联想记忆模型

应用到机器学习



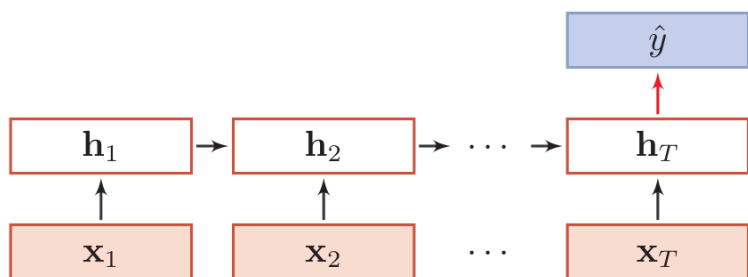
- 序列到类别模式
- 同步的序列到序列模式
- 异步的序列到序列模式

应用到机器学习

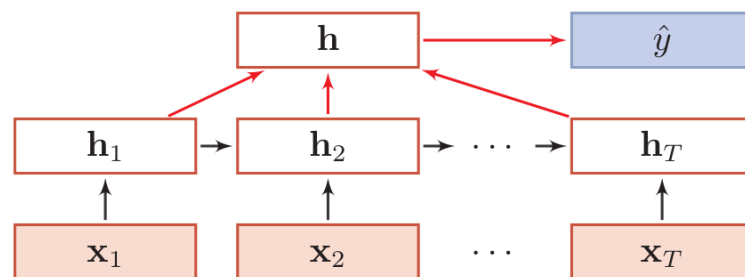


● 序列到类别

序列到类别模式主要用于序列数据的分类问题：输入为序列，输出为类别。如在文本分类中，输入数据为单词的序列，输出为该文本的类别。



(a) 正常模式



(b) 按时间进行平均采样模式

序列到类别



● 情感分类

带着愉悦的心情
看了这部电影

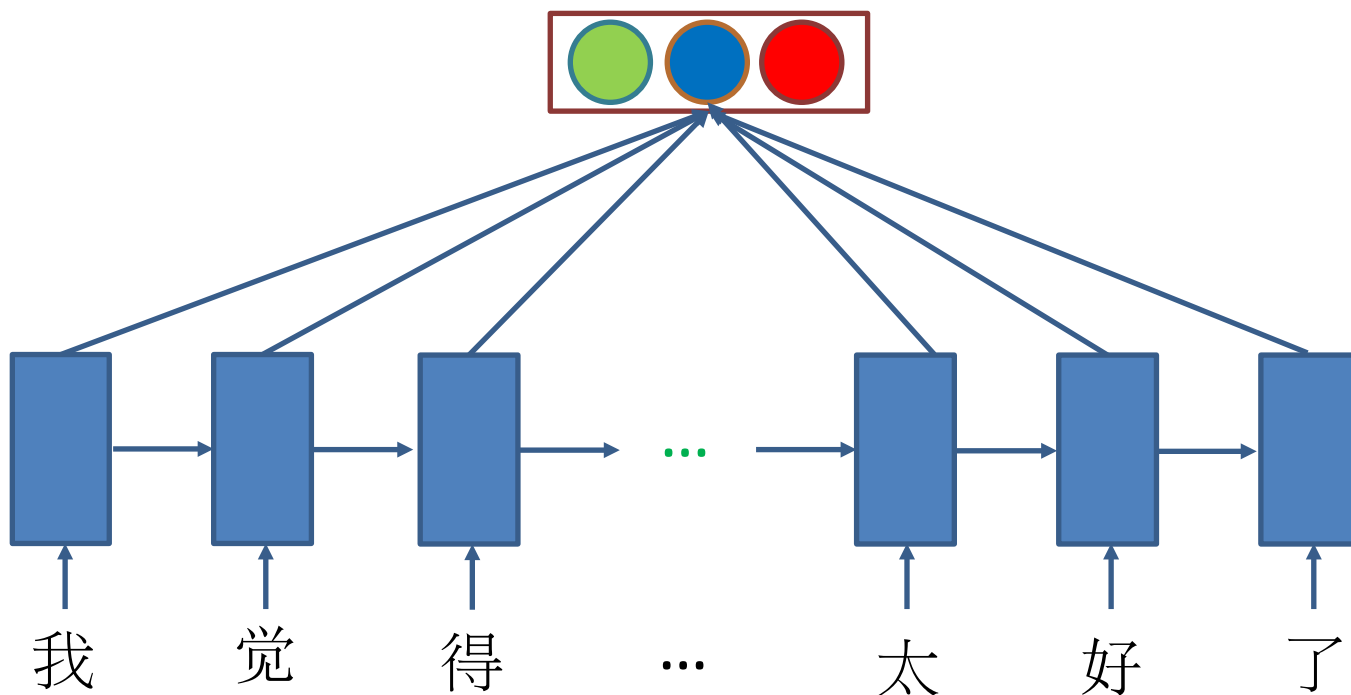
Positive (正面)

这部电影太糟了

Negative (负面)

这部电影很棒

Positive (正面)

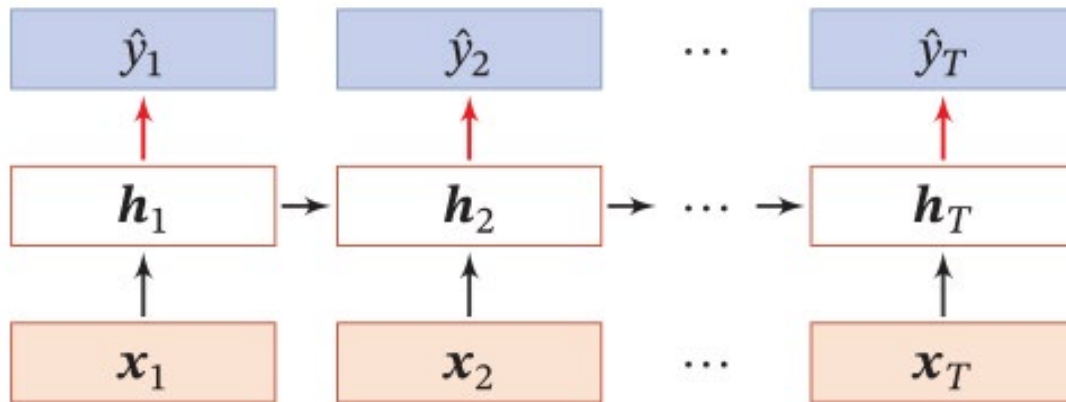


应用到机器学习



- 同步的序列到序列模式

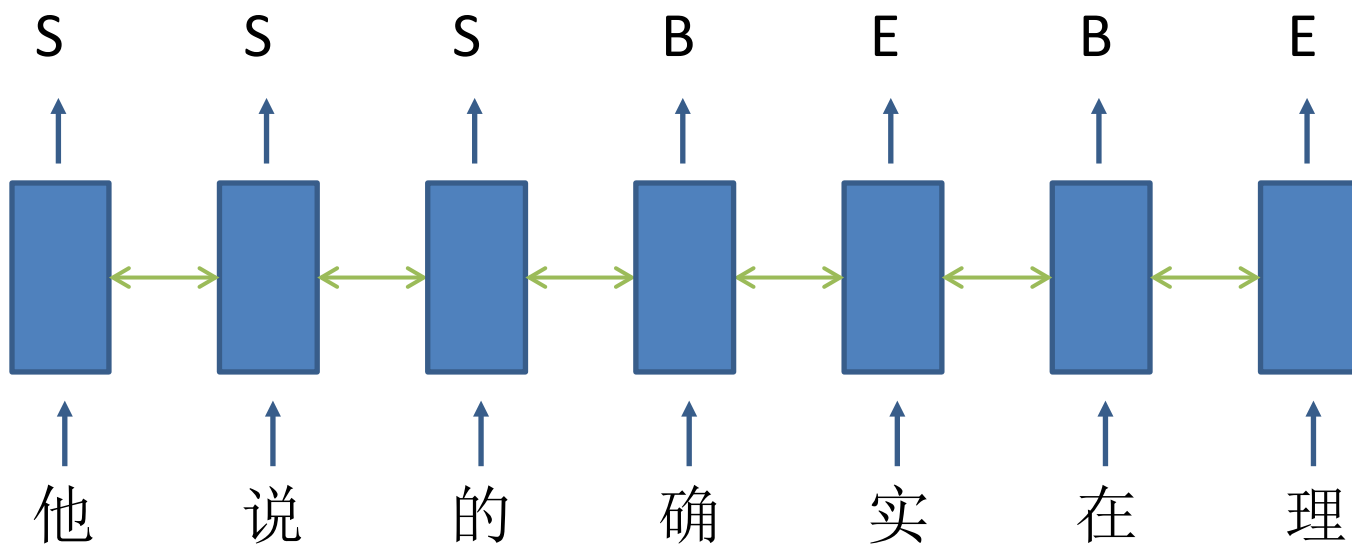
同步的序列到序列模式主要用于序列标注任务，即每一时刻都有输入和输出，输入序列和输出序列的长度相同。



同步的序列到序列模式



- 中文分词



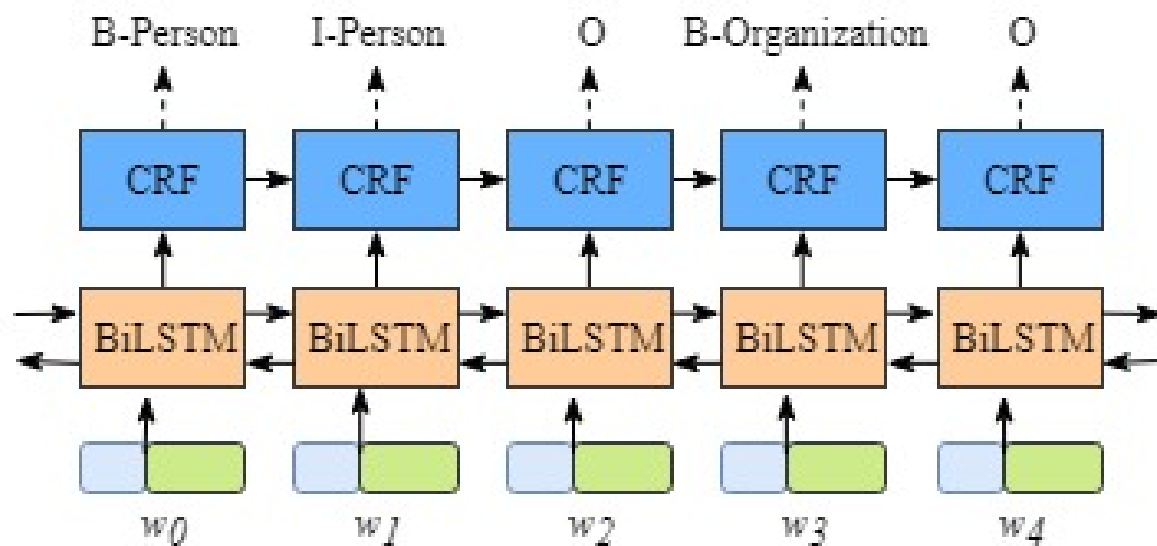
同步的序列到序列模式



- 信息抽取(Information Extraction, IE)

从无结构的文本中抽取结构化的信息，形成知识

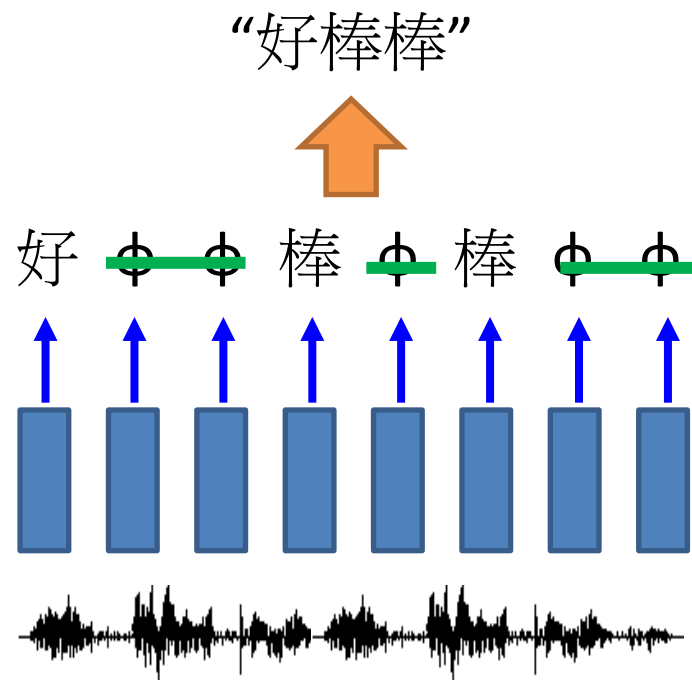
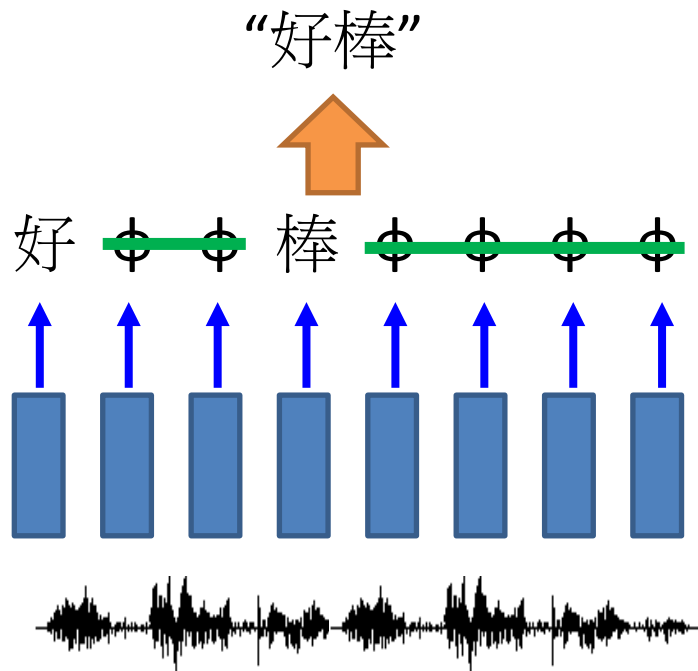
小米创始人雷军表示，该公司2015年营收达到780
亿元人民币，较2014年的743亿元人民币增长了5%。



同步的序列到序列模式



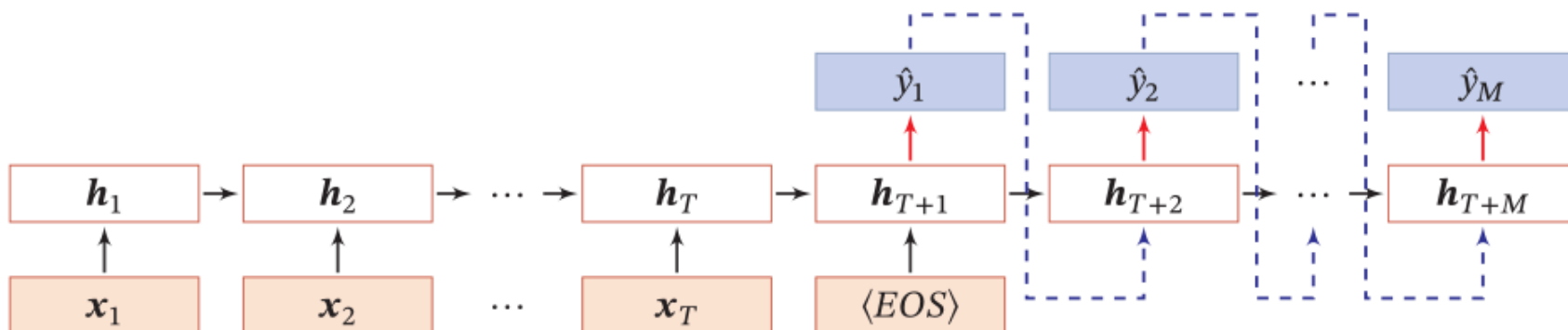
- Connectionist Temporal Classification (CTC) [Alex Graves, ICML'06][Alex Graves, ICML'14][Haşim Sak, Interspeech'15][Jie Li, Interspeech'15][Andrew Senior, ASRU'15]





● 异步的序列到序列模式

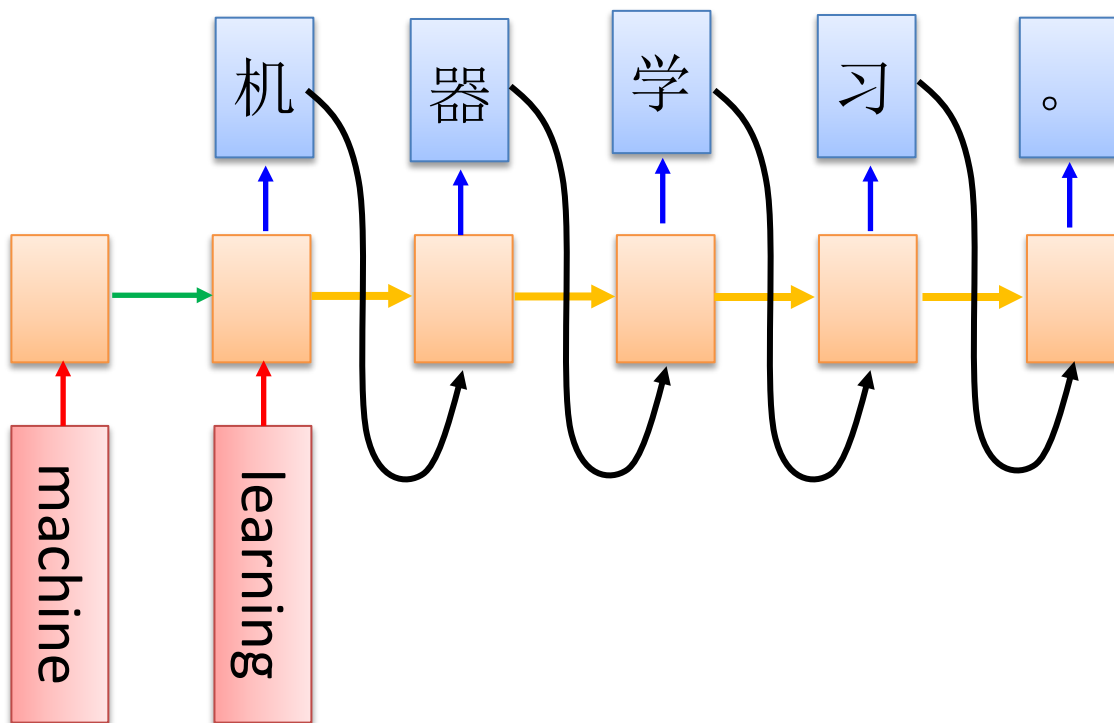
异步的序列到序列模式也称为编码器-译码器模型，即输入序列和输出序列不需要有严格的对应关系，也不需要保持相同的长度。如在机器翻译中，输入为源语言的单词，输出为目标语言的单词序列。



异步的序列到序列模式



- 机器翻译





- 机器学习

给定一个训练样本 $(\mathbf{x}, \mathbf{y})$ ，其中 $\mathbf{x}_{1:T} = (x_1, \dots, x_T)$ 为长度是 T 的输入序列， $\mathbf{y}_{1:T} = (y_1, \dots, y_T)$ 是长度为 T 的标签序列。

- t 时刻的瞬时损失函数为

$$\mathcal{L}_t = \mathcal{L}(\mathbf{y}_t, g(\mathbf{h}_t)),$$

- 总损失函数

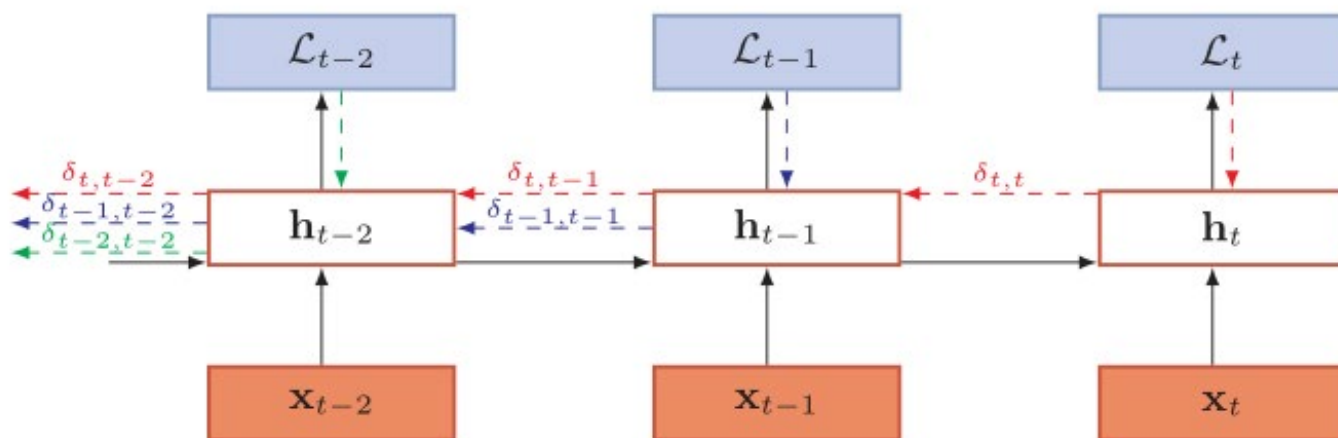
$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t.$$

梯度



- 随时间反向传播算法

$$\mathbf{h}_{t+1} = f(\mathbf{z}_{t+1}) = f(U\mathbf{h}_t + W\mathbf{x}_{t+1} + \mathbf{b})$$



$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \left(\text{diag}(f'(\mathbf{z}_\tau)) U^T \right) \delta_{t,t}$$

$\delta_{t,k}$ 为第 t 时刻的损失对第 k 步隐藏神经元的净输入 $\mathbf{z}_k$ 的导数

梯度消失/爆炸



- 梯度

$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

- 其中

$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \left(\text{diag}(f'(\mathbf{z}_{\tau})) U^T \right) \delta_{t,t}$$

由于梯度爆炸或消失问题，实际上只能学习到短周期的依赖关系。这就是所谓的长程依赖问题。

长程依赖问题



循环神经网络在时间维度上非常深！

如何改进？

- 梯度爆炸问题：权重衰减 梯度截断
- 梯度消失问题：改进模型

改进方法

- 循环边改为线性依赖关系 $\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t; \theta),$
- 增加非线性

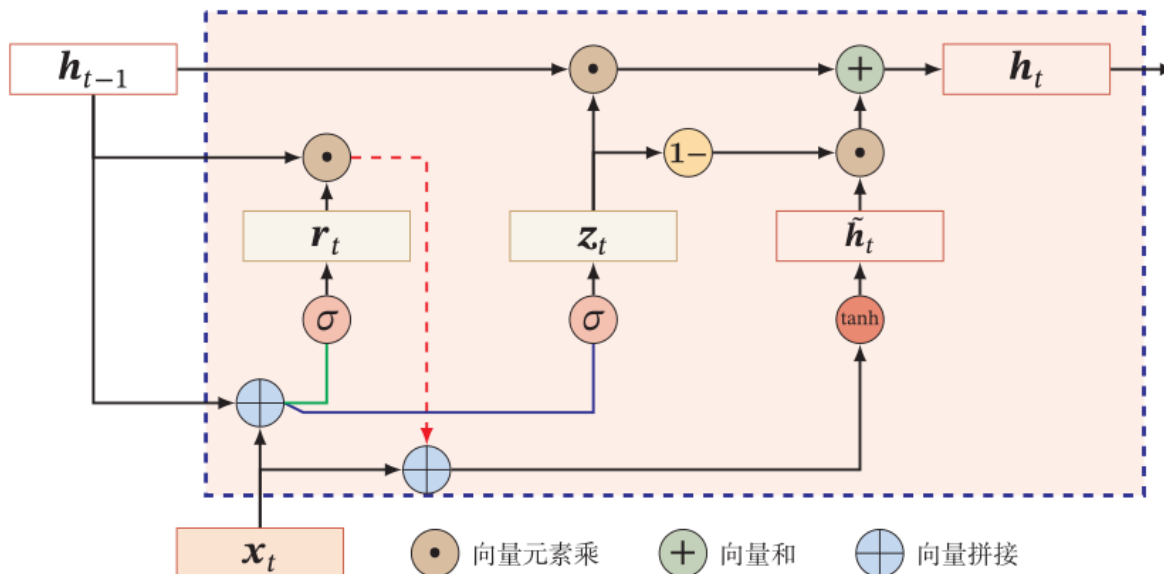
$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta),$$

残差网络？

Gated Recurrent Unit, GRU



$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \mathbf{g}(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta)$$



重置门

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r), \quad \tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}(\mathbf{r}_t \odot \mathbf{h}_{t-1}))$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z), \quad \mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t,$$

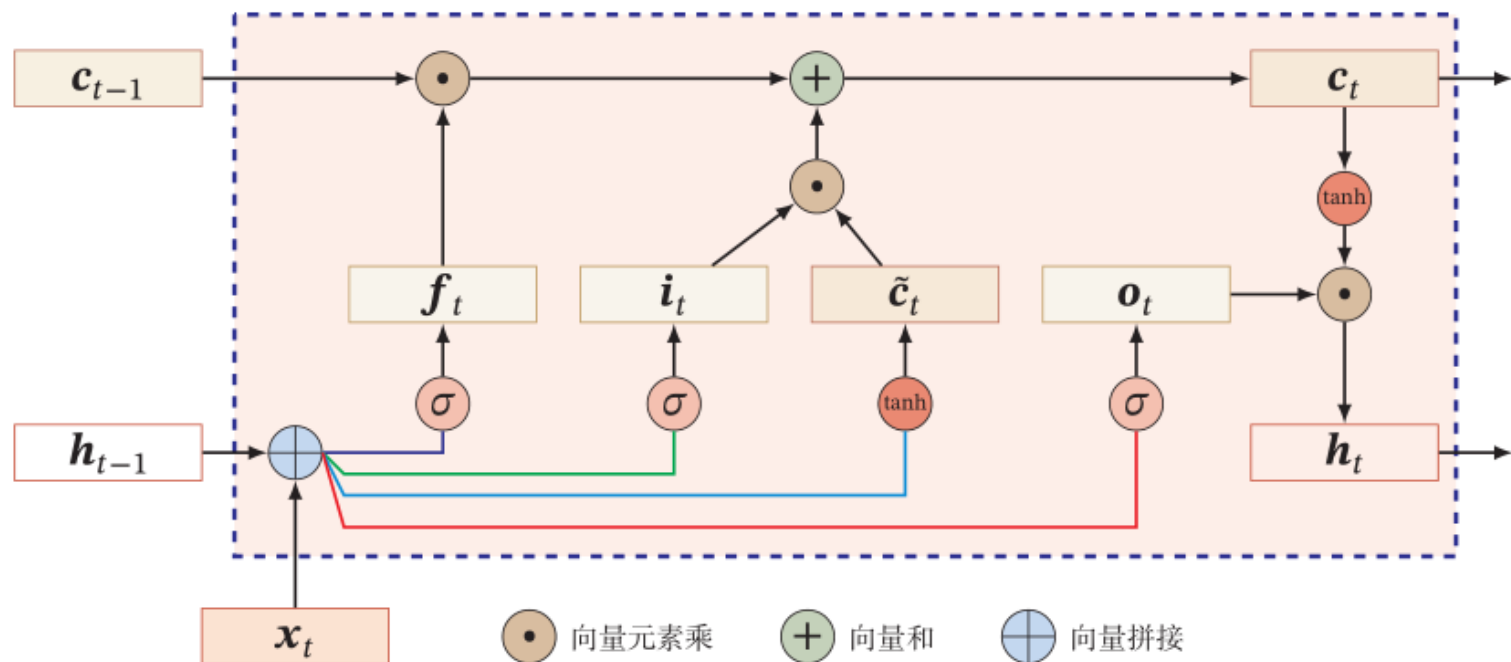
更新门

t时刻的候选状态

t时刻的状态

长短期记忆神经网络

Long Short-Term Memory, LSTM



$$\begin{aligned}i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), \\f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o),\end{aligned}$$

$$\begin{aligned}\tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) \\c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\h_t &= o_t \odot \tanh(c_t),\end{aligned}$$

LSTM网络引入门控机制来控制信息传递的路径。

LSTM的各种变体



- 没有遗忘门

$$\mathbf{c}_t = \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

- 耦合输入门和遗忘门

$$\mathbf{f}_t + \mathbf{i}_t = \mathbf{1}.$$

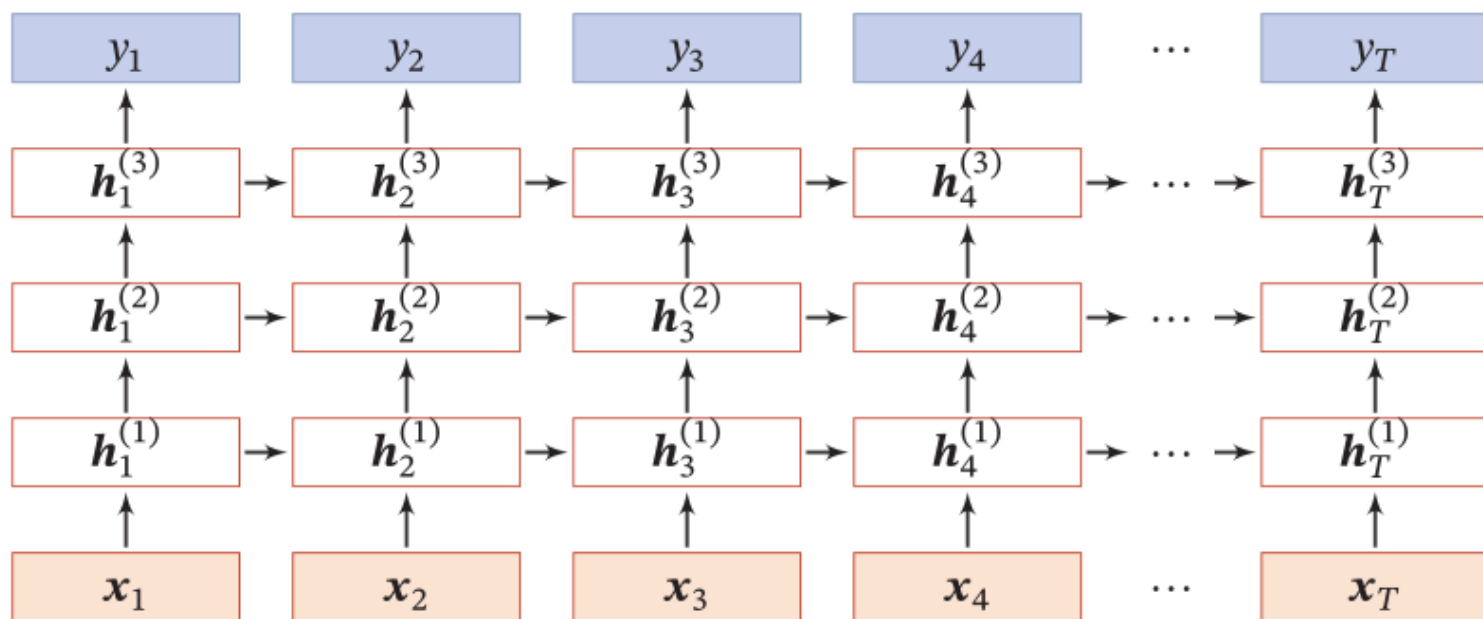
- peephole连接

$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + V_i \mathbf{c}_{t-1} + \mathbf{b}_i),$$

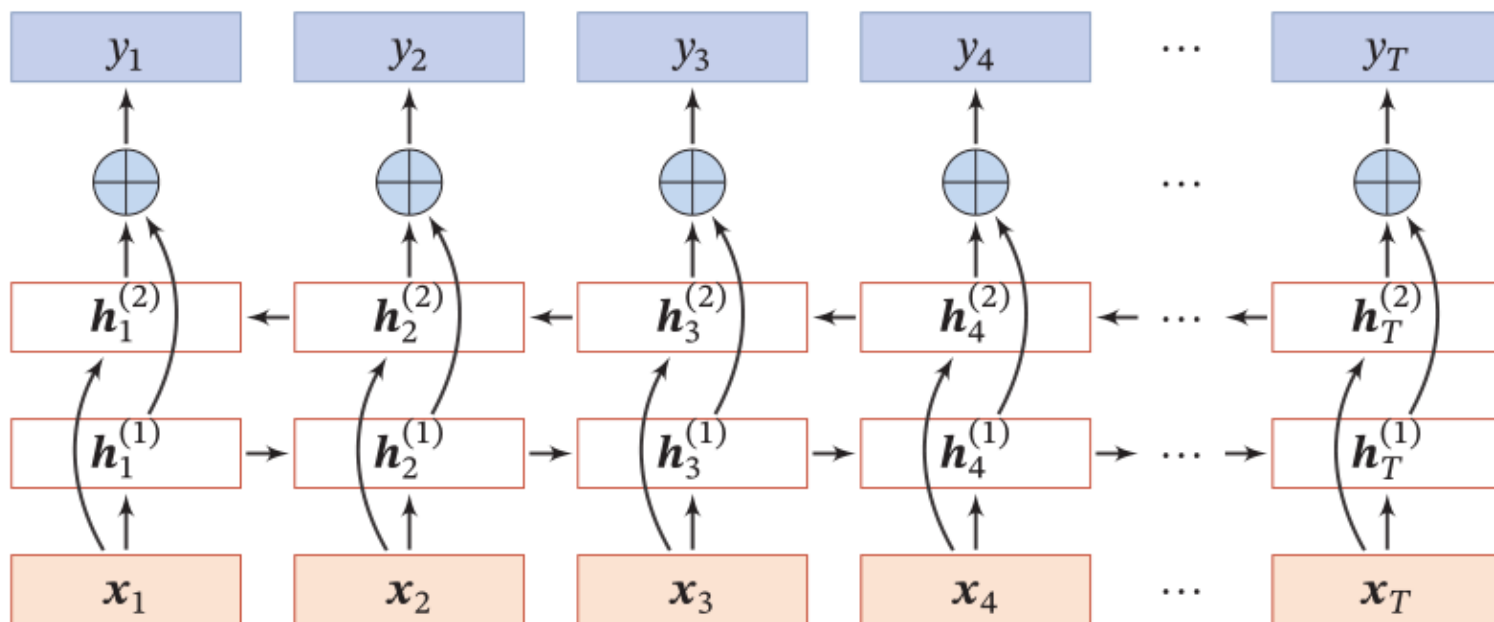
$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + V_f \mathbf{c}_{t-1} + \mathbf{b}_f),$$

$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + V_o \mathbf{c}_t + \mathbf{b}_o),$$

堆叠循环神经网络



双向循环神经网络





优点:

- 引入记忆
- 图灵完备

缺点:

- 长程依赖问题
- 记忆容量问题
- 并行能力

语言模型



- 自然语言理解 → 一个句子的可能性/合理性

！ 在报那猫告做只



那只猫在作报告！



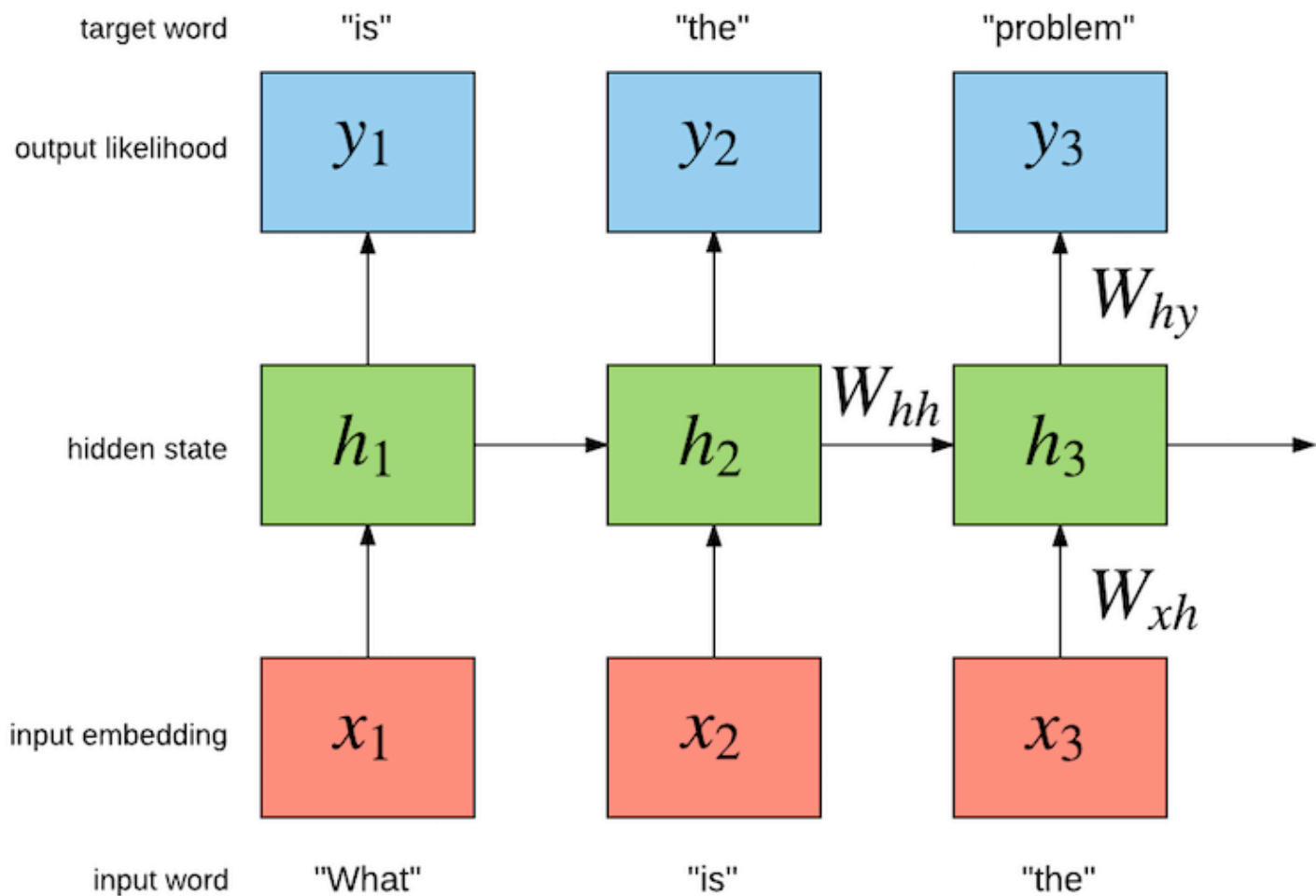
那个人在作报告！



一切都是概率！

$$p(x_{1:T}) = \prod_t p(x_t | x_{1:t-1}) \approx \prod_t p(x_t | x_{t-1}, \dots, x_{t-n+1}) = \prod_t g(h_t)$$

语言模型



生成Linux内核代码



```
/*
 * If this error is set, we will need anything right after that BSD.
 */
static void action_new_function(struct s_stat_info *wb)
{
    unsigned long flags;
    int lel_idx_bit = e->edd, *sys & ~((unsigned long) *FIRST_COMPAT);
    buf[0] = 0xFFFFFFFF & (bit << 4);
    min(inc, slist->bytes);
    printk(KERN_WARNING "Memory allocated %02x/%02x, "
        "original MLL instead\n"),
        min(min(multi_run - s->len, max) * num_data_in),
        frame_pos, sz + first_seg);
    div_u64_w(val, inb_p);
    spin_unlock(&disk->queue_lock);
    mutex_unlock(&s->sock->mutex);
    mutex_unlock(&func->mutex);
    return disassemble(info->pending_bh);
}

static void num_serial_settings(struct tty_struct *tty)
{
    if (tty == tty)
        disable_single_st_p(dev);
    pci_disable_spool(port);
}
```



- RNN在“学习”过汪峰全部作品后自动生成的歌词

<https://github.com/phunterlau/wangfeng-rnn>

我在这里中的夜里
就像一场是一种生命的意叶
就像我的生活变得在我一样
可我们这是一个知道
我只是一天你会怎吗
可我们这是我们是不要为你
我们想这有一种生活的时候

作诗



白鹭窥鱼立，

Egrets stood, peeping fishes.

青山照水开。

Water was still, reflecting mountains.

夜来风不动，

The wind went down by nightfall,

明月见楼台。

as the moon came up by the tower.

满怀风月一枝春，

Budding branches are full of romance.

未见梅花亦可人。

Plum blossoms are invisible but adorable.

不为东风无此客，

With the east wind comes Spring.

世间何处是前身。

Where on earth do I come from?

传统统计机器翻译

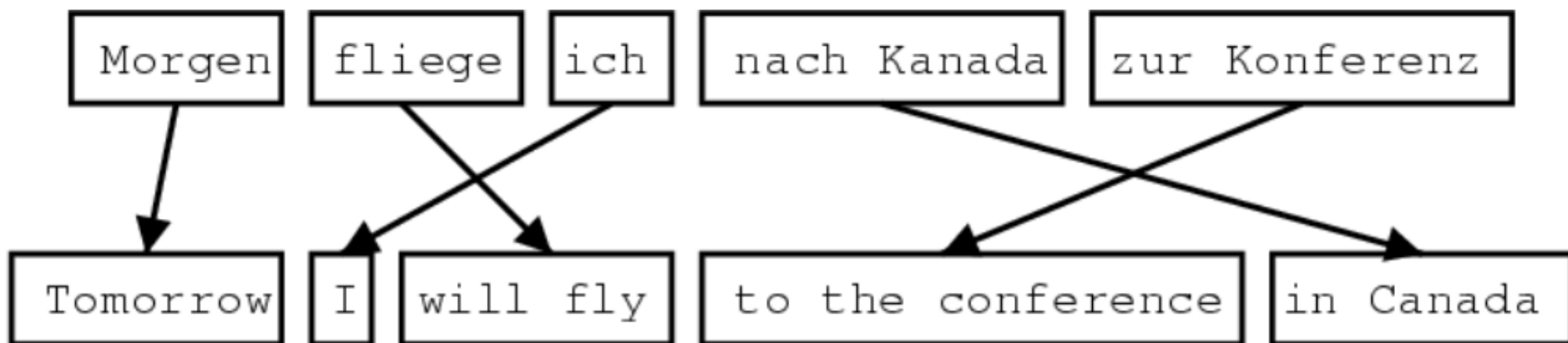


- 源语言：f
- 目标语言：e

模型： $\hat{e} = \arg \max_e p(e | f) = \arg \max_e p(f | e)p(e)$

翻译模型： $p(f | e)$

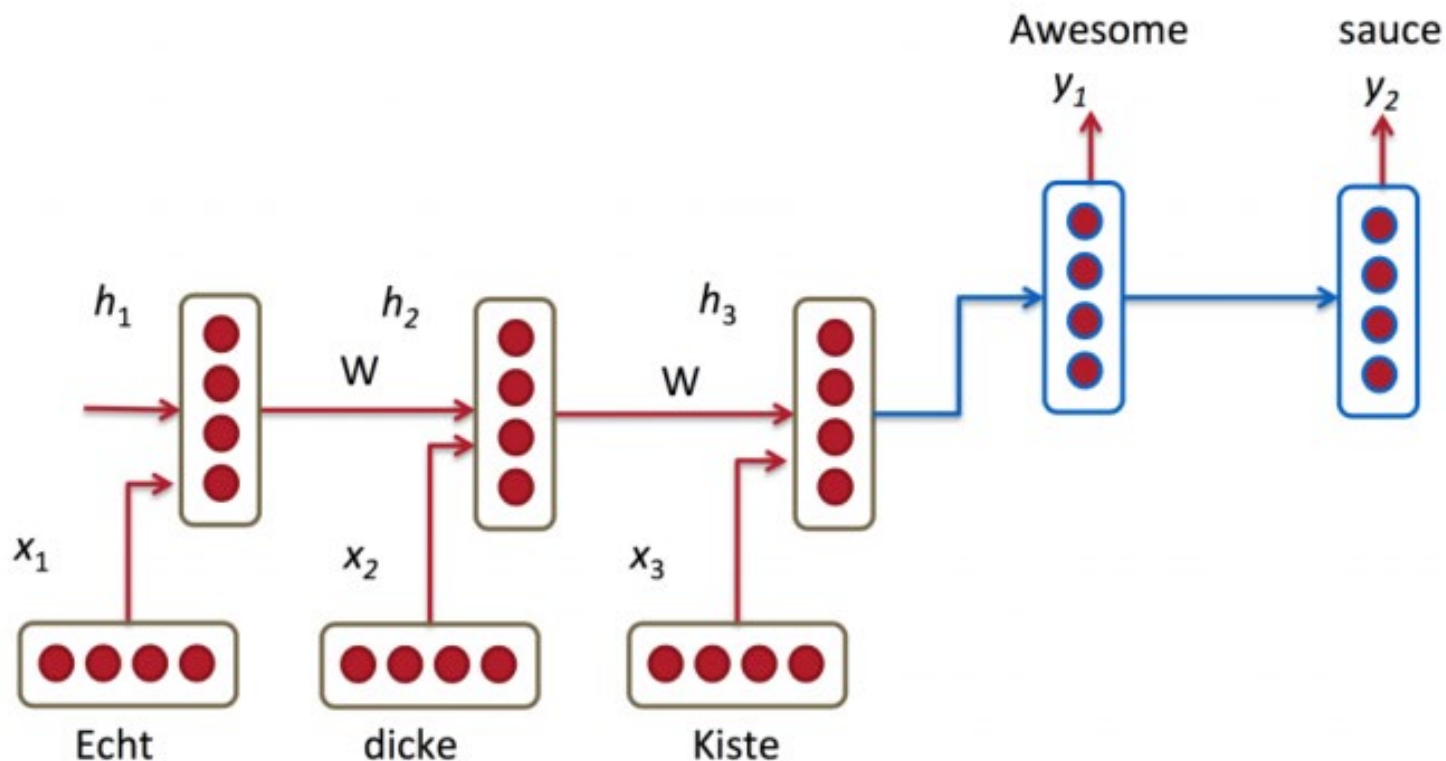
语言模型： $p(e)$



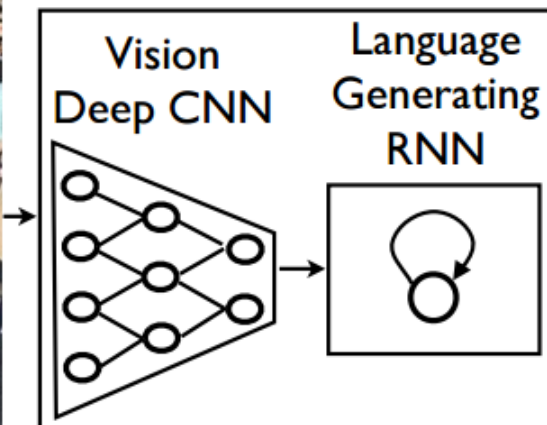
基于序列到序列的机器翻译



- 一个RNN用来编码
- 另一个RNN用来解码



看图说话



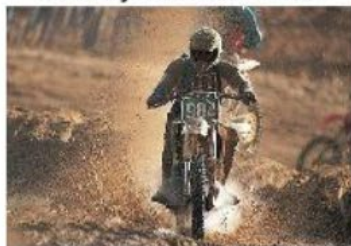
A group of people shopping at an outdoor market.

There are many vegetables at the fruit stand.

看图说话



A person riding a motorcycle on a dirt road.



Two dogs play in the grass.



A skateboarder does a trick on a ramp.



A dog is jumping to catch a frisbee.



A group of young people playing a game of frisbee.



Two hockey players are fighting over the puck.



A little girl in a pink hat is blowing bubbles.



A refrigerator filled with lots of food and drinks.



A herd of elephants walking across a dry grass field.



A close up of a cat laying on a couch.



A red motorcycle parked on the side of the road.



A yellow school bus parked in a parking lot.



Describes without errors

Describes with minor errors

Somewhat related to the image

Unrelated to the image

Figure 5. A selection of evaluation results, grouped by human rating.



Demo Website

VQA: Given an image and a natural language question about the image, the task is to provide an accurate natural language answer



What is the mustache made of?

AI System

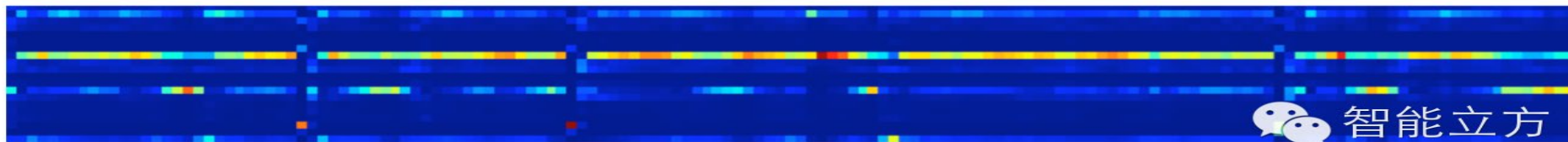
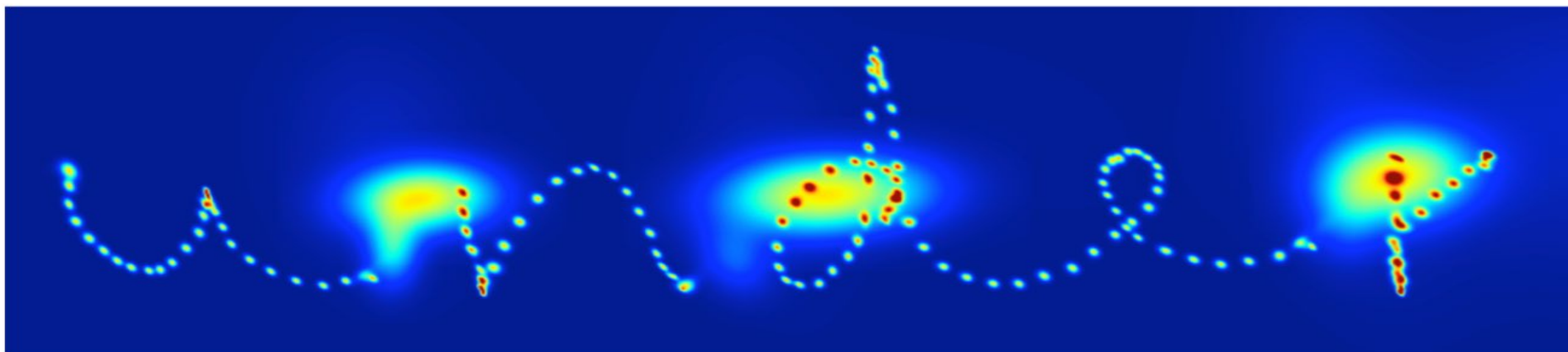
bananas

Picture from (Antol et al., 2015)

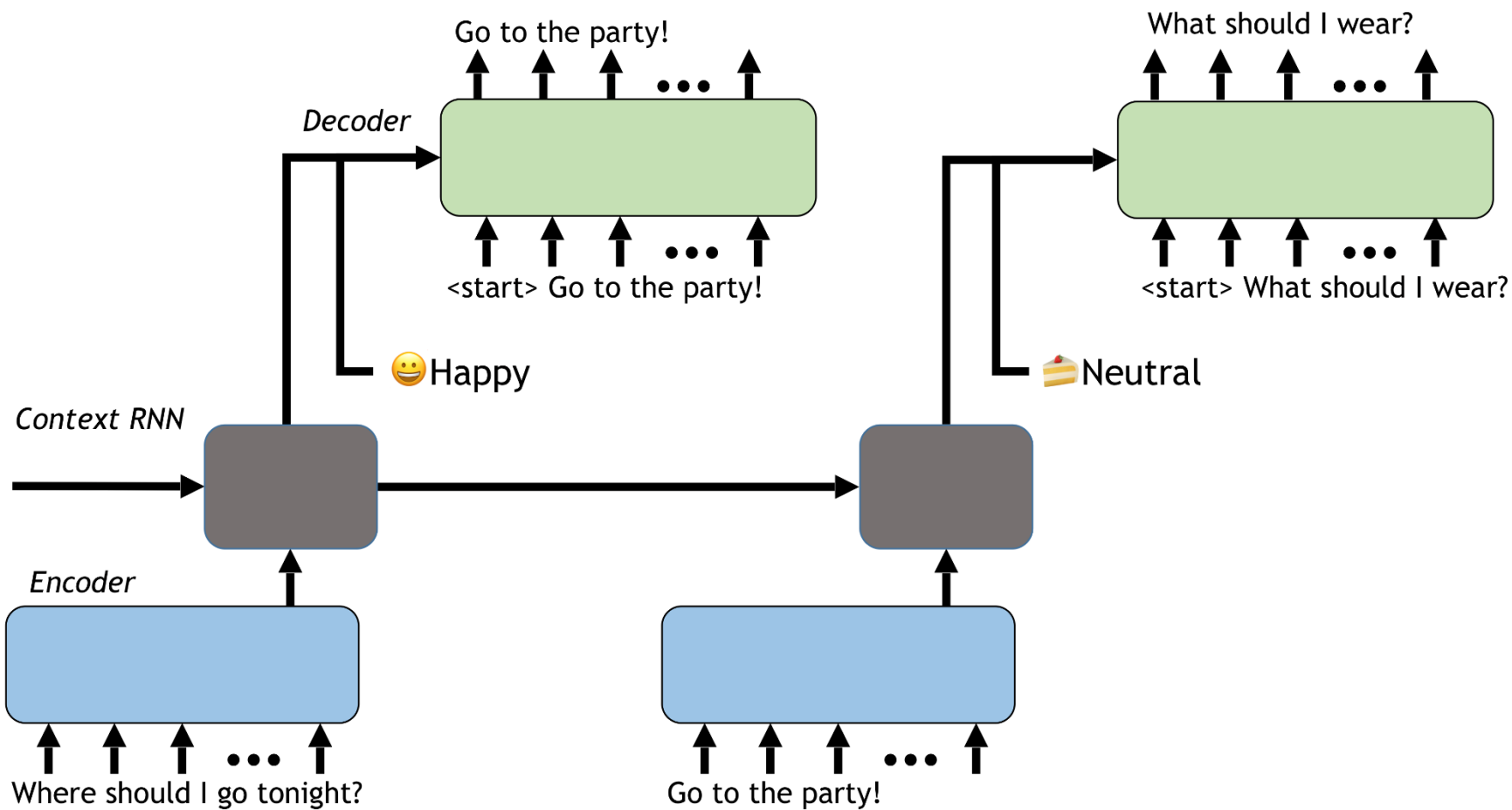
写字



- 把一个字母的书写轨迹看作是一连串的点。一个字母的“写法”其实是每一个点相对于前一个点的偏移量，记为(offset x, offset y)。再增加一维取值为0或1来记录是否应该“提笔”。

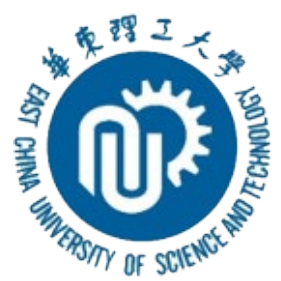


对话系统

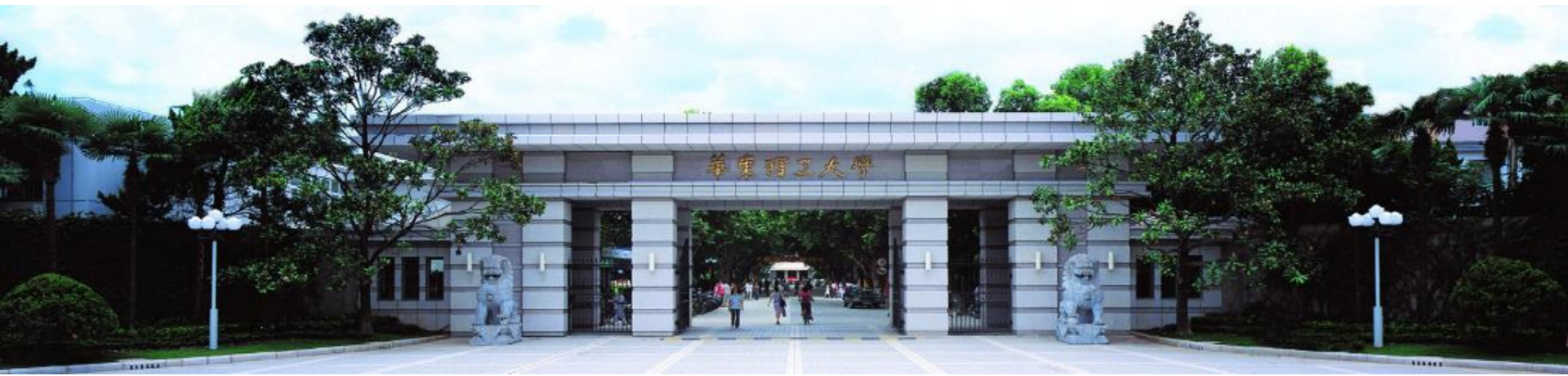




- 把一个字母的书写轨迹看作是一连串的点。一个字母的“写法”其实是每一个点相对于前一个点的偏移量，记为(offset x, offset y)。再增加一维取值为0或1来记录是否应该“提笔”。



网络优化与正则化



深度学习的矛与盾？



优化

经验风险最小

正则化

降低模型复杂度



网络优化的难点



结构差异大

- 没有通用的优化算法
- 超参数多

结构差异大

- 参数初始化
- 逃离局部最优

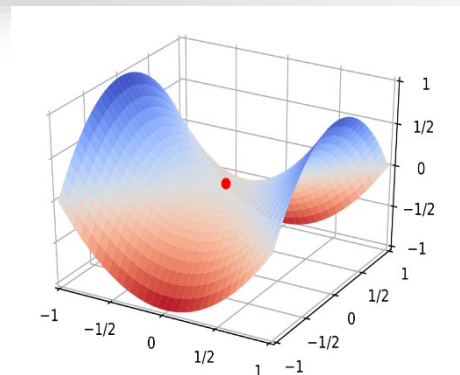
梯度消失（爆炸）问题

高维空间的非凸优化问题



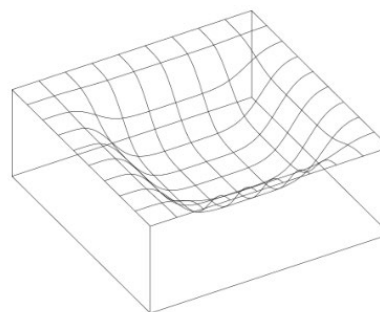
鞍点 (Saddle Point)

驻点 (Stationary Point) :
梯度为 0 的点。

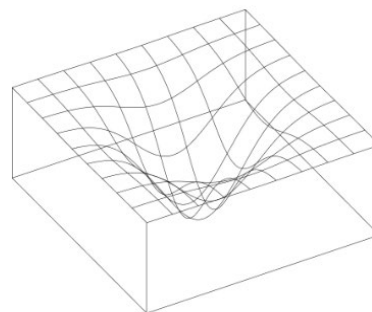


平坦最小值 (Flat Minima)

- 一个平坦最小值的邻域内，所有点对应的训练损失都比较接近
- 大部分的局部最小解是等价的
- 局部最小解对应的训练损失都可能非常接近于全局最小解对应的训练损失



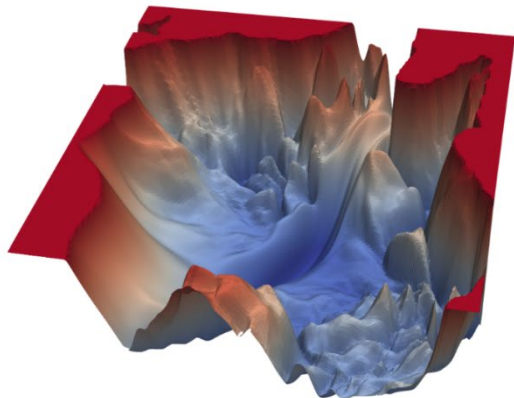
(a) 平坦最小值



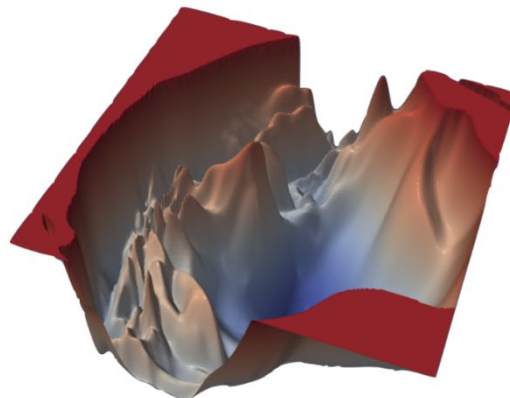
(b) 尖锐最小值

VISUALIZING THE LOSS LANDSCAPE OF NEURAL NETS

VGG-56

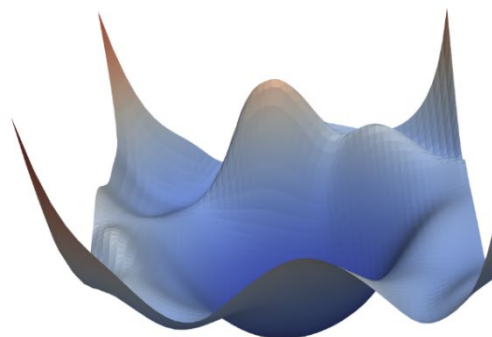


VGG-110

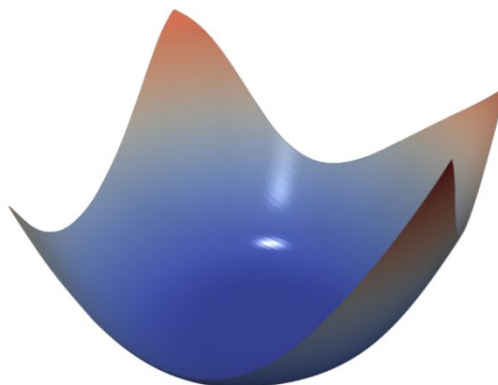


without skip connections

Renset-56



Densenet-121



with skip connections

神经网络优化的改善方法



1. 使用更有效的优化算法来提高梯度下降优化方法的效率和稳定性
动态学习率调整、梯度估计修正
2. 使用更好的参数初始化方法、数据预处理方法来提高优化效率
3. 修改网络结构来得到更好的优化地形（ Optimization Landscape ）
优化地形指在高维空间中损失函数的曲面形状。好的优化地形通常比较平滑。使用 ReLU 激活函数、残差连接、逐层归一化等。
4. 使用更好的超参数优化方法。

优化算法：随机梯度下降



算法 2.1 随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α

1 随机初始化 θ ;

2 **repeat**

3 对训练集 $\mathcal{D}$ 中的样本随机排序;

4 **for** $n = 1 \cdots N$ **do**

5 从训练集 $\mathcal{D}$ 中选取样本 $(\mathbf{x}^{(n)}, y^{(n)})$;

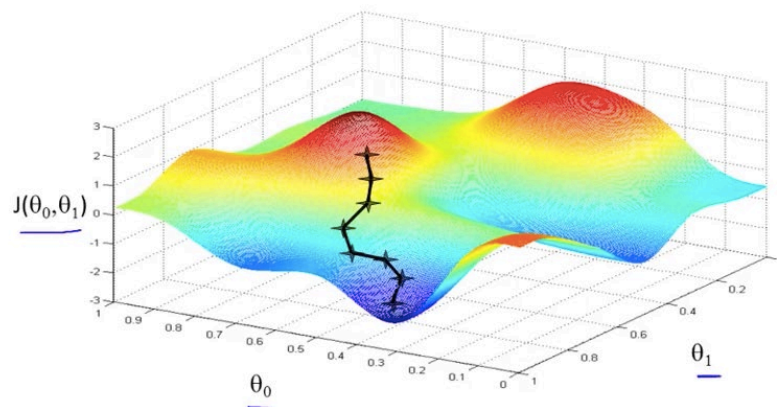
6 $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; \mathbf{x}^{(n)}, y^{(n)})}{\partial \theta}$;

// 更新参数

7 **end**

8 **until** 模型 $f(\mathbf{x}; \theta)$ 在验证集 $\mathcal{V}$ 上的错误率不再下降;

输出: θ



优化算法：

小批量随机梯度下降 MiniBatch



- 选取K个训练样本 $\{\mathbf{x}^{(k)}, \mathbf{y}^{(k)}\}_{k=1}^K$ ，计算偏导数

$$\mathbf{g}_t(\theta) = \frac{1}{K} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{S}_t} \frac{\partial \mathcal{L}(\mathbf{y}, f(\mathbf{x}; \theta))}{\partial \theta}$$

- 定义梯度

$$\mathbf{g}_t \triangleq \mathbf{g}_t(\theta_{t-1})$$

- 更新参数 其中 $\alpha > 0$ 为学习率

$$\theta_t \leftarrow \theta_{t-1} - \alpha \mathbf{g}_t$$

几个关键因素：

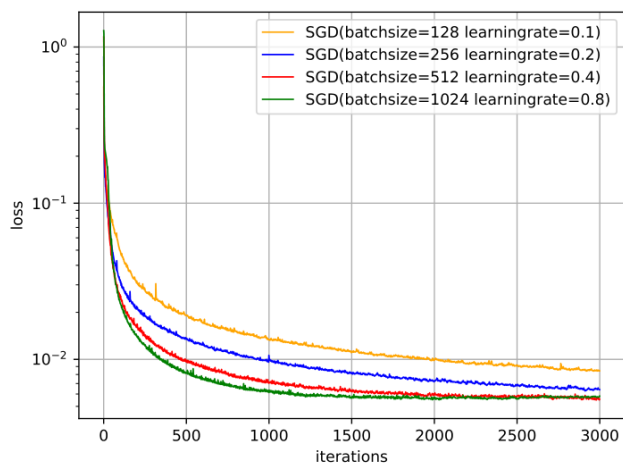
- 小批量样本数量
- 梯度
- 学习率

批量大小的影响

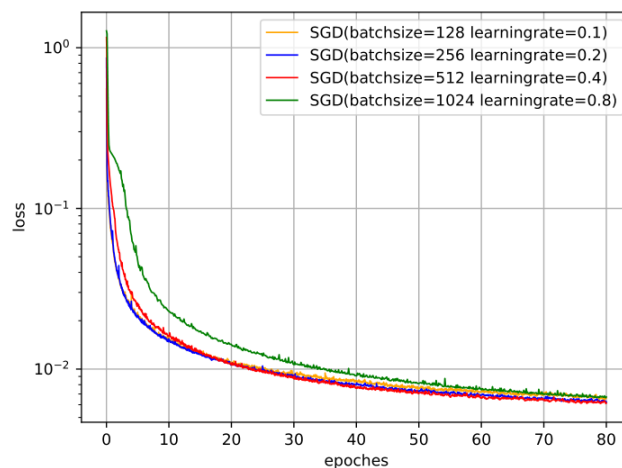


批量大小不影响随机梯度的期望，但会影响随机梯度的方差。

- 批量越大，随机梯度的方差越小，引入的噪声也越小，训练也越稳定，因此可以设置较大的学习率。
- 而批量较小时，需要设置较小的学习率，否则模型会不收敛。



(a) 按 Iteration 的损失变化



(b) 按 Epoch 的损失变化

4 种批量大小
对应的学习
率设置不同，
因此并不是
严格对比。

小批量梯度下降中，每次选取样本数量对损失下降的影响。

如何改进？



AdaGrad 算法 (Adaptive Gradient Algorithm) [Duchi et al., 2011] 是借鉴 ℓ_2 正则化的思想, 每次迭代时自适应地调整每个参数的学习率. 在第 t 次迭代时, 先计算每个参数梯度平方的累计值

$$G_t = \sum_{\tau=1}^t \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau}, \quad (7.14)$$

- 标准的（小批量）梯度下降

- 学习率

- 学习率衰减
- Adagrad
- Adadelata
- RMSprop

- 梯度

- Momentum（计算负梯度的“加权移动平均”作为参数的更新方）
- Nesterov accelerated gradient
- 梯度截断

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

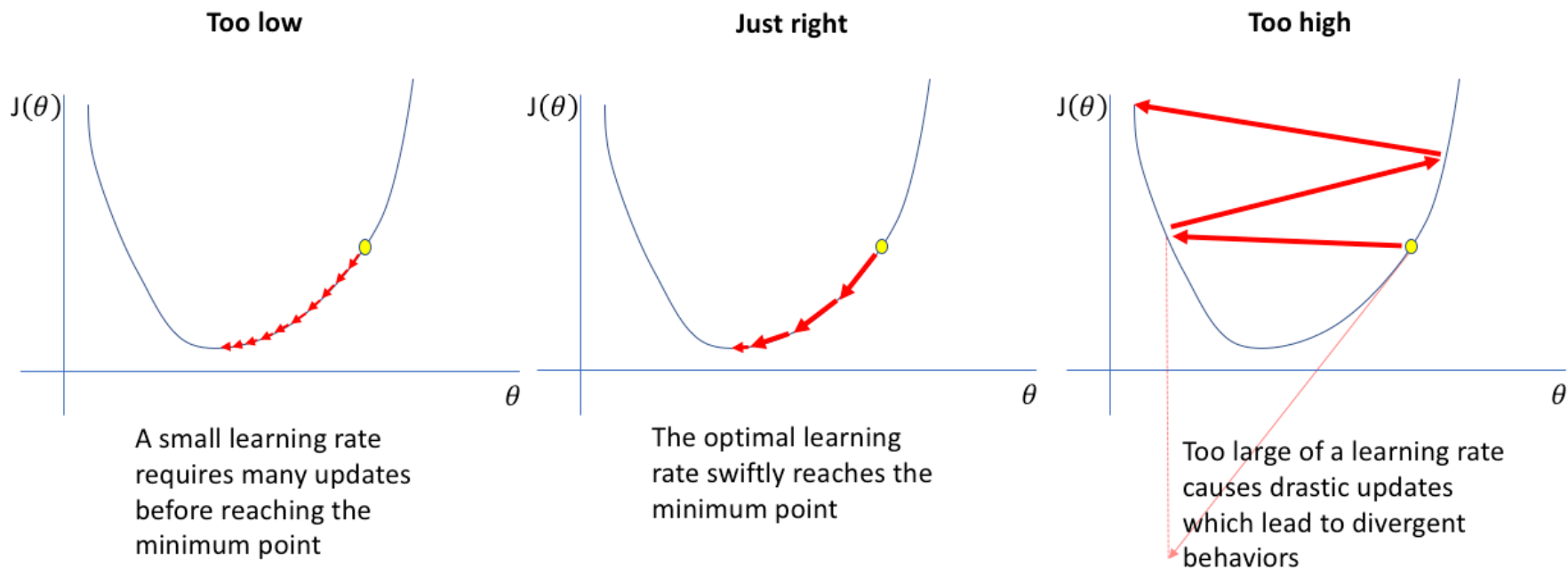
实际更新方向 梯度方向

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t$$

Adam

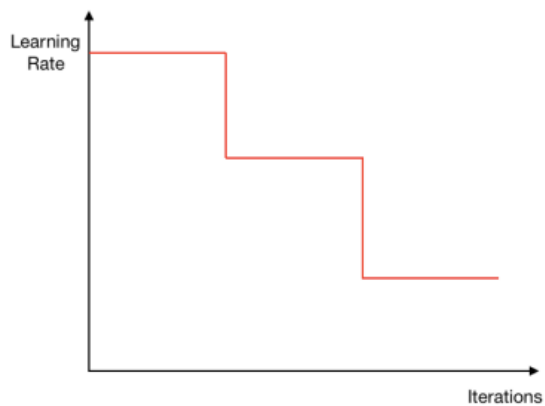
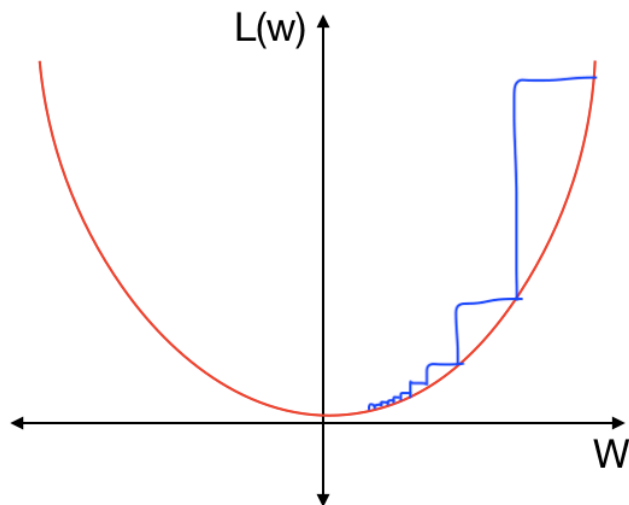
Adam is better choice!

学习率的影响

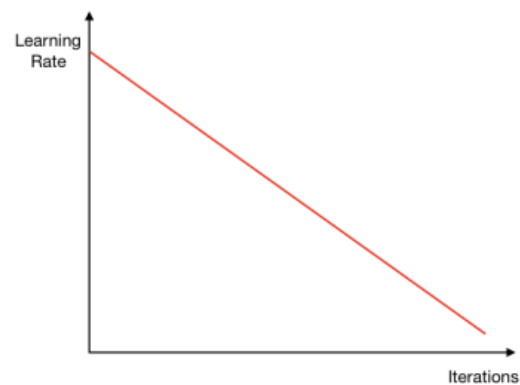


- 学习率设置过小，网络收敛缓慢，增大找到最优值的时间
- 合适的学习率使得网络快速收敛
- 学习率设置过大会导致网络不能收敛，在最优解附近徘徊

学习率衰减



梯级衰减 (step decay)



线性衰减 (Linear Decay)

学习率衰减



假设初始学习率为 α_0 ，在第 t 次迭代时的学习率 α_t 。常用的衰减方式可以设置为按迭代次数进行衰减。比如逆时衰减（inverse time decay）

$$\alpha_t = \alpha_0 \frac{1}{1 + \beta \times t}, \quad (7.5)$$

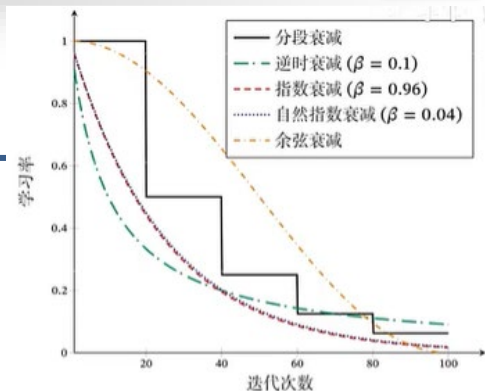
或指数衰减（exponential decay）

$$\alpha_t = \alpha_0 \beta^t, \quad (7.6)$$

或自然指数衰减（natural exponential decay）

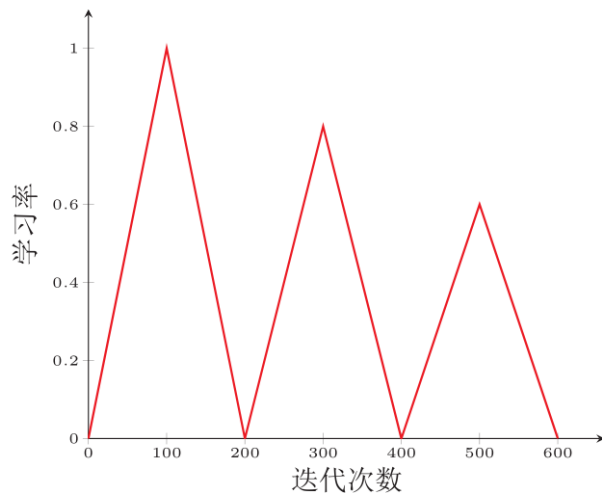
$$\alpha_t = \alpha_0 \exp(-\beta \times t), \quad (7.7)$$

其中 β 为衰减率，一般取值为0.96。

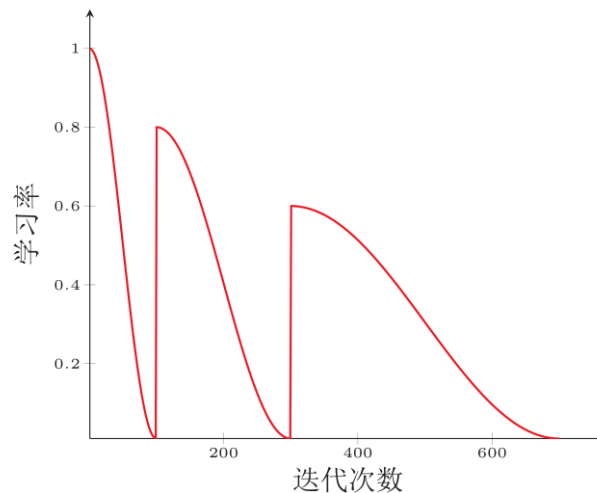


周期性学习率调整

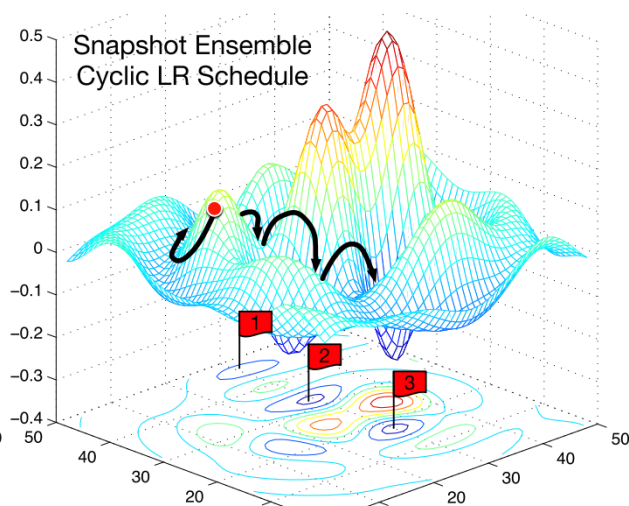
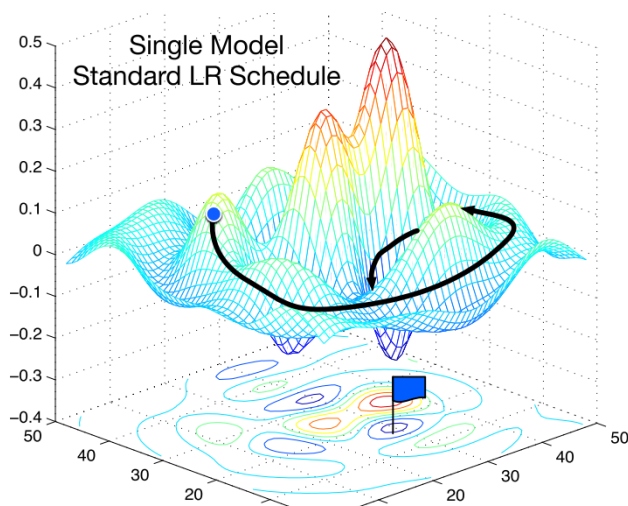
Cyclical Learning Rates



(a) 三角循环学习率



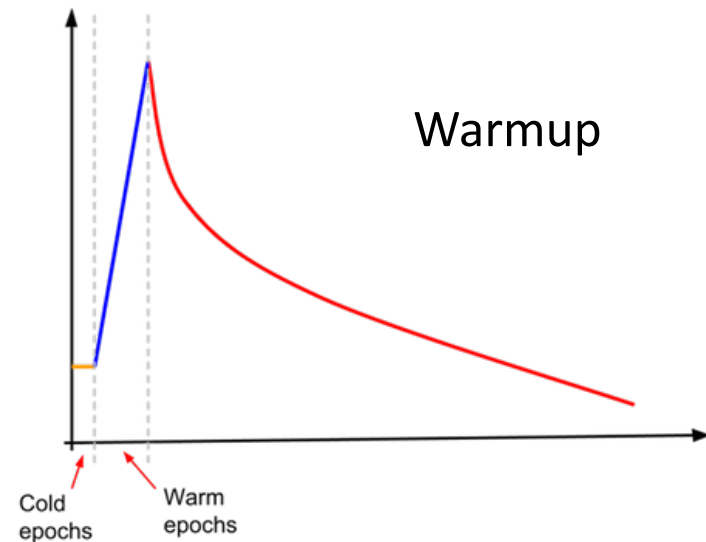
(b) 带热重启的余弦衰减



Others



- Don't Decay the Learning Rate, Increase the Batch Size
<https://openreview.net/pdf?id=B1Yy1BxCZ>
- Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour Warmup
<https://arxiv.org/abs/1706.02677>



自适应学习率



Adagrad

RMSprop

Adadelta

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

$$G_t = \sum_{\tau=1}^t \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau}$$

$$\begin{aligned} G_t &= \beta G_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t \\ &= (1 - \beta) \sum_{\tau=1}^t \beta^{t-\tau} \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau} \end{aligned}$$

$$\Delta\theta_t = -\frac{\sqrt{\Delta X_{t-1}^2 + \epsilon}}{\sqrt{G_t + \epsilon}} \mathbf{g}_t$$

$$\Delta X_{t-1}^2 = \beta_1 \Delta X_{t-2}^2 + (1 - \beta_1) \Delta\theta_{t-1} \odot \Delta\theta_{t-1}$$

梯度方向优化



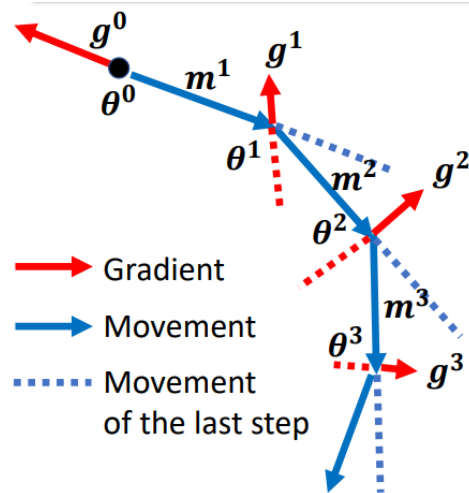
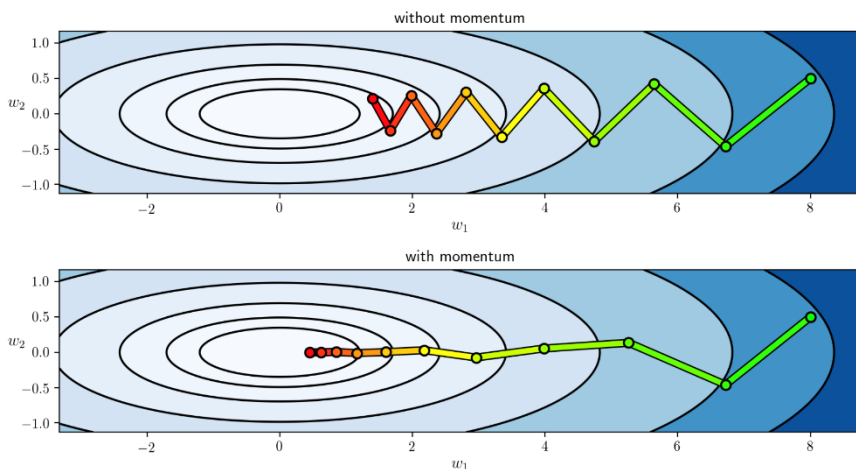
动量法 (Momentum Method)

- 用之前积累动量来替代真正的梯度。每次迭代的梯度可以看作是加速度。

在第 t 次迭代时, 计算负梯度的“加权移动平均”作为参数的更新方向,

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t, = -\alpha \sum_{\tau=1}^t \beta^{t-\tau} \mathbf{g}_\tau, \quad (7.15)$$

其中 ρ 为动量因子, 通常设为 0.9, α 为学习率。



梯度方向优化



Nesterov加速梯度

- Nesterov加速梯度是一种对动量法的改进

在动量法中,实际的参数更新方向 $\Delta\theta_t$ 为上一步的参数更新方向 $\Delta\theta_{t-1}$ 和当前梯度的反方向 $-\mathbf{g}_t$ 的叠加. 这样, $\Delta\theta_t$ 可以被拆分为两步进行,先根据 $\Delta\theta_{t-1}$ 更新一次得到参数 $\hat{\theta}$,再用 $-\mathbf{g}_t$ 进行更新.

$$\hat{\theta} = \theta_{t-1} + \rho\Delta\theta_{t-1}, \quad (7.22)$$

$$\theta_t = \hat{\theta} - \alpha\mathbf{g}_t, \quad (7.23)$$

其中梯度 $\mathbf{g}_t$ 为点 θ_{t-1} 上的梯度,因此在第二步更新中有些不太合理. 更合理的更新方向应该为 $\hat{\theta}$ 上的梯度.

这样,合并后的更新方向为

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t(\theta_{t-1} + \rho\Delta\theta_{t-1}), \quad (7.24)$$

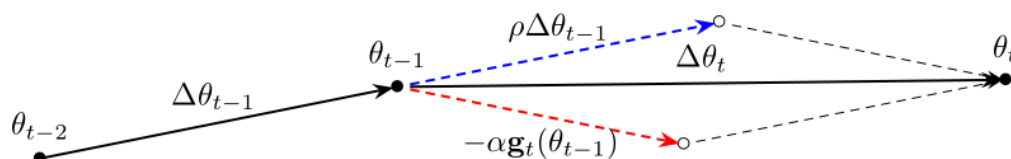
梯度方向优化



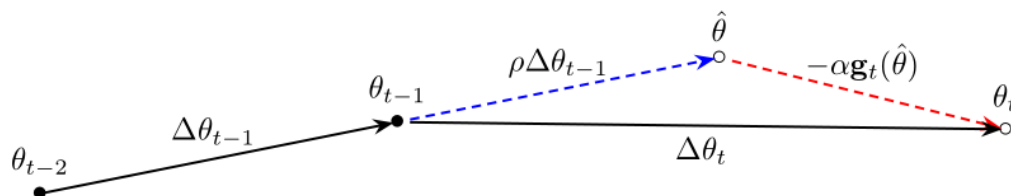
Nesterov加速梯度

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t(\theta_{t-1} + \rho\Delta\theta_{t-1})$$

● 动量法和Nesterov加速度梯度比较



(a) 动量法



(b) Nesterov 加速梯度

梯度方向优化+自适应学习率



Adam算法 $\approx$ 动量法(Momentum)+RMSprop

- 先计算两个移动平均

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) \mathbf{g}_t,$$
$$G_t = \beta_2 G_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t,$$

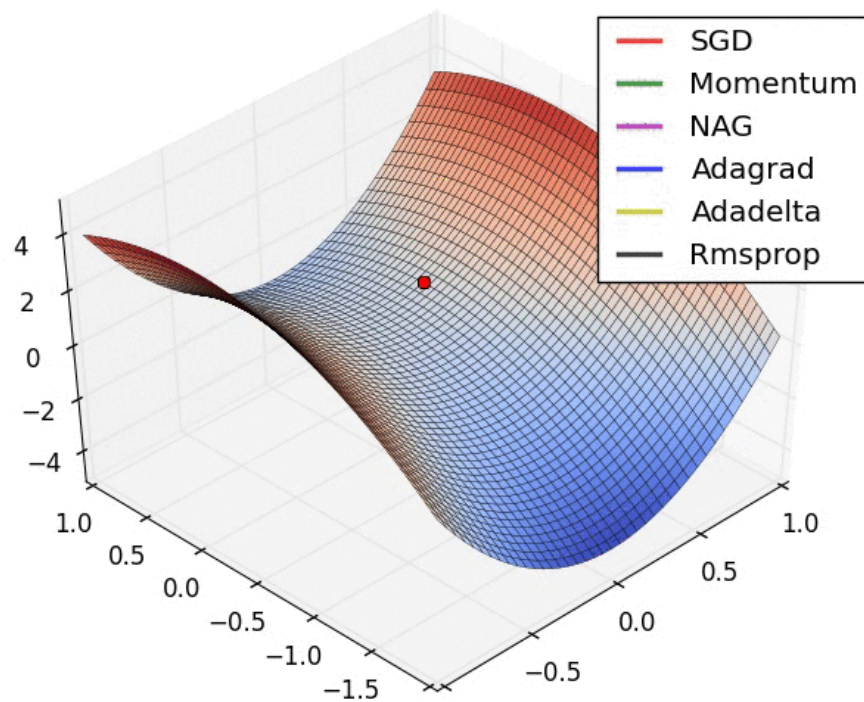
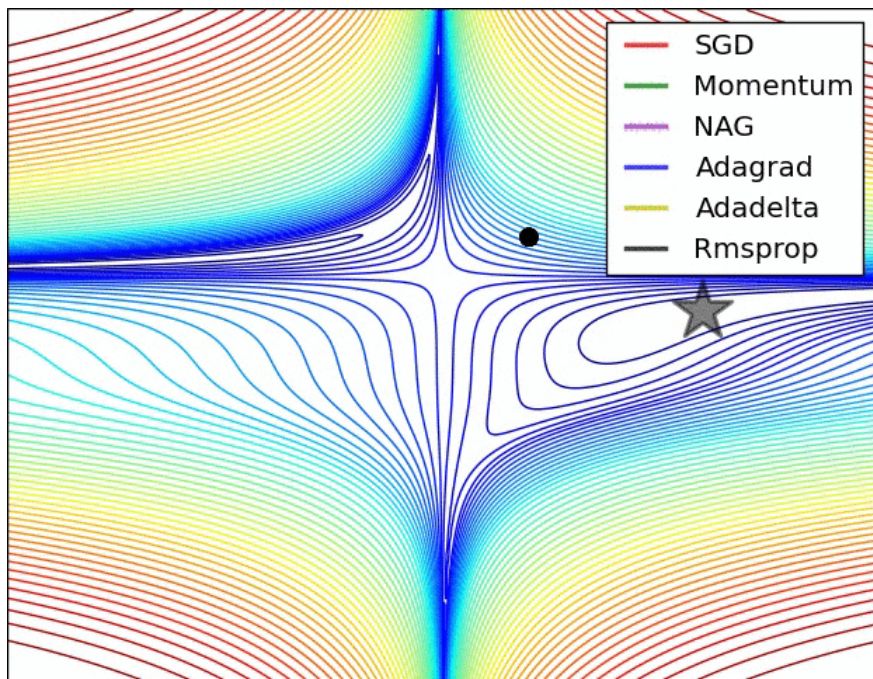
- 偏差修正

$$\hat{M}_t = \frac{M_t}{1 - \beta_1^t},$$
$$\hat{G}_t = \frac{G_t}{1 - \beta_2^t}.$$

- 更新

$$\Delta \theta_t = -\frac{\alpha}{\sqrt{\hat{G}_t + \epsilon}} \hat{M}_t$$

优化



鞍点 (Saddle point)

梯度截断



梯度截断是一种比较简单的启发式方法，把梯度的模限定在一个区间，当梯度的模小于或大于这个区间时就进行截断。

● 按值截断

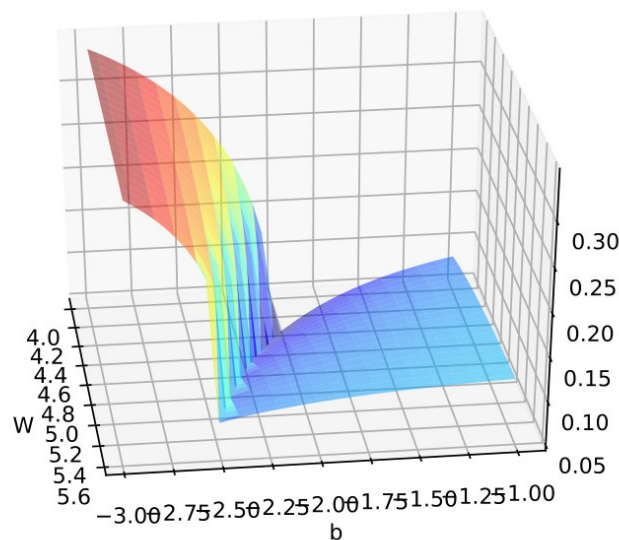
- 在第 t 次迭代时，梯度为 $\mathbf{g}_t$ ，给定一个区间 $[a, b]$ ，如果一个参数的梯度小于 a 时，就将其设置为 a ，如果大于 b 时，就将其设置为 b 。

$$\mathbf{g}_t = \max(\min(\mathbf{g}_t, b), a)$$

● 按模截断

- 按模截断是将梯度的模截断到一个给定的截断阈值 b 。如果 $\|\mathbf{g}_t\|^2 \leq b$ ，保持 $\mathbf{g}_t$ 不变，如果 $\|\mathbf{g}_t\|^2 > b$ ，则令

$$\mathbf{g}_t = \frac{b}{\|\mathbf{g}_t\|^2} \mathbf{g}_t$$



优化算法改进小结



大部分优化算法可以使用下面公式来统一描述概括：

$$\Delta\theta_t = -\frac{\alpha_t}{\sqrt{G_t + \epsilon}} M_t,$$

$\mathbf{g}_t$ 为第 t 步的梯度

$$G_t = \psi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

α_t 为第 t 步的学习率

$$M_t = \phi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

类别		优化算法
学习率调整	固定衰减学习率	分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减
	周期性学习率	循环学习率、SGDR
	自适应学习率	AdaGrad、RMSprop、AdaDelta
梯度估计修正		动量法、Nesterov 加速梯度、梯度截断
综合方法		Adam $\approx$ 动量法 + RMSprop

其中 $\mathbf{g}_t$ 是第 t 步的梯度； α_t 是第 t 步的学习率，可以进行衰减，也可以不变； $\psi(\cdot)$ 是学习率缩放函数，可以取1 或历史梯度的模的移动平均； $\phi(\cdot)$ 是优化后的参数更新方向，可以取当前的梯度 $\mathbf{g}_t$ 或历史梯度的移动平均。

参数初始化



- 参数不能初始化为0！为什么？ 对称权重问题！

- 对称权重现象：在线性模型的训练（比如感知机和Logistic）回归中，我们一般将参数全部初始化为0。但是这在神经网络中可能会出现问题。如果参数都为0，在第一遍前向计算时，所有的隐藏层神经元的激活值都相同；在反向传播时，所有权重的更新也都相同，这样会导致隐藏层神经元没有区分性。这种现象也成为对称权重现象。

- 初始化方法

- 预训练初始化：一个已经在大规模数据上的模型可以提供一个好的参数初始值
- 随机初始化：对每个参数随机初始化
- 固定值初始化：偏置（Bias）通常用0来初始化

随机初始化



- **Gaussian分布初始化**

Gaussian初始化方法是最简单的初始化方法，参数从一个固定均值（比如0）和固定方差（比如0.01）的Gaussian分布进行随机初始化。

- **均匀分布初始化**

参数可以在区间 $[-r, r]$ 内采用均匀分布进行初始化。

范数保持性 (Norm-Preserving)

一个 M 层的等宽线性网络

$$\mathbf{y} = \mathbf{W}^{(L)} \mathbf{W}^{(L-1)} \dots \mathbf{W}^{(1)} \mathbf{x}$$

为了避免梯度消失或梯度爆炸问题，我们希望误差项

$$\|\delta^{(l-1)}\|^2 = \|\delta^{(l)}\|^2 = \|(\mathbf{W}^{(l)})^\top \delta^{(l)}\|^2$$

基于方差缩放的参数初始化

正交初始化

参数初始化



● 基于方差缩放的参数初始化

Xavier 初始化和 He 初始化

初始化方法	激活函数	均匀分布 $[-r, r]$	高斯分布 $\mathcal{N}(0, \sigma^2)$
Xavier 初始化	Logistic	$r = 4\sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = 16 \times \frac{2}{M_{l-1}+M_l}$
Xavier 初始化	Tanh	$r = \sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = \frac{2}{M_{l-1}+M_l}$
He 初始化	ReLU	$r = \sqrt{\frac{6}{M_{l-1}}}$	$\sigma^2 = \frac{2}{M_{l-1}}$

● 正交初始化

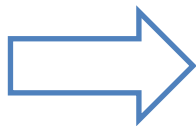
- 1) 用均值为 0、方差为 1 的高斯分布初始化一个矩阵;
- 2) 将这个矩阵用奇异值分解得到两个正交矩阵, 并使用其中之一作为权重矩阵。

细致平稳条件

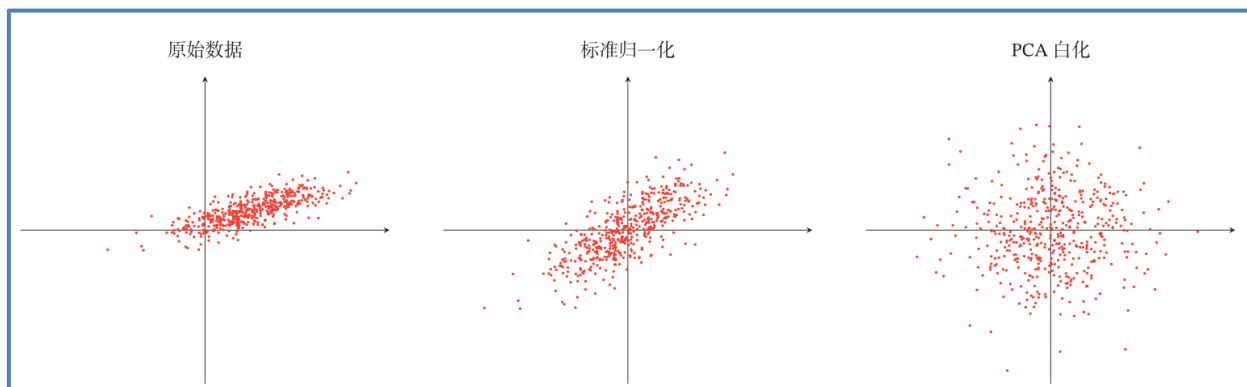


数据归一化

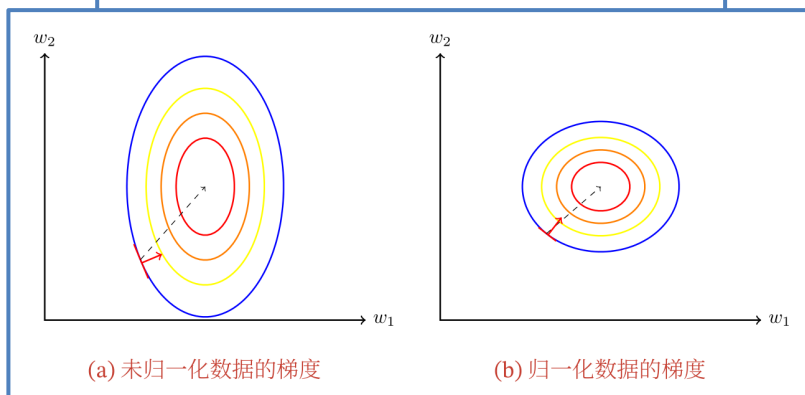
- 最小最大值归一化
- 标准化
- PCA



$$\mu = \frac{1}{N} \sum_{n=1}^N x^{(n)},$$
$$\sigma^2 = \frac{1}{N} \sum_{n=1}^N (x^{(n)} - \mu)^2.$$



数据归一化对梯度的影响



(a) 未归一化数据的梯度

(b) 归一化数据的梯度

逐层归一化



目的

- 更好的尺度不变性 内部协变量偏移
- 更平滑的优化地形

归一化方法

- 批量归一化 (Batch Normalization, BN)
- 层归一化 (Layer Normalization)
- 权重归一化 (Weight Normalization)
- 局部响应归一化 (Local Response Normalization, LRN)

批量归一化



对于一个深层神经网络，令第 l 层的净输入为 $\mathbf{z}^{(l)}$ ，神经元的输出为 $\mathbf{a}^{(l)}$ ，即

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}) = f(W\mathbf{a}^{(l-1)} + \mathbf{b}), \quad (7.42)$$

其中 $f(\cdot)$ 是激活函数， W 和 $\mathbf{b}$ 是可学习的参数。

- 给定一个包含 K 个样本的小批量样本集合，计算均值和方差

$$\begin{aligned} \mu_{\mathcal{B}} &= \frac{1}{K} \sum_{k=1}^K \mathbf{z}^{(k,l)}, \\ \sigma_{\mathcal{B}}^2 &= \frac{1}{K} \sum_{k=1}^K (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}) \odot (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}). \end{aligned}$$

- 批量归一化

$$\begin{aligned} \hat{\mathbf{z}}^{(l)} &= \frac{\mathbf{z}^{(l)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \odot \gamma + \beta \\ &\triangleq \text{BN}_{\gamma, \beta}(\mathbf{z}^{(l)}), \end{aligned}$$

层归一化



- 第 l 层神经元的净输入为 $\mathbf{z}^{(l)}$

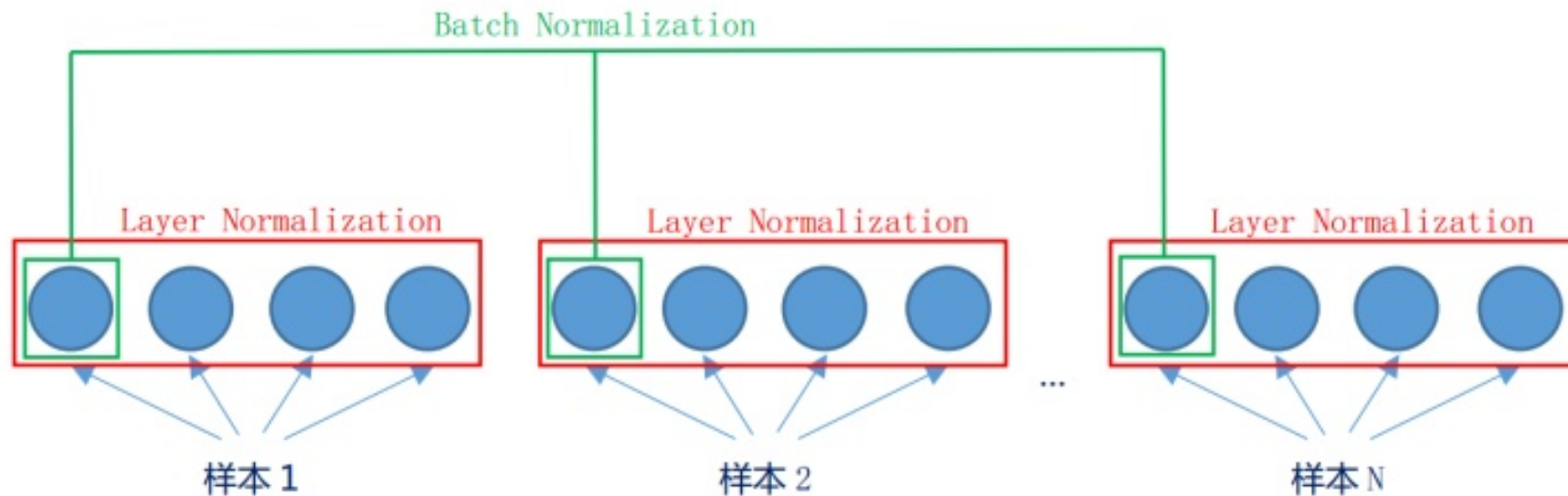
$$\mu^{(l)} = \frac{1}{n^l} \sum_{i=1}^{n^l} z_i^{(l)},$$

$$\sigma^{(l)2} = \frac{1}{n^l} \sum_{i=1}^{n^l} (z_i^{(l)} - \mu^{(l)})^2,$$

- 层归一化定义为

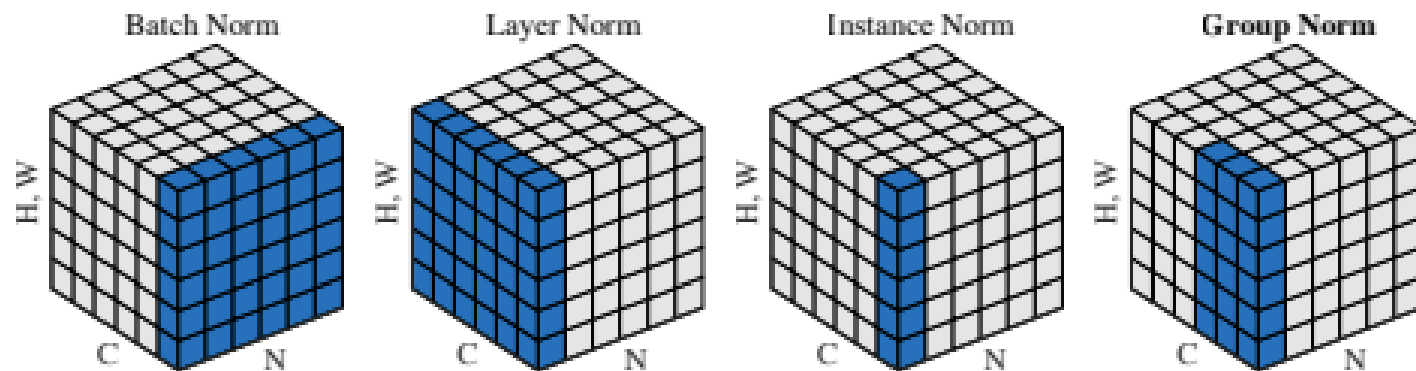
$$\begin{aligned} \hat{\mathbf{z}}^{(l)} &= \frac{\mathbf{z}^{(l)} - \mu^{(l)}}{\sqrt{\sigma^{(l)2} + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta} \\ &\triangleq \text{LN}_{\boldsymbol{\gamma}, \boldsymbol{\beta}}(\mathbf{z}^{(l)}), \end{aligned}$$

批量归一化VS层归一化



- 层归一化和批量归一化整体上十分类似，差别在于归一化的方法不同。
- 对于K个样本的小批量集合 $Z^{(l)} = [z^{(1,l)}; \dots; z^{(K,l)}]$:
 - 1) 层归一化是对矩阵 $Z^{(l)}$ 的每一列进行归一化;
 - 2) 批量归一化是对每一行进行归一化。
- 批量归一化是一种更好的选择，当小批量样本数量比较少时，可以选择层归一化。

更多的归一化



超参数优化



超参数

- 层数
- 每层神经元个数
- 激活函数
- 学习率（以及动态调整算法）
- 正则化系数
- mini-batch 大小



优化方法

- 网格搜索
- 随机搜索
- 贝叶斯优化
- 动态资源分配
- 神经架构搜索



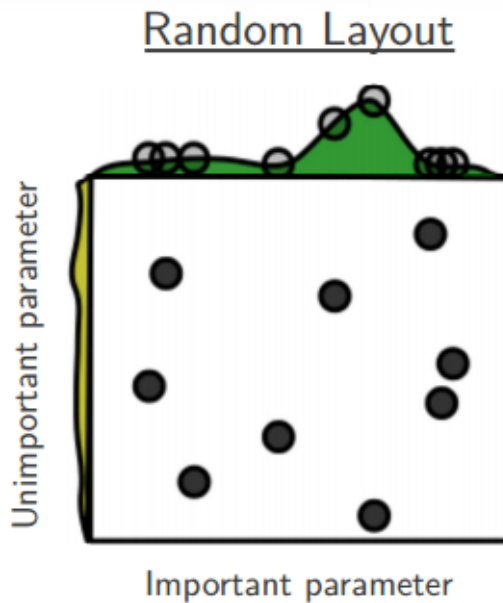
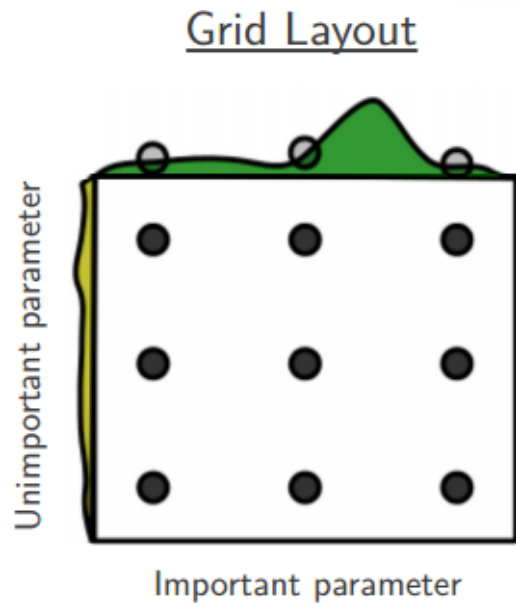
网格搜索

- 假设总共有K个超参数，第k个超参数的可以取 m_k 个值
- 如果参数是连续的，可以将参数离散化，选择几个“经验”值。比如学习率 α ，我们可以设置

$$\alpha \in \{0.01, 0.1, 0.5, 1.0\}$$

- 这些超参数可以有 $m_1 \times m_2 \times \dots \times m_k$ 个取值组合。

超参数优化



Grid and random search of nine trials for optimizing a function $f(x,y) = g(x) + h(y) \approx g(x)$ with low effective dimensionality. Above each square $g(x)$ is shown in green, and left of each square $h(y)$ is shown in yellow. With grid search, nine trials only test $g(x)$ in three distinct places. With random search, all nine trials explore distinct values of g . This failure of grid search is the rule rather than the exception in high dimensional hyper-parameter optimization.

重新思考泛化性



- 神经网络
 - 过度参数化
 - 拟合能力强

↓
~~泛化性差~~



Zhang C, Bengio S, Hardt M, et al. Understanding deep learning requires rethinking generalization[J]. arXiv preprint arXiv:1611.03530, 2016.

正则化

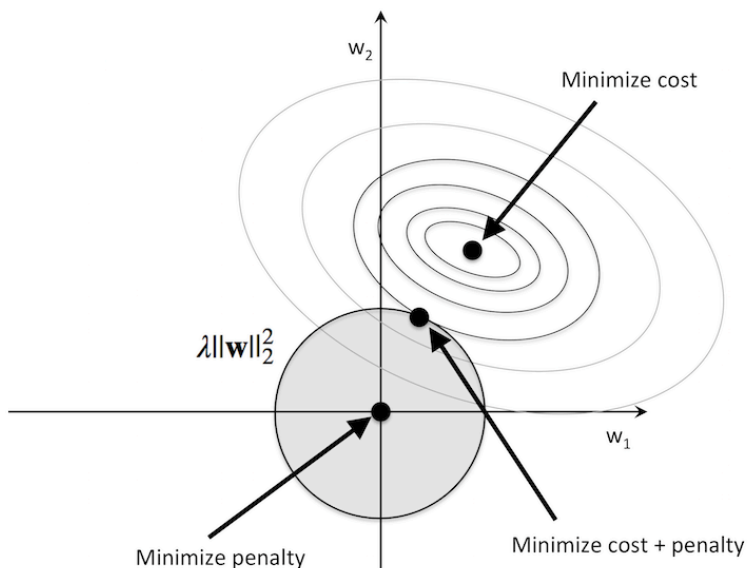
Regularization



所有损害优化的方法都是正则化。

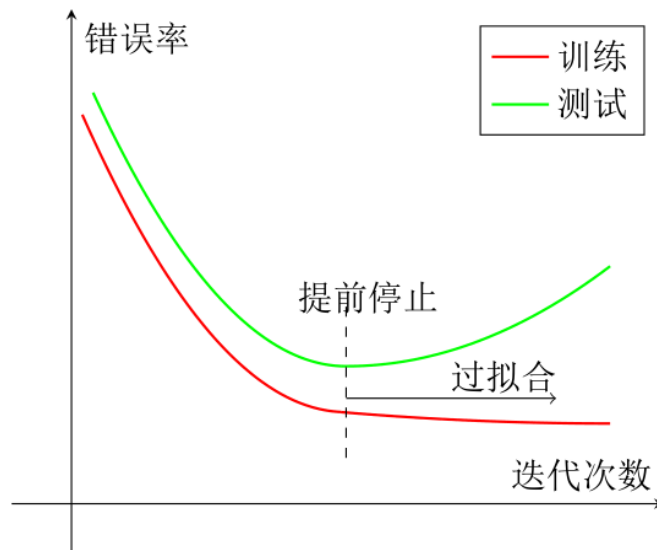
增加优化约束

L1/L2约束、数据增强



干扰优化过程

权重衰减、随机梯度下降、提前停止



正则化



💡 如何提高神经网络的泛化能力？

1. 增加优化约束

数据增强

标签平滑

L1/L2 正则化

2. 干扰优化过程

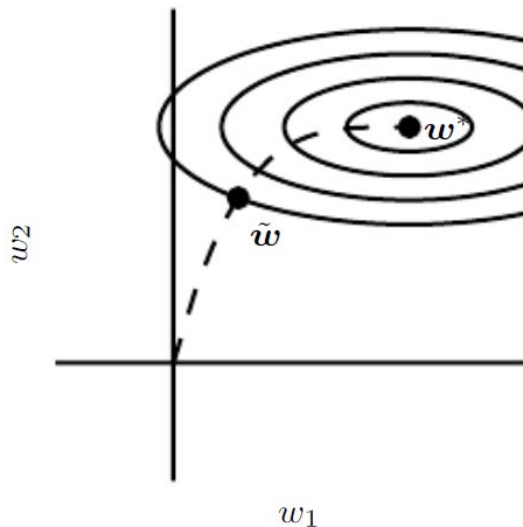
提前停止 early stop

权重衰减

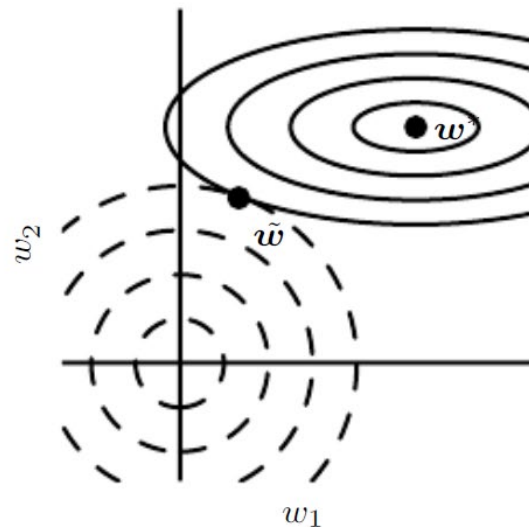
SGD

Dropout

提前停止



L2 正则化



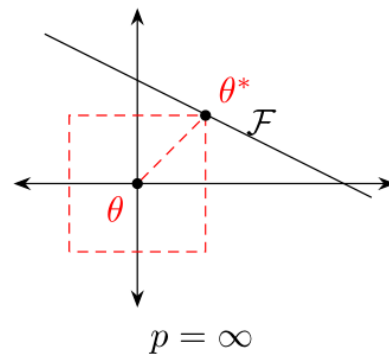
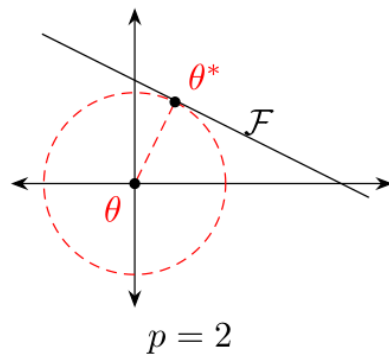
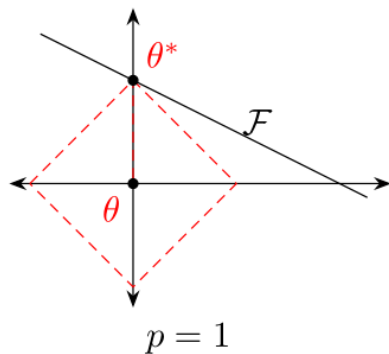
l_1 和 l_2 正则化



- 优化问题可以写为

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}, \theta)) + \lambda \ell_p(\theta)$$

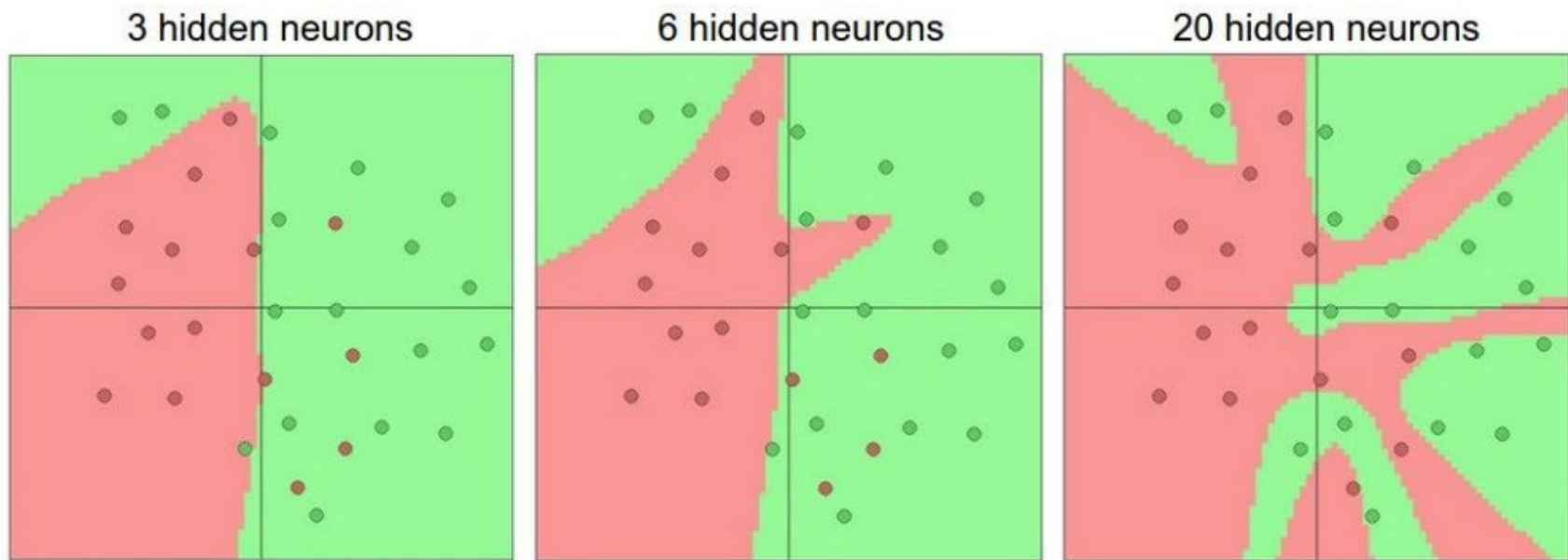
- l_p 为范数函数， p 的取值通常为 $\{1, 2\}$ 代表 l_1 和 l_2 的范数， λ 为正则化系数。



神经网络示例



隐藏层的不同神经元个数

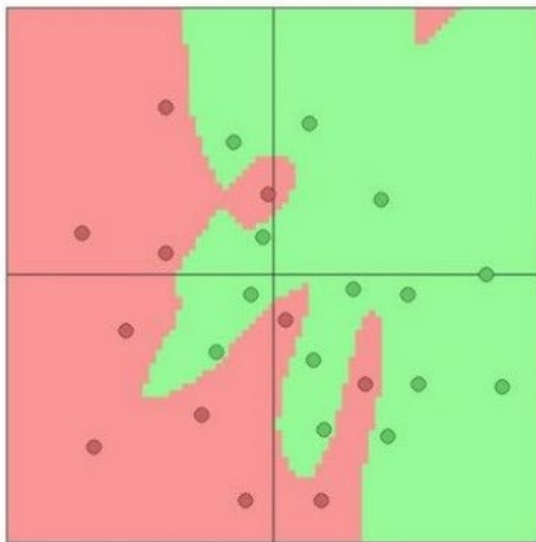


神经网络示例

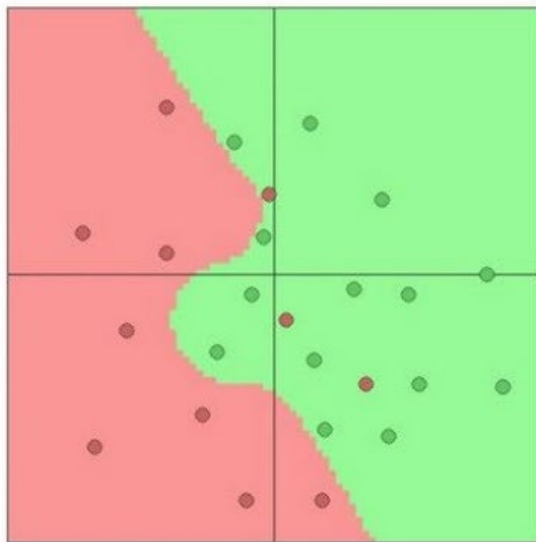


不同的正则化系数

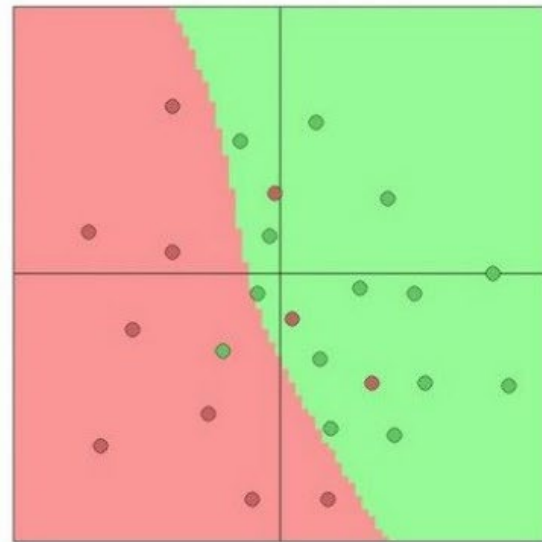
$\lambda = 0.001$



$\lambda = 0.01$



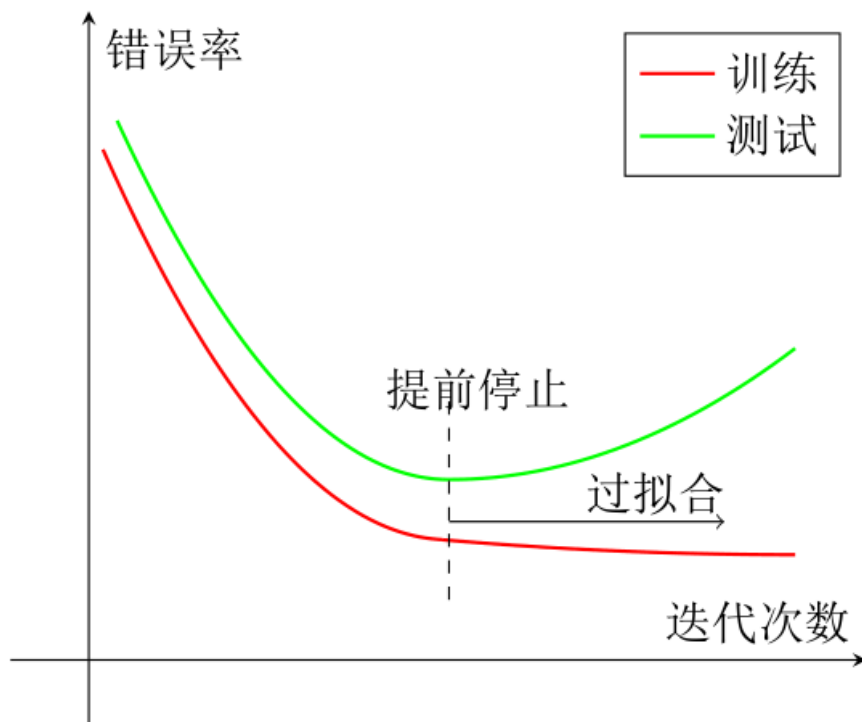
$\lambda = 0.1$



提前停止



- 我们使用一个验证集（Validation Dataset）来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降，就停止迭代。





- 在每次参数更新时，引入一个衰减系数 w 。

$$\theta_t \leftarrow (1 - w)\theta_{t-1} - \alpha \mathbf{g}_t$$

- 在标准的随机梯度下降中，权重衰减正则化和 l_2 正则化的效果相同。
- 在较为复杂的优化方法（如Adam）中，权重衰减和 l_2 正则化并不等价。

丢弃法

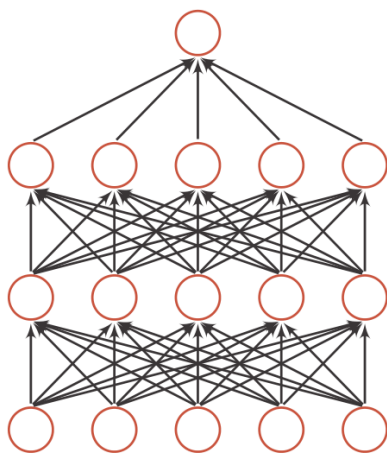
Dropout Method



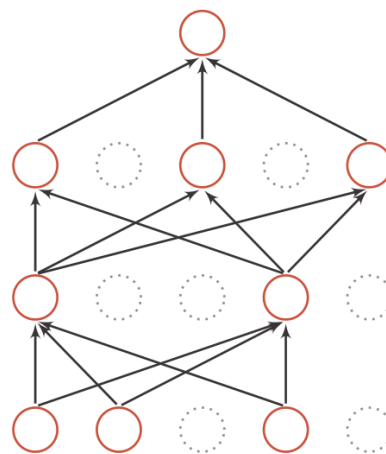
- 对于一个神经层 $y=f(Wx+b)$ ，引入一个丢弃函数 $d(\cdot)$ 使得 $y=f(Wd(x)+b)$ 。

$$d(\mathbf{x}) = \begin{cases} \mathbf{m} \odot \mathbf{x} & \text{当训练阶段时} \\ p\mathbf{x} & \text{当测试阶段时} \end{cases}$$

- 其中 $\mathbf{m} \in \{0,1\}^d$ 是丢弃掩码（dropout mask），通过以概率为 p 的贝努力分布随机生成。



(a) 标准网络



(b) Dropout 后的网络

Dropout意义



集成学习的解释

每做一次丢弃，相当于从原始的网络中采样得到一个子网络。如果一个神经网络有 n 个神经元，那么总共可以采样出 2^n 个子网络。

贝叶斯学习的解释

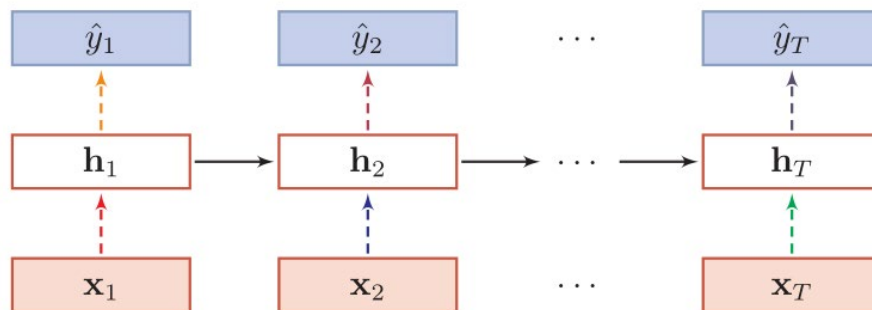
其中 $f(\mathbf{x}, \theta_m)$ 为第 m 次应用丢弃方法后的网络。

$$\begin{aligned}\mathbb{E}_{q(\theta)}[y] &= \int_{\mathcal{Q}} f(\mathbf{x}, \theta) q(\theta) d\theta \\ &\approx \frac{1}{M} \sum_{m=1}^M f(\mathbf{x}, \theta_m),\end{aligned}$$

循环神经网络上的丢弃法



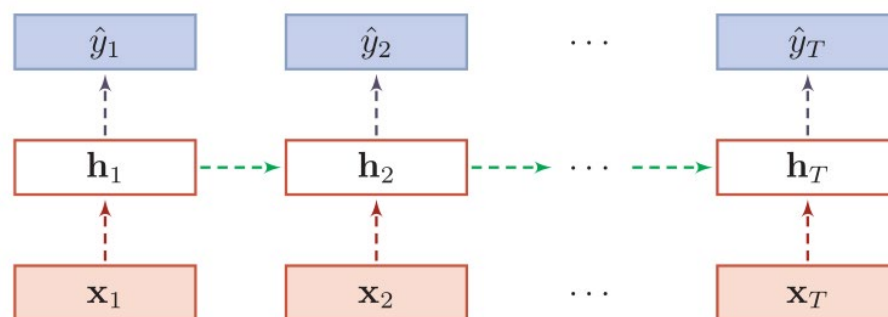
- 当在循环神经网络上应用丢弃法，不能直接对每个时刻的隐状态进行随机丢弃，这样会损害循环网络在时间维度上记忆能力。



虚线边表示进行随机丢弃，不同的颜色表示不同的丢弃掩码。

变分Dropout

- 根据贝叶斯学习的解释，丢弃法是一种对参数 θ 的采样。每次采样的参数需要在每个时刻保持不变。因此，对循环神经网络上使用丢弃法，需要对参数矩阵的每个元素进行随机丢弃，并在所有时刻都使用相同的丢弃掩码。





- 图像数据的增强主要是通过算法对图像进行转变，引入噪声等方法来增加数据的多样性。
- 图像数据的增强方法：
 - 旋转(Rotation): 将图像按顺时针或逆时针方向随机旋转一定角度；
 - 翻转(Flip): 将图像沿水平或垂直方法随机翻转一定角度；
 - 缩放(Zoom In/Out): 将图像放大或缩小一定比例；
 - 平移(Shift): 将图像沿水平或垂直方法平移一定步长；
 - 加噪声(Noise): 加入随机噪声。



原始图像



数据增强



- 在输出标签中添加噪声来避免模型过拟合。
- 一个样本 $\mathbf{x}$ 的标签一般用onehot向量表示

$$\mathbf{y} = [0, \dots, 0, 1, 0, \dots, 0]^T \quad \text{硬目标 (Hard Targets)}$$

- 引入一个噪声对标签进行平滑，即假设样本以 ϵ 的概率为其它类。平滑后的标签为

$$\tilde{\mathbf{y}} = \left[\frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}, 1 - \epsilon, \frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1} \right]^T$$

总结



模型

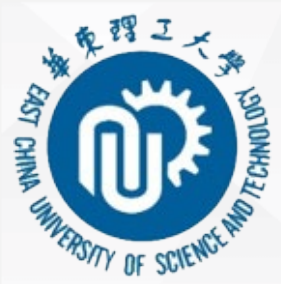
- 用ReLU作为激活函数
- 分类时用交叉熵作为损失函数
- 逐层归一化

优化

- SDG+mini-batch（动态学习率、Adam算法优先）
- 每次迭代都重新随机排序
- 数据预处理（标准归一化）
- 参数初始化

正则化

- l_1 和 l_2 正则化（跳过前几轮）
- Dropout
- Early-stop
- 数据增强



谢谢！

THANK YOU FOR LISTENING

